

文件名: run_example.sh

功能: 命令行运行入口, 启动配置检查与完整处理流程。

```
#!/usr/bin/env bash
set -euo pipefail
5 cd -- "$(dirname -- "${BASH_SOURCE[0]}")"
  exec "${EGOSMPLX_CONTROLLER_PYTHON:-python3}" -m egosmplx_pipeline "${1:-check}" --con
  fig "${EGOSMPLX_CONFIG_FILE:-configs/pipeline.example.json}" "${@:2}"
```

文件名: egosmplx_pipeline/__init__.py

功能: EgoSmplx: automatic fisheye body fitting and fixed-rule MANO fusion.

```
10 """EgoSmplx: automatic fisheye body fitting and fixed-rule MANO fusion."""
   __version__ = "1.0.0"
```

文件名: egosmplx_pipeline/__main__.py

功能: Command-line interface: check, run, sample.

```
"""Command-line interface: check, run, sample."""
15 import argparse
   from pathlib import Path
   from egosmplx_pipeline.configuration import load_config, preflight
   from egosmplx_pipeline.io import write
   def main():
20     parser = argparse.ArgumentParser(description='EgoSmplx V1.0 automatic fisheye pose pipeline')
       sub = parser.add_subparsers(dest='command', required=True)
       for name in ['check', 'run']:
           cmd = sub.add_parser(name)
           cmd.add_argument('--config', type=Path, required=True)
25     cmd.add_argument('--cached-predictions', action='store_true')
           if name == 'run':
               cmd.add_argument('--resume', action='store_true')
           sample = sub.add_parser('sample')
           sample.add_argument('--video', type=Path, required=True)
30     sample.add_argument('--calibration', type=Path, required=True)
           sample.add_argument('--camera', required=True)
           sample.add_argument('--count', type=int, default=4)
           sample.add_argument('--output', type=Path, required=True)
           args = parser.parse_args()
35     if args.command == 'check':
           config, rows = load_config(args.config)
           print(preflight(config, rows, args.cached_predictions))
       elif args.command == 'run':
           from egosmplx_pipeline.pipeline import run
40     print(run(args.config, args.cached_predictions, args.resume))
       else:
           from egosmplx_pipeline.sampling import sample_video
           print(sample_video(args.video, args.calibration, args.camera, args.count, args.output))
       if __name__ == '__main__':
45     main()
文件名: egosmplx_pipeline/configuration.py
功能: Portable JSON configuration and fail-fast input checks.
"""Portable JSON configuration and fail-fast input checks."""
import os
50 import re
```

```

from pathlib import Path
from egosmplx_pipeline.io import read, sha
from egosmplx_pipeline.profiles import selected
PACKAGE_ROOT = Path(__file__).resolve().parents[1]
5 def expand(value, base):
    if isinstance(value, dict):
        return {k: expand(v, base) for k, v in value.items()}
    if isinstance(value, list):
        return [expand(v, base) for v in value]
10 if isinstance(value, str):
    value = value.replace('${PACKAGE_ROOT}', str(PACKAGE_ROOT))
    value = os.path.expandvars(value)
    if re.search(r'\${[^}]+\}', value):
        raise ValueError('Unresolved environment variable: ' + value)
15 return value
def absolute(path, base):
    p = Path(path).expanduser()
    return str((base / p).resolve() if not p.is_absolute() else p.resolve())
def load_config(path):
20 path = Path(path).resolve()
    config = expand(read(path), path.parent)
    config['pipeline_profile'] = selected(config)
    for key, value in config['paths'].items():
        if value:
25         config['paths'][key] = absolute(value, path.parent)
    config['manifest'] = absolute(config['manifest'], path.parent)
    config['output'] = absolute(config['output'], path.parent)
    fit = config.setdefault('fitting', {})
    for key, default in [(('rigid_steps', 1800), ('pose_steps', 1600), ('mano_guided_steps', 1400))]:
30         fit.setdefault(key, default)
    if 'silhouette_weight' in fit:
        raise ValueError('The reference profile applies contours after wrist refinement; remove fitting.silhouette_weight')
    if any(not isinstance(fit[k], int) or isinstance(fit[k], bool) or fit[k] < 1 for k in ['rigid_steps', 'pose_steps', 'mano_guided_steps']):
35         raise ValueError('Invalid fitting settings')
    manifest = read(config['manifest'])
    frames = expand(manifest['frames'], Path(config['manifest']).parent)
    ids = set()
40 for row in frames:
    identifier = row['id']
    if not re.fullmatch(r'[A-Za-z0-9_-]+', identifier) or identifier in ids:
        raise ValueError('Frame IDs must be unique safe filenames: ' + identifier)
    ids.add(identifier)
45 for key in ['image', 'calibration']:
    row[key] = absolute(row[key], Path(config['manifest']).parent)
    for key, value in row.get('predictions', {}).items():
        row['predictions'][key] = absolute(value, Path(config['manifest']).parent)
    if not frames:
50         raise ValueError('Empty input manifest')

```

```

    return config, frames
def preflight(config, frames, cached):
    """Check required files and image/observation provenance before any output."""
    import cv2
5   required = ['body_python', 'egosmplx_repo', 'egosmplx_config']
    if not cached:
        required += ['pose_python', 'wilor_python', 'sapiens_repo', 'wilor_repo',
                    'egosmplx_checkpoint', 'pose_checkpoint', 'seg_checkpoint',
                    'wilor_checkpoint', 'wilor_config', 'wilor_detector']
10   for key in required:
        if key not in config['paths'] or not Path(config['paths'][key]).exists():
            raise FileNotFoundError('Required configured path: ' + key)
    for row in frames:
        image = cv2.imread(row['image'])
15   if image is None:
        raise ValueError('Unreadable image: ' + row['image'])
    if image.shape[1::-1] != (1280, 720):
        raise ValueError('Reference profile requires 1280 x 720 images: ' + row['id'])
    calibration = read(row['calibration'])
20   if calibration['size'] != list(image.shape[1::-1]):
        raise ValueError('Image and calibration sizes differ: ' + row['id'])
    if cached:
        for key in ['egosmplx', 'sapiens', 'segmentation', 'wilor']:
            if not Path(row.get('predictions', {}).get(key, "")).exists() or key not in row.get('predictions', {}):
25             raise FileNotFoundError(row['id'] + ': missing cached prediction ' + key)
        points = read(row['predictions']['sapiens'])
        if points['image_sha256'] != sha(row['image']):
            raise ValueError('Stale Sapiens2 observation: ' + row['id'])
        for name in ['labels.npy', 'probabilities.npz']:
30             if not (Path(row['predictions']['segmentation']) / name).is_file():
                raise FileNotFoundError('Incomplete segmentation: ' + row['id'])
    if not cached:
        for name in ['pose', 'seg']:
            expected = config.get('checkpoint_sha256', {}).get(name)
35             if expected is None or not re.fullmatch('[a-f0-9]{64}', expected):
                raise ValueError('Configure checkpoint_sha256.' + name)
    return dict(passed=True, frame_count=len(frames), cached_predictions=cached,
                scope='paths, input sizes, cache image hashes; models are loaded strictly by workers')
文件名: egosmplx_pipeline/pipeline.py
40 功能: Sequential multi-environment controller with input hashing and verified resume.
    """Sequential multi-environment controller with input hashing and verified resume."""
    import copy
    import datetime
    import fcntl
45   import hashlib
    import os
    from pathlib import Path
    import shutil
    import subprocess
50   import sys

```

```

from egosmplx_pipeline.configuration import PACKAGE_ROOT, load_config, preflight
from egosmplx_pipeline.io import read, write, sha
from egosmplx_pipeline.profiles import selected
def command(args, log, cwd, environment):
5     print('RUN', log.name, flush=True)
    with log.open('w') as stream:
        process = subprocess.run(list(map(str, args)), cwd=str(cwd), env=environment,
                                stdout=stream, stderr=subprocess.STDOUT)
    if process.returncode:
10         raise RuntimeError('Worker failed; inspect ' + str(log))
def environment(config):
    result = os.environ.copy()
    result.update(OMP_NUM_THREADS='2', MKL_NUM_THREADS='2', PYTHONDONTWRITEBYT
ECODE='1',
15         CUDA_VISIBLE_DEVICES=str(config.get('gpu', '0')),
        EGOSMPLX_CONFIG=config['paths']['egosmplx_config'],
        EGOSMPLX_REPO_ROOT=config['paths']['egosmplx_repo'])
    result['PYTHONPATH'] = os.pathsep.join([str(PACKAGE_ROOT), config['paths'].get('wilor_repo', "")])
    return result
20 def observe(config, rows, output, env):
    """Invoke independent external networks. No manual annotation input."""
    p = config['paths']
    observations = output / 'observations'
    observations.mkdir(exist_ok=False)
25     staged = observations / 'images'
    staged.mkdir()
    observed_rows = copy.deepcopy(rows)
    for row in observed_rows:
        image = Path(row['image'])
30         target = staged / (row['id'] + image.suffix.lower())
        shutil.copy2(image, target)
        row['image'] = str(target)
    manifest = observations / 'manifest.json'
    write(manifest, dict(frames=observed_rows))
35     for row in observed_rows:
        directory = observations / row['id']
        directory.mkdir()
        command([p['body_python'], '-m', 'egosmplx_pipeline.egosmplx_predict', '--image-dir', staged, '--ima
ge-glob', Path(row['image']).name,
40         '--fisheye-calibration', row['calibration'], '--config-path', p['egosmplx_config'],
        '--checkpoint-path', p['egosmplx_checkpoint'], '--output-dir', directory / 'network',
        '--smplx-input-mode', 'det_undistort_contiguous', '--contiguous-roii-mode', 'aspect_pad'],
        directory / 'network.log', p['egosmplx_repo'], env)
    candidates = sorted((directory / 'network').rglob(row['id'] + '*_smplx.npz'))
45     # Some backend versions return the timestamp directory beside the request.
    if not candidates:
        candidates = sorted(directory.glob('network*/smplx/' + row['id'] + '*_smplx.npz'))
    if len(candidates) != 1:
        raise RuntimeError('Expected one wearer prediction, found ' + str(len(candidates)))
50     args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_pose', '--repo-root', p['sapiens_repo'],

```

```

        '--image', row['image'], '--output-dir', directory / 'sapiens',
        '--checkpoint', p['pose_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha256']['pose']]
    if p.get('sapiens_dependencies'):
        args += ['--dependencies', p['sapiens_dependencies']]
5    command(args, directory / 'sapiens.log', PACKAGE_ROOT, env)
    row['predictions'] = dict(egosmplx=str(candidates[0]), sapiens=str(directory / 'sapiens/keypoints.json')
)
    command([p['wilor_python'], '-m', 'egosmplx_pipeline.wilor_predict', '--manifest', manifest,
        '--output-dir', observations / 'wilor', '--checkpoint', p['wilor_checkpoint'],
10    '--config', p['wilor_config'], '--detector', p['wilor_detector']],
        observations / 'wilor.log', p['wilor_repo'], env)
    args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_seg', '--repo-root', p['sapiens_repo'],
        '--manifest', manifest, '--output-dir', observations / 'segmentation',
        '--checkpoint', p['seg_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha256']['seg']]
15    if p.get('sapiens_dependencies'):
        args += ['--dependencies', p['sapiens_dependencies']]
    command(args, observations / 'segmentation.log', PACKAGE_ROOT, env)
    for row in observed_rows:
        row['predictions'].update(wilor=str(observations / 'wilor/npz' / (row['id'] + '_wilor.npz')),
20        segmentation=str(observations / 'segmentation/frames' / row['id']))
    write(output / 'automatic_manifest.json', dict(frames=observed_rows))
    return observed_rows
def fingerprint(config, rows):
    import json
25    hashes = {}
    for row in rows:
        for key in ['image', 'calibration']:
            hashes[row['id'] + ':' + key] = sha(row[key])
        for key, value in row.get('predictions', {}).items():
30            if key == 'segmentation':
                for name in ['labels.npy', 'probabilities.npy']:
                    hashes[row['id'] + ':' + name] = sha(Path(value) / name)
            else:
                hashes[row['id'] + ':' + key] = sha(value)
35    code = {str(p.relative_to(PACKAGE_ROOT)): sha(p) for p in (PACKAGE_ROOT / 'egosmplx_pipeline'
).rglob('*.py'))
    return hashlib.sha256(json.dumps(dict(config=config, inputs=hashes, code=code), sort_keys=True).encode()).hexdigest()
def run(config_path, cached=False, resume=False):
40    config, rows = load_config(config_path)
    check = preflight(config, rows, cached)
    output = Path(config['output'])
    if output.exists() and not resume:
        raise FileExistsError('Output exists; choose a new directory or --resume')
45    output.mkdir(parents=True, exist_ok=True)
    with (output / '.lock').open('w') as lock:
        fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
        env = environment(config)
        write(output / 'preflight.json', check)
50    signature = fingerprint(config, rows)

```

```

state_path = output / 'run_state.json'
if state_path.exists() and read(state_path)['fingerprint'] != signature:
    raise ValueError('Inputs, code, or configuration changed; use a new output directory')
write(state_path, dict(fingerprint=signature, state='running', cached_predictions=cached))
5 if not cached:
    ready = output / 'automatic_manifest.json'
    if resume and ready.exists():
        rows = read(ready)['frames']
    else:
10         if (output / 'observations').exists():
            stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
            (output / 'observations').rename(output / ('incomplete_observations_' + stamp))
            rows = observe(config, rows, output, env)
records = []
15 for index, row in enumerate(rows):
    destination = output / 'frames' / row['id']
    validation = destination / 'validation.json'
    if resume and validation.exists():
        report = read(validation)
20     valid = report.get('passed') and all((destination / f).is_file() and sha(destination / f) == h
        for f, h in report['artifact_sha256'].items())
    if valid:
        records.append(report)
        print('VERIFIED_RESUME', row['id'], flush=True)
25     continue
    if destination.exists():
        stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
        destination.rename(destination.with_name(destination.name + '_incomplete_' + stamp))
    job_path = output / 'jobs' / (row['id'] + '.json')
30     write(job_path, dict(config=config, row=row, output=str(destination)))
    command([config['paths']['body_python'], '-m', 'egosmplx_pipeline.frame', '--job', job_path,
            output / (row['id'] + '.log'), PACKAGE_ROOT, env)
    records.append(read(validation))
    write(output / 'progress.json', dict(completed=index+1, total=len(rows), last_frame=row['id']))
35 if not all(r.get('pipeline_profile') == selected(config) for r in records):
    job_path = output / 'jobs/reference.json'
    write(job_path, dict(config=config, rows=rows, output=str(output)))
    command([config['paths']['body_python'], '-m', 'egosmplx_pipeline.reference.pipeline', '--job', job_p
ath],
40         output / 'reference.log', PACKAGE_ROOT, env)
    records = [read(output / 'frames' / row['id'] / 'validation.json') for row in rows]
    from egosmplx_pipeline.report import build
    build(output, records)
    write(state_path, dict(fingerprint=signature, state='complete', frame_count=len(records), cached_predi
45 ctions=cached))
    return output
文件名: egosmplx_pipeline/frame.py
功能: Execute the two-stage body fit and prepare baseline fusion for refinement.
"""Execute the two-stage body fit and prepare baseline fusion for refinement."""
50 import argparse

```

```

import json
from pathlib import Path
import sys
import cv2
5 import numpy as np
from egosmplx_pipeline.io import read, write, load, sha, rmse
def canonicalize(parameters, model, camera):
    from egosmplx_pipeline.fusion import geometry
    vertices, joints = geometry(parameters)
10 pixels = camera.world2camera(joints.astype(np.float64)).astype(np.float32)
    parameters.update(smplx_mesh_cam=vertices, smplx_joint_cam=joints, smplx_joint_img=pixels,
                      smplx_joint_cam_root_relative=joints-joints[:1],
                      smplx_body_joints_3d_local=(joints-parameters['transl'].reshape(3))[25],
                      smplx_body_joints_3d_render_cam=joints[25],
15                      smplx_body_joints_2d_render_cam=pixels[25])
    return parameters
def invoke(function, arguments):
    previous = sys.argv
    try:
20     sys.argv = [function.__module__] + list(map(str, arguments))
        function()
    finally:
        sys.argv = previous
def run(config, row, output):
25     from egosmplx_pipeline.bootstrap import setup
    model, Camera = setup(config['paths']['egosmplx_repo'])
    from egosmplx_pipeline import fusion, body_rigid, body_pose
    from egosmplx_pipeline.matching import assign
    from egosmplx_pipeline.render import render, write_obj, read_obj
30     fusion.initialize(model)
    camera = Camera(row['calibration'])
    image = cv2.imread(row['image'])
    if image is None or tuple(camera.img_size) != tuple(image.shape[1::-1]):
        raise ValueError('Image or calibration dimensions invalid')
35     output.mkdir(parents=True, exist_ok=False)
    predictions = row['predictions']
    targets = read(predictions['sapiens'])
    image_digest = sha(row['image'])
    if targets['image_sha256'] != image_digest:
40         raise ValueError('Automatic target image hash differs from input')
    raw = load(predictions['egosmplx'])
    for key in ['global_orient', 'transl']:
        if key + '_network' not in raw:
            raise ValueError('Raw prediction requires explicit original network field: ' + key)
45     raw[key] = raw[key + '_network'].copy()
    # Retain reconstruction inputs and original network rigid outputs only.
    keep = fusion.PARAMS + ['transl', 'global_orient_network', 'transl_network']
    raw = canonicalize({k: raw[k] for k in keep}, model, camera)
    raw_dir = output / 'raw'
50     raw_dir.mkdir()

```

```

np.savez_compressed(raw_dir / 'params.npz', **raw)
write_obj(raw_dir / 'mesh.obj', raw['smplx_mesh_cam'], model.face)
cv2.imwrite(str(raw_dir / 'wireframe.jpg'), render(image, raw['smplx_mesh_cam'].astype(np.float64), model.face, camera, .12))
5   common = ['--repo-root', config['paths']['egosmplx_repo'], '--image', row['image'], '--calibration', row['calibration']]
    invoke(body_rigid.main, common + ['--source-npz', raw_dir / 'params.npz', '--keypoints', predictions['sapiens'],
                                     '--output-dir', output / 'rigid', '--steps', config['fitting']['rigid_steps'], '--device', 'cpu'])
10  invoke(body_pose.main, common + ['--source-npz', output / 'rigid/optimized_params.npz',
                                     '--automatic-keypoints', predictions['sapiens'],
                                     '--output-dir', output / 'body_fit', '--steps', config['fitting']['pose_steps']])
    body = canonicalize(load(output / 'body_fit/automatic/refined_params.npz'), model, camera)
    body_dir = output / 'body'
15  body_dir.mkdir()
    np.savez_compressed(body_dir / 'params.npz', **body)
    write_obj(body_dir / 'mesh.obj', body['smplx_mesh_cam'], model.face)
    cv2.imwrite(str(body_dir / 'wireframe.jpg'), render(image, body['smplx_mesh_cam'].astype(np.float64), model.face, camera, .12))
20  hands = load(predictions['wilor'])
    if 'image_sha256' in hands and str(hands['image_sha256']) != image_digest:
        raise ValueError("WiLoR image hash differs from input")
    if 'focal_length_px' not in hands:
        if 'cached_wilor_focal_px' not in config:
25      raise ValueError("Legacy WiLoR cache requires an explicit verified focal length")
        hands['focal_length_px'] = np.array(config['cached_wilor_focal_px'])
        hands['image_size_wh'] = np.array(image.shape[1::-1])
    if not np.array_equal(hands['image_size_wh'], image.shape[1::-1]):
        raise ValueError("WiLoR image size mismatch")
30  lookup = {q['name']: q for q in targets['keypoints']}
    matches = assign(hands, lookup, body['smplx_joint_img'], *image.shape[1::-1])
    mesh, hand_reports = fusion.stitch(body, hands, matches, camera)
    fused = output / 'fused'
    fused.mkdir()
35  np.savez_compressed(fused / 'mesh.npz', **mesh)
    np.savez_compressed(fused / 'wilor_predictions.npz', **hands)
    write_obj(fused / 'mesh.obj', mesh['vertices_cam'], mesh['faces'])
    write(fused / 'recipe.json', dict(matches=matches, blend_rings=7, body_parameter_path='../body/params.npz',
40      manual_annotations_used=False, local_surface_optimizer=False,
      standard_smplx_parameter_only_reconstruction=False))
    cv2.imwrite(str(fused / 'wireframe.jpg'), render(image, mesh['vertices_cam'].astype(np.float64), mesh['faces'], camera, .12))
    cv2.imwrite(str(fused / 'skeleton.jpg'), fusion.draw_geometry(image, camera, mesh))
45  # Reload actual saved artifacts, including the fusion inputs and recipe.
    reloaded_body = load(body_dir / 'params.npz')
    v, j = fusion.geometry(reloaded_body)
    body_error = float(np.max(np.abs(v-reloaded_body['smplx_mesh_cam'])))
    rebuilt, _ = fusion.stitch(reloaded_body, load(fused / 'wilor_predictions.npz'), read(fused / 'recipe.json')['
50  matches'], camera)

```

```

stored = load(fused / 'mesh.npz')
fusion_error = float(np.max(np.abs(rebuilt['vertices_cam']-stored['vertices_cam'])))
obj_vertices, obj_faces = read_obj(fused / 'mesh.obj')
obj_error = float(np.max(np.abs(obj_vertices-stored['vertices_cam'])))
5 if max(body_error, fusion_error, obj_error) > 1e-5 or not np.array_equal(obj_faces, stored['faces']):
    raise RuntimeError('Saved geometry failed reload validation')
indices = body['stage2_selected_joint_indices'].astype(int)
truth = body['stage2_target_joint_pixels']
body_errors = np.linalg.norm(body['smplx_joint_img'][indices]-truth, axis=1)
10 hand_errors = []
for side, match in matches.items():
    k = fusion.SIDES.index(side)
    projected = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
    hand_errors.extend(np.linalg.norm(projected-hands['joints_2d'][match['candidate']], axis=1).tolist())
15 report = dict(id=row['id'], passed=True, matched_hands=len(matches), matches=matches,
    body_target_count=len(indices), body_error_px=body_errors.tolist(), hand_error_px=hand_errors,
rs,
    body_rmse_px=rmse(body['smplx_joint_img'][indices], truth),
    hand_rmse_px=float(np.sqrt(np.mean(np.square(hand_errors)))) if hand_errors else None,
20 body_reload_max_abs_m=body_error, fusion_reload_max_abs_m=fusion_error,
    obj_reload_max_abs_m=obj_error, minimum_fused_vertex_z_m=float(mesh['vertices_cam'][:,
2].min()),
    hand_reports=hand_reports, manual_annotations_used=False,
    accuracy_reference='automatic predictions, not human ground truth',
25 input_image_sha256=image_digest, calibration_sha256=sha(row['calibration']))
report['artifact_sha256'] = {str(p.relative_to(output)): sha(p) for p in output.rglob('*') if p.is_file()}
write(output / 'validation.json', report)
print('FRAME_COMPLETE', row['id'], report['body_rmse_px'], report['hand_rmse_px'], flush=True)
return report
30 def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--job', type=Path, required=True)
    args = parser.parse_args()
    job = read(args.job)
35 run(job['config'], job['row'], Path(job['output']))
if __name__ == '__main__':
    main()
文件名: egosmplx_pipeline/bootstrap.py
功能: Initialize the separately installed EgoSMPLX backend in a worker process.
40 """Initialize the separately installed EgoSMPLX backend in a worker process."""
import os
import sys
from pathlib import Path
def setup(repo):
45 repo = Path(repo).resolve()
    for sub in ['', 'main', 'common', 'data']:
        sys.path.insert(0, str(repo / sub))
    os.environ['EGOSMPLX_REPO_ROOT'] = str(repo)
    from config import cfg
50 path = os.environ.get('EGOSMPLX_CONFIG', str(repo / 'main/config/config_ft_egopw_scenego_ego

```

```

wholebody_unrealego_egofishbody.py'))
    cfg.get_config_fromfile(path)
    from egosmplx_pipeline import camera as calibrated_camera
    sys.modules['FishEyeCalibrated'] = calibrated_camera
5    from egosmplx_pipeline.camera import FishEyeCameraCalibrated
    from egosmplx_pipeline.projection import stable_world2camera
    FishEyeCameraCalibrated.world2camera_pytorch = stable_world2camera
    from utils.human_models import smpl_x
    import torch
10    torch.set_num_threads(2)
    torch.backends.cuda.matmul.allow_tf32 = False
    torch.backends.cudnn.allow_tf32 = False
    return smpl_x, FishEyeCameraCalibrated
文件名: egosmplx_pipeline/io.py
15 功能: Strict JSON/NPZ I/O and reproducibility metadata.
    """Strict JSON/NPZ I/O and reproducibility metadata."""
    import hashlib
    import json
    from pathlib import Path
20    import numpy as np
    def read(path):
        return json.loads(Path(path).read_text(encoding='utf-8'))
    def write(path, value):
        path = Path(path)
25    path.parent.mkdir(parents=True, exist_ok=True)
        temporary = path.with_suffix(path.suffix + '.tmp')
        temporary.write_text(json.dumps(value, indent=2, ensure_ascii=False, allow_nan=False) + '\n', encoding
                              ='utf-8')
        temporary.replace(path)
30    def sha(path):
        h = hashlib.sha256()
        with Path(path).open('rb') as handle:
            for block in iter(lambda: handle.read(8 * 1024**2), b''):
                h.update(block)
35    return h.hexdigest()
    def load(path):
        with np.load(path, allow_pickle=False) as data:
            result = {k: data[k].copy() for k in data.files}
            for key, value in result.items():
40                if np.issubdtype(value.dtype, np.number) and not np.isfinite(value).all():
                    raise ValueError('Nonfinite NPZ field: ' + key)
            return result
    def arr(tensor):
        return tensor.detach().cpu().numpy()
45    def rmse(prediction, target):
        """Root mean squared 2D Euclidean distance, not coordinate-wise RMSE."""
        prediction, target = np.asarray(prediction), np.asarray(target)
        if prediction.shape != target.shape or prediction.ndim != 2 or prediction.shape[1] != 2:
            raise ValueError('Expected matching N x 2 arrays')
50    if not len(prediction):

```

```

        return None
    return float(np.sqrt(np.mean(np.sum((prediction - target)**2, axis=1))))
文件名: egosmplx_pipeline/sampling.py
功能: Decode evenly spaced video frames; do not claim stereo synchronization.
5  """Decode evenly spaced video frames; do not claim stereo synchronization."""
    from pathlib import Path
    import re
    import cv2
    from egosmplx_pipeline.io import write, sha
10  def sample_video(video, calibration, camera, count, output):
        if count < 1 or not re.fullmatch('[A-Za-z0-9_-]+', camera):
            raise ValueError('Positive frame count and safe camera name required')
        output = Path(output)
        if output.exists():
15             raise FileExistsError(output)
        cap = cv2.VideoCapture(str(video))
        try:
            n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
            fps = cap.get(cv2.CAP_PROP_FPS)
20             if not cap.isOpened() or n < count or fps <= 0:
                raise ValueError('Video cannot provide requested frames')
            output.mkdir(parents=True)
            rows = []
            for k in range(count):
25                 index = min(n-1, int((k+.5) * n/count))
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                ok, image = cap.read()
                if not ok:
                    raise RuntimeError('Failed decoding frame ' + str(index))
30                 identifier = camera + '_f' + str(index).zfill(8)
                target = output / (identifier + '.png')
                if not cv2.imwrite(str(target), image):
                    raise IOError(target)
                rows.append(dict(id=identifier, camera=camera, image=target.name,
35                             calibration=str(Path(calibration).resolve()), image_sha256=sha(target),
                             frame_index_zero_based=index, approximate_video_seconds=index/fps))
            write(output / 'manifest.json', dict(frames=rows, exact_stereo_synchronization_claimed=False))
            return output / 'manifest.json'
        finally:
40             cap.release()
文件名: egosmplx_pipeline/egosmplx_predict.py
功能: Run person detection, EgoSMPLX inference, and fisheye projection on DownwardStereo extracted frames.
#!/usr/bin/env python
45  """Run person detection, EgoSMPLX inference, and fisheye projection on DownwardStereo extracted frames."""
    import argparse
    import json
    import os
50    from pathlib import Path

```

```

import sys
import time
import cv2
import numpy as np
5 import torch
import torchvision.transforms as transforms
from tqdm import tqdm
from mmdet.apis import init_detector, inference_detector
REPO_ROOT = Path(os.environ["EGOSMPLX_REPO_ROOT"]).resolve()
10 from egosmplx_pipeline import camera as calibrated_camera
sys.modules["FishEyeCalibrated"] = calibrated_camera
sys.path.insert(0, str(REPO_ROOT))
sys.path.insert(0, str(REPO_ROOT / "main"))
sys.path.insert(0, str(REPO_ROOT / "data"))
15 sys.path.insert(0, str(REPO_ROOT / "common"))
from config import cfg
from utils.inference_utils import process_mmdet_results, non_max_suppression
def parse_args():
    parser = argparse.ArgumentParser(description=__doc__)
20     parser.add_argument("--image-dir", type=Path, default=REPO_ROOT / "DownwardStereo" / "left_frames_stride10")
    parser.add_argument("--image-glob", default="*.jpg")
    parser.add_argument("--fisheye-calibration", type=Path, default=REPO_ROOT / "DownwardStereo" / "fisheye.left.calibration.json")
25     parser.add_argument(
        "--config-path",
        type=Path,
        default=REPO_ROOT / "main" / "config" / "config_ft_egopw_sceneego_egowholebody_unrealego_egofishbody.py",
30     )
    parser.add_argument(
        "--checkpoint-path",
        type=Path,
        default=REPO_ROOT / "output" / "train_5datasets_4gpu_resume_s7_to_s12_20260801_191616" / "model_dump" / "snapshot_11.pth.tar",
35     )
    parser.add_argument(
        "--det-config",
        type=Path,
        default=REPO_ROOT / "pretrained_models" / "mmdet" / "mmdet_faster_rcnn_r50_fpn_coco.py",
40     )
    parser.add_argument(
        "--det-checkpoint",
        type=Path,
        default=REPO_ROOT / "pretrained_models" / "mmdet" / "faster_rcnn_r50_fpn_1x_coco_20200130-047c8118.pth",
45     )
    parser.add_argument("--output-dir", type=Path, default=None)
    parser.add_argument("--max-images", type=int, default=None)
50     parser.add_argument("--score-thr", type=float, default=0.3)

```

```

    parser.add_argument("--bbox-thr", type=float, default=50.0)
    parser.add_argument("--iou-thr", type=float, default=0.5)
    parser.add_argument("--multi-person", action="store_true")
    parser.add_argument("--no-detector", action="store_true", help="Run one full-image SMPL-X predictio
5   n per image without Faster R-CNN.")
    parser.add_argument("--save-mesh", action="store_true")
    parser.add_argument(
        "--save-wireframe-overlay",
        action="store_true",
10    help="Also save a wireframe overlay from the same prediction and camera projection.",
    )
    parser.add_argument("--save-video", action="store_true")
    parser.add_argument("--show-verts", action="store_true")
    parser.add_argument("--show-bbox", action="store_true", default=True)
15    parser.add_argument("--mesh-mode", choices=("solid", "wireframe", "points"), default="solid")
    parser.add_argument(
        "--smplx-input-mode",
        choices=(
            "full_image",
20    "det_crop",
            "det_tangent",
            "det_undistort_contiguous",
        ),
        default="full_image",
25    help=(
        "det_tangent uses the detected person box to place the 16x12 ViT "
        "tangent patches in original fisheye-camera coordinates; "
        "det_undistort_contiguous first undistorts the full frame, detects "
        "there, and samples one continuous 12-column x 16-row ViT crop."
30    ),
    )
    parser.add_argument(
        "--det-tangent-expand",
        type=float,
35    default=1.2,
        help="Isotropic expansion applied to the detector box in det_tangent mode.",
    )
    parser.add_argument(
        "--det-tangent-patch-size",
40    type=float,
        default=0.1,
        help="Half-size of each tangent-plane patch in det_tangent mode.",
    )
    parser.add_argument(
        "--global-undistort-alpha",
45    type=float,
        default=1.0,
        help=(
        "OpenCV optimal-new-camera alpha for det_undistort_contiguous: "
50    "0 crops invalid borders, 1 retains the full calibrated field."

```

```

    ),
)
parser.add_argument(
    "--contiguous-roi-mode",
5    choices=("stretch", "aspect_pad"),
    default="stretch",
    help=(
        "For det_undistort_contiguous, stretch the clipped expanded box "
        "to the 12:16 token canvas, or preserve 12:16 geometry with "
10    "out-of-image padding."
    ),
)
parser.add_argument(
    "--input-crop-mode",
15    choices=("none", "center_square", "training_horizontal"),
    default="none",
    help="Crop the network input without changing the original image used for fisheye mesh overlay.",
)
parser.add_argument(
20    "--horizontal-crop-pixels",
    type=int,
    default=128,
    help="Pixels removed from both sides with --input-crop-mode training_horizontal.",
)
25 parser.add_argument("--contact-sheet-limit", type=int, default=40)
parser.add_argument(
    "--refine-camera-from-positionnet",
    action="store_true",
    help=(
30    "Keep the predicted SMPL-X local pose/shape fixed and refine a "
        "camera extrinsic rotation/translation against PositionNet upper-body joints."
    ),
)
parser.add_argument("--camera-fit-iters", type=int, default=300)
35 parser.add_argument("--camera-fit-lr", type=float, default=0.03)
parser.add_argument(
    "--camera-fit-mode",
    choices=("translation", "translation_global_orient", "extrinsics"),
    default="translation",
40    help=(
        "With --refine-camera-from-positionnet, optimize tx,ty,tz only; "
        "SMPL-X global_orient plus translation; or the legacy camera-frame "
        "rotation plus translation."
    ),
)
45 )
parser.add_argument("--num-gpus", type=int, default=1)
return parser.parse_args()
def make_output_dir(args):
    if args.output_dir is not None:
50    return args.output_dir

```

```

stamp = time.strftime("%Y%m%d_%H%M%S")
suffix = "full" if args.max_images is None else f"{args.max_images}imgs"
return REPO_ROOT / "output" / f"downwardstereo_left_stride10_detect_s11_{suffix}_{stamp}"

def setup_model(args, output_dir):
5   os.environ.setdefault("EGOSMPLX_REPO_ROOT", str(REPO_ROOT))
   os.environ["EGOSMPLX_FISHEYE_CALIBRATION"] = str(args.fisheye_calibration.resolve())
   cfg.get_config_fromfile(str(args.config_path.resolve()))
   cfg.update_test_config(
       testset="EHF",
10      agora_benchmark="na",
       shapy_eval_split=None,
       pretrained_model_path=str(args.checkpoint_path.resolve()),
       use_cache=False,
   )
15  cfg.fisheye_camera_path = str(args.fisheye_calibration.resolve())
   cfg.fisheye_camera_paths = [str(args.fisheye_calibration.resolve())]
   cfg.update_config(args.num_gpus, str(output_dir))
   from base import Demoer
   demoer = Demoer()
20  demoer._make_model()
   demoer.model.eval()
   return demoer

def load_runtime_utils():
   from FishEyeCalibrated import FishEyeCameraCalibrated
25  from utils.human_models import smpl_x
   from utils.preprocessing import load_img, process_bbox, generate_patch_image
   from utils.vis import render_mesh_fisheye, save_obj
   return FishEyeCameraCalibrated, smpl_x, load_img, process_bbox, generate_patch_image, render_mesh
   _fisheye, save_obj

30  def filter_bboxes(raw_bboxes, score_thr, bbox_thr):
   kept = []
   for bbox in raw_bboxes:
       x1, y1, x2, y2, score = [float(v) for v in bbox[:5]]
       w = abs(x2 - x1)
35      h = abs(y2 - y1)
       if score < score_thr:
           continue
       if w < bbox_thr or h < bbox_thr:
           continue
40      kept.append(np.array([x1, y1, x2, y2, score], dtype=np.float32))
   return kept

def save_contact_sheet(image_paths, out_path, max_images=40, thumb_w=360):
   paths = image_paths[:max_images]
   if not paths:
45      return
   thumbs = []
   for path in paths:
       img = cv2.imread(str(path), cv2.IMREAD_COLOR)
       if img is None:
50      continue

```

```

        h, w = img.shape[:2]
        thumb_h = max(1, int(h * thumb_w / max(1, w)))
        thumbs.append(cv2.resize(img, (thumb_w, thumb_h), interpolation=cv2.INTER_AREA))
    if not thumbs:
5         return
    cols = min(4, len(thumbs))
    rows = int(np.ceil(len(thumbs) / cols))
    max_h = max(t.shape[0] for t in thumbs)
    canvas = np.full((rows * max_h, cols * thumb_w, 3), 255, dtype=np.uint8)
10    for i, thumb in enumerate(thumbs):
        r, c = divmod(i, cols)
        canvas[r * max_h : r * max_h + thumb.shape[0], c * thumb_w : c * thumb_w + thumb.shape[1]] = thumb
    mb
    cv2.imwrite(str(out_path), canvas)
15 def draw_no_detection(img_rgb, text="NO PERSON DETECTED"):
    img = img_rgb.copy()
    cv2.putText(img, text, (40, 70), cv2.FONT_HERSHEY_SIMPLEX, 1.4, (255, 40, 40), 3, cv2.LINE_AA
)
    return img
20 def rgb_to_bgr_u8(img_rgb):
    img = np.clip(img_rgb, 0, 255).astype(np.uint8)
    return img[:, :, ::-1]
def expand_bbox_xyxy(box_xyxy, scale, image_w, image_h):
    """Expand a detector box about its center and clip it to the source image."""
25    x1, y1, x2, y2 = [float(value) for value in box_xyxy[:4]]
    center_x = (x1 + x2) * 0.5
    center_y = (y1 + y2) * 0.5
    width = max(2.0, abs(x2 - x1) * float(scale))
    height = max(2.0, abs(y2 - y1) * float(scale))
30    x1 = np.clip(center_x - width * 0.5, 0.0, float(image_w - 1))
    x2 = np.clip(center_x + width * 0.5, 0.0, float(image_w - 1))
    y1 = np.clip(center_y - height * 0.5, 0.0, float(image_h - 1))
    y2 = np.clip(center_y + height * 0.5, 0.0, float(image_h - 1))
    if x2 - x1 < 2.0 or y2 - y1 < 2.0:
35        raise ValueError(f"Expanded detector box is degenerate: {(x1, y1, x2, y2)}")
    return np.asarray([x1, y1, x2, y2], dtype=np.float32)
def create_global_undistort_model(calibration_path, image_w, image_h, alpha):
    """Build one continuous Brown-Conrady rectification for the full frame."""
    calibration = json.loads(Path(calibration_path).read_text())
40    required = (
        "sourceFocalLength",
        "sourcePrincipalPoint",
        "sourceRadialDistortion",
    )
45    missing = [key for key in required if key not in calibration]
    if missing:
        raise KeyError(
            "Continuous global undistortion requires Brown-Conrady source "
            f"fields in {calibration_path}; missing {missing}"
50    )

```

```

calibration_size = tuple(int(value) for value in calibration.get("size", (image_w, image_h)))
if calibration_size != (int(image_w), int(image_h)):
    raise ValueError(
        "Calibration/image size mismatch for global undistortion: "
5         f'calibration={calibration_size}, image={{image_w, image_h}}'
    )
focal = np.asarray(calibration["sourceFocalLength"], dtype=np.float64)
principal = np.asarray(calibration["sourcePrincipalPoint"], dtype=np.float64)
radial = list(calibration["sourceRadialDistortion"])
10 tangential = list(calibration.get("sourceTangentialDistortion", (0.0, 0.0)))
while len(radial) < 3:
    radial.append(0.0)
source_k = np.asarray(
    [
15     [focal[0], 0.0, principal[0]],
        [0.0, focal[1], principal[1]],
        [0.0, 0.0, 1.0],
    ],
    dtype=np.float64,
20 )
distortion = np.asarray(
    [radial[0], radial[1], tangential[0], tangential[1], radial[2]],
    dtype=np.float64,
)
25 new_k, valid_roi = cv2.getOptimalNewCameraMatrix(
    source_k,
    distortion,
    (int(image_w), int(image_h)),
    float(alpha),
30 (int(image_w), int(image_h)),
    centerPrincipalPoint=False,
)
map_x, map_y = cv2.initUndistortRectifyMap(
    source_k,
35 distortion,
    None,
    new_k,
    (int(image_w), int(image_h)),
    cv2.CV_32FC1,
40 )
return {
    "source_k": source_k,
    "distortion": distortion,
    "new_k": new_k,
45 "valid_roi": tuple(int(value) for value in valid_roi),
    "map_x": map_x,
    "map_y": map_y,
    "alpha": float(alpha),
}
50 def undistort_full_image(image_rgb, undistort_model):

```

```

        """Remap a complete fisheye frame into one continuous pinhole image."""
        return cv2.remap(
            image_rgb,
            undistort_model["map_x"],
5            undistort_model["map_y"],
            interpolation=cv2.INTER_LINEAR,
            borderMode=cv2.BORDER_CONSTANT,
            borderValue=(0, 0, 0),
        )
10    def undistorted_pixels_to_original(points_xy, undistort_model):
        """Map pixels from the continuous undistorted view to the raw frame."""
        points = np.asarray(points_xy, dtype=np.float64).reshape(-1, 2)
        new_k = undistort_model["new_k"]
        normalized = np.column_stack(
15            (
                (points[:, 0] - new_k[0, 2]) / new_k[0, 0],
                (points[:, 1] - new_k[1, 2]) / new_k[1, 1],
                np.ones((len(points),), dtype=np.float64),
            )
20        )
        projected, _ = cv2.projectPoints(
            normalized,
            np.zeros((3, 1), dtype=np.float64),
            np.zeros((3, 1), dtype=np.float64),
25            undistort_model["source_k"],
            undistort_model["distortion"],
        )
        return projected.reshape(-1, 2).astype(np.float32)
    def expand_bbox_to_aspect(box_xyxy, scale, target_width_over_height):
30        """Expand a box and pad it to one aspect ratio without clipping."""
        x1, y1, x2, y2 = [float(value) for value in box_xyxy[:4]]
        center_x = (x1 + x2) * 0.5
        center_y = (y1 + y2) * 0.5
        width = max(2.0, abs(x2 - x1) * float(scale))
35        height = max(2.0, abs(y2 - y1) * float(scale))
        if width / height > float(target_width_over_height):
            height = width / float(target_width_over_height)
        else:
            width = height * float(target_width_over_height)
40        return np.asarray(
            [
                center_x - width * 0.5,
                center_y - height * 0.5,
                center_x + width * 0.5,
45                center_y + height * 0.5,
            ],
            dtype=np.float32,
        )
    def create_contiguous_bbox_pixel_grid(
50        bbox_xyxy,

```

```

    patch_rows,
    patch_columns,
    patch_h,
    patch_w,
5   ):
    """Split one continuous rectified ROI into row-major ViT patch grids."""
    x1, y1, x2, y2 = [float(value) for value in bbox_xyxy]
    output_h = int(patch_rows) * int(patch_h)
    output_w = int(patch_columns) * int(patch_w)
10   xs = np.linspace(x1, x2, output_w, dtype=np.float32)
    ys = np.linspace(y1, y2, output_h, dtype=np.float32)
    grid_x, grid_y = np.meshgrid(xs, ys)
    full_grid = np.stack((grid_x, grid_y), axis=-1)
    return (
15       full_grid.reshape(
           patch_rows,
           patch_h,
           patch_columns,
           patch_w,
20       2,
       )
       .transpose(0, 2, 1, 3, 4)
       .reshape(patch_rows * patch_columns, patch_h, patch_w, 2)
       .astype(np.float32)
25   )
def draw_contiguous_sampling_grid(image_rgb, grid_pixels, bbox_xyxy):
    """Draw the regular 12-column x 16-row crop on the undistorted image."""
    canvas = image_rgb.copy()
    for patch in grid_pixels:
30       corners = np.asarray(
           [patch[0, 0], patch[0, -1], patch[-1, -1], patch[-1, 0]],
           dtype=np.float32,
       )
       cv2.polylines(
35           canvas,
           [np rint(corners).astype(np.int32)],
           isClosed=True,
           color=(255, 0, 0),
           thickness=1,
40           lineType=cv2.LINE_AA,
       )
       x1, y1, x2, y2 = np rint(bbox_xyxy).astype(int)
       cv2.rectangle(canvas, (x1, y1), (x2, y2), (255, 0, 0), 3)
       cv2.putText(
45           canvas,
           "continuous undistorted ROI -> 12 cols x 16 rows",
           (max(10, x1), max(30, y1 - 12)),
           cv2.FONT_HERSHEY_SIMPLEX,
           0.68,
50           (255, 0, 0),

```

```

        2,
        cv2.LINE_AA,
    )
    return canvas
5 def create_bbox_tangent_pixel_grid(
    sampler,
    fisheye_camera,
    bbox_xyxy,
    patch_rows=16,
10    patch_columns=12,
    patch_size=0.1,
):
    """Create row-major tangent-patch samples in original fisheye pixels."""
    x1, y1, x2, y2 = [float(value) for value in bbox_xyxy]
15    horizontal = (np.arange(patch_columns, dtype=np.float32) + 0.5) / patch_columns
    vertical = (np.arange(patch_rows, dtype=np.float32) + 0.5) / patch_rows
    normalized_centers = np.dstack(np.meshgrid(horizontal, vertical)).reshape(-1, 2)
    centers_2d = normalized_centers.copy()
    centers_2d[:, 0] = x1 + centers_2d[:, 0] * (x2 - x1)
20    centers_2d[:, 1] = y1 + centers_2d[:, 1] * (y2 - y1)
    centers_2d_up = centers_2d.copy()
    centers_2d_up[:, 1] -= (y2 - y1) / (2.0 * patch_rows)
    depth = np.ones((len(centers_2d),), dtype=np.float32)
    centers_3d = fisheye_camera.camera2world(centers_2d, depth)
25    centers_3d_up = fisheye_camera.camera2world(centers_2d_up, depth)
    patch_h, patch_w = [
        int(value) for value in sampler.patches_2d_per_camera_pixels.shape[-3:-1]
    ]
    tangent_patches = [
30        sampler._generate_patch_coordinates(
            center,
            center_up,
            (float(patch_size), float(patch_size)),
            (patch_h, patch_w),
35            visualize=False,
        )
        for center, center_up in zip(centers_3d, centers_3d_up)
    ]
    grid = sampler._project_patches_to_original_image(
40        tangent_patches,
        fisheye_camera=fisheye_camera,
        image_h=int(fisheye_camera.img_size[1]),
        image_w=int(fisheye_camera.img_size[0]),
        normalize=False,
45    )
    return np.asarray(grid, dtype=np.float32)
def draw_tangent_sampling_grid(image_rgb, grid_pixels, bbox_xyxy):
    """Draw every actual tangent-patch footprint and its detected ROI."""
    canvas = image_rgb.copy()
50    for patch in grid_pixels:

```

```

corners = np.asarray(
    [patch[0, 0], patch[0, -1], patch[-1, -1], patch[-1, 0]],
    dtype=np.float32,
)
5   if np.isfinite(corners).all():
        cv2.polylines(
            canvas,
            [np rint(corners).astype(np.int32)],
            True,
10      (0, 255, 0),
            1,
            cv2.LINE_AA,
        )
x1, y1, x2, y2 = np rint(bbox_xyxy).astype(int)
15   cv2.rectangle(canvas, (x1, y1), (x2, y2), (255, 0, 0), 3)
        cv2.putText(
            canvas,
            "det ROI -> 16x12 tangent patches",
            (max(10, x1), max(30, y1 - 12)),
20      cv2.FONT_HERSHEY_SIMPLEX,
            0.75,
            (255, 0, 0),
            2,
            cv2.LINE_AA,
25   )
        return canvas
def save_vit_patch_atlas(
    sampled_patches,
    out_path,
30   rows=16,
    columns=12,
    scale=4,
    gap=2,
    grid_color_rgb=None,
35   ):
    """Save the exact RGB patches returned by UndistortPatch before PatchEmbed."""
    patches = sampled_patches.detach().cpu().numpy()[0]
    if patches.shape[0] != rows * columns:
        raise ValueError(
40      f'Expected {rows * columns} ViT patches, got {patches.shape[0]}'
        )
    patches = np.clip(patches.transpose(0, 2, 3, 1) * 255.0, 0, 255).astype(np.uint8)
    patch_h, patch_w = patches.shape[1:3]
    atlas_h = rows * patch_h + (rows - 1) * gap
45   atlas_w = columns * patch_w + (columns - 1) * gap
    atlas = np.zeros((atlas_h, atlas_w, 3), dtype=np.uint8)
    for patch_index, patch in enumerate(patches):
        row, column = divmod(patch_index, columns)
        top = row * (patch_h + gap)
50      left = column * (patch_w + gap)

```

```

    atlas[top:top + patch_h, left:left + patch_w] = patch
atlas = cv2.resize(
    atlas,
    (atlas.shape[1] * scale, atlas.shape[0] * scale),
5    interpolation=cv2.INTER_NEAREST,
)
if grid_color_rgb is not None:
    patch_h_scaled = patch_h * scale
    patch_w_scaled = patch_w * scale
10    for row in range(1, rows):
        y = row * patch_h_scaled
        cv2.line(
            atlas,
            (0, y),
15            (atlas.shape[1] - 1, y),
            grid_color_rgb,
            2,
            cv2.LINE_AA,
        )
20    for column in range(1, columns):
        x = column * patch_w_scaled
        cv2.line(
            atlas,
            (x, 0),
25            (x, atlas.shape[0] - 1),
            grid_color_rgb,
            2,
            cv2.LINE_AA,
        )
30    cv2.imwrite(str(out_path), rgb_to_bgr_u8(atlas))
POSITIONNET_BODY_EDGES = (
    (0, 1), (1, 3), (3, 5),
    (0, 2), (2, 4), (4, 6),
    (0, 7),
35    (7, 8), (8, 10), (10, 12),
    (7, 9), (9, 11), (11, 13),
    (5, 14), (5, 15), (5, 16),
    (6, 17), (6, 18), (6, 19),
    (7, 24), (24, 22), (22, 20), (24, 23), (23, 21),
40 )
def positionnet_coords_to_sampling_grid(
    body_joint_img,
    sampling_grid_pixels,
    patch_rows=16,
45    patch_columns=12,
):
    """Map PositionNet coordinates through the actual tangent-patch centers."""
    coords = np.asarray(body_joint_img, dtype=np.float32)
    grid = np.asarray(sampling_grid_pixels, dtype=np.float32)
50    if grid.shape[0] != patch_rows * patch_columns:

```

```

        raise ValueError(
            f'Expected {patch_rows * patch_columns} sampling patches, got {grid.shape[0]}'
        )
    patch_h, patch_w = grid.shape[1:3]
5   center_rows = slice((patch_h - 1) // 2, patch_h // 2 + 1)
    center_columns = slice((patch_w - 1) // 2, patch_w // 2 + 1)
    patch_centers = grid.reshape(
        patch_rows, patch_columns, patch_h, patch_w, 2
    )[:, :, center_rows, center_columns].mean(axis=(2, 3))
10  x = np.clip(coords[:, 0], 0.0, float(patch_columns - 1))
    y = np.clip(coords[:, 1], 0.0, float(patch_rows - 1))
    x0 = np.floor(x).astype(np.int64)
    y0 = np.floor(y).astype(np.int64)
    x1 = np.minimum(x0 + 1, patch_columns - 1)
15  y1 = np.minimum(y0 + 1, patch_rows - 1)
    wx = (x - x0).reshape(-1, 1)
    wy = (y - y0).reshape(-1, 1)
    pixels = (
        (1.0 - wx) * (1.0 - wy) * patch_centers[y0, x0]
20  + wx * (1.0 - wy) * patch_centers[y0, x1]
        + (1.0 - wx) * wy * patch_centers[y1, x0]
        + wx * wy * patch_centers[y1, x1]
    )
    return pixels.astype(np.float32), patch_centers.astype(np.float32)
25  def draw_positionnet_result(image_rgb, body_joint_hm, body_joint_pixels, sampling_bbox_xyxy):
    """Overlay the depth-marginalized PositionNet heatmap and body skeleton."""
    canvas = image_rgb.copy()
    heatmap_2d = np.asarray(body_joint_hm, dtype=np.float32).sum(axis=(0, 1))
    heatmap_2d -= heatmap_2d.min()
30  peak = float(heatmap_2d.max())
    if peak > 0:
        heatmap_2d /= peak
        heatmap_color_bgr = cv2.applyColorMap(
            np rint(heatmap_2d * 255.0).astype(np.uint8),
35  cv2.COLORMAP_JET,
        )
        heatmap_color = heatmap_color_bgr[:, :, ::-1]
        image_h, image_w = canvas.shape[:2]
        x1, y1, x2, y2 = [int(round(value)) for value in sampling_bbox_xyxy]
40  x1, x2 = np.clip([x1, x2], 0, image_w)
        y1, y2 = np.clip([y1, y2], 0, image_h)
        if x2 > x1 and y2 > y1:
            heatmap_roi = cv2.resize(
                heatmap_color,
45  (x2 - x1, y2 - y1),
                interpolation=cv2.INTER_CUBIC,
            ).astype(canvas.dtype, copy=False)
            canvas[y1:y2, x1:x2] = cv2.addWeighted(
                canvas[y1:y2, x1:x2],
50  0.68,

```

```

        heatmap_roi,
        0.32,
        0.0,
    )
5   for start, end in POSITIONNET_BODY_EDGES:
        points = body_joint_pixels[[start, end]]
        if np.isfinite(points).all():
            cv2.line(
                canvas,
10             tuple(np rint(points[0]).astype(int)),
                tuple(np rint(points[1]).astype(int)),
                (255, 255, 0),
                4,
                cv2.LINE_AA,
15             )
        for point in body_joint_pixels:
            if np.isfinite(point).all():
                cv2.circle(
                    canvas,
20                 tuple(np rint(point).astype(int)),
                    6,
                    (255, 0, 0),
                    -1,
                    cv2.LINE_AA,
25                 )
            cv2.putText(
                canvas,
                "PositionNet: 25 body joints (yellow) + heatmap",
                (24, 42),
30                 cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255, 255, 0),
                2,
                cv2.LINE_AA,
35             )
        return canvas
def draw_positionnet_smplx_2d_comparison(
    image_rgb,
    positionnet_pixels,
40    smplx_pixels,
):
    """Draw all 25 corresponding body joints from PositionNet and SMPL-X."""
    canvas = image_rgb.copy()
    positionnet_pixels = np.asarray(positionnet_pixels, dtype=np.float32)
    smplx_pixels = np.asarray(smplx_pixels, dtype=np.float32)
45    if positionnet_pixels.shape != (25, 2) or smplx_pixels.shape != (25, 2):
        raise ValueError(
            "PositionNet and SMPL-X body points must both have shape (25, 2)"
        )
50    valid = np.isfinite(positionnet_pixels).all(axis=1) & np.isfinite(smplx_pixels).all(axis=1)

```

```

valid &= (np.abs(positionnet_pixels) < 1e6).all(axis=1)
valid &= (np.abs(smplx_pixels) < 1e6).all(axis=1)
errors = np.linalg.norm(smplx_pixels - positionnet_pixels, axis=1)
# Draw per-joint correspondence first so the two skeletons remain visible.
5  for joint_index in np.flatnonzero(valid):
    positionnet_point = tuple(np rint(positionnet_pixels[joint_index]).astype(int))
    smplx_point = tuple(np rint(smplx_pixels[joint_index]).astype(int))
    cv2.line(
10         canvas,
        positionnet_point,
        smplx_point,
        (0, 255, 255),
        1,
        cv2.LINE_AA,
15     )
    for start, end in POSITIONNET_BODY_EDGES:
        if valid[start] and valid[end]:
            cv2.line(
                canvas,
20         tuple(np rint(positionnet_pixels[start]).astype(int)),
                tuple(np rint(positionnet_pixels[end]).astype(int)),
                (255, 0, 0),
                3,
                cv2.LINE_AA,
25         )
            cv2.line(
                canvas,
                tuple(np rint(smplx_pixels[start]).astype(int)),
                tuple(np rint(smplx_pixels[end]).astype(int)),
30         (0, 255, 0),
                3,
                cv2.LINE_AA,
            )
        for joint_index in np.flatnonzero(valid):
35         positionnet_point = tuple(np rint(positionnet_pixels[joint_index]).astype(int))
            smplx_point = tuple(np rint(smplx_pixels[joint_index]).astype(int))
            cv2.circle(canvas, positionnet_point, 6, (255, 0, 0), -1, cv2.LINE_AA)
            cv2.circle(canvas, smplx_point, 7, (0, 255, 0), 2, cv2.LINE_AA)
            midpoint = tuple(
40         np rint((positionnet_pixels[joint_index] + smplx_pixels[joint_index]) * 0.5)
                .astype(int)
            )
            cv2.putText(
                canvas,
45         str(joint_index),
                midpoint,
                cv2.FONT_HERSHEY_SIMPLEX,
                0.42,
                (255, 255, 255),
50         1,

```

```

        cv2.LINE_AA,
    )
    selected_valid = valid[CAMERA_FIT_BODY_INDICES]
    selected_errors = errors[CAMERA_FIT_BODY_INDICES][selected_valid]
5    all_mean_error = float(errors[valid].mean()) if valid.any() else float("nan")
    fit_mean_error = (
        float(selected_errors.mean()) if len(selected_errors) else float("nan")
    )
    labels = (
10        ((255, 0, 0), "red: PositionNet 2D (filled)"),
        ((0, 255, 0), "green: SMPL-X projected 2D (ring)"),
        ((0, 255, 255), "cyan: corresponding joint"),
        ((255, 255, 255), f"mean all25={all_mean_error:.1f}px, fit12={fit_mean_error:.1f}px"),
    )
15    for row, (color, label) in enumerate(labels):
        cv2.putText(
            canvas,
            label,
            (24, 38 + row * 31),
20            cv2.FONT_HERSHEY_SIMPLEX,
            0.68,
            color,
            2,
            cv2.LINE_AA,
25        )
    return canvas

CAMERA_FIT_BODY_INDICES = np.asarray(
    [7, 8, 9, 10, 11, 12, 13, 20, 21, 22, 23, 24], dtype=np.int64
)
30 CAMERA_FIT_BODY_EDGES = (
    (7, 8), (7, 9), (8, 10), (10, 12), (9, 11), (11, 13),
    (7, 20), (7, 21), (20, 22), (21, 23), (22, 24), (23, 24),
)
def heatmap_joints_to_original_pixels(joint_img, input_hw, input_offset):
35     """Invert the normalization used by Model.get_coord for full-image input."""
    input_h, input_w = [float(v) for v in input_hw]
    offset_x, offset_y = [float(v) for v in input_offset]
    joint_px = np.asarray(joint_img, dtype=np.float64).copy()
    joint_px[:, 0] = joint_px[:, 0] / float(cfg.output_hm_shape[2]) * input_w + offset_x
40    joint_px[:, 1] = joint_px[:, 1] / float(cfg.output_hm_shape[1]) * input_h + offset_y
    return joint_px

def axis_angle_to_matrix(axis_angle):
    """Differentiable Rodrigues transform for a single three-vector."""
    theta = torch.linalg.vector_norm(axis_angle).clamp_min(1e-8)
45    axis = axis_angle / theta
    x, y, z = axis.unbind()
    zero = torch.zeros((), device=axis.device, dtype=axis.dtype)
    skew = torch.stack(
        (zero, -z, y, z, zero, -x, -y, x, zero)
50    ).reshape(3, 3)

```

```

identity = torch.eye(3, device=axis.device, dtype=axis.dtype)
return identity + torch.sin(theta) * skew + (1.0 - torch.cos(theta)) * (skew @ skew)
def compose_global_orient(delta_rotation, network_global_orient):
    """Left-compose a camera-frame delta into the SMPL-X root axis-angle."""
5   delta_np = delta_rotation.detach().cpu().numpy().astype(np.float64)
    network_np = (
        network_global_orient.detach().cpu().numpy().reshape(3).astype(np.float64)
    )
    network_rotation, _ = cv2.Rodrigues(network_np)
10   refined_rotation = delta_np @ network_rotation
    refined_axis_angle, _ = cv2.Rodrigues(refined_rotation)
    return torch.as_tensor(
        refined_axis_angle.reshape(3),
        device=network_global_orient.device,
15   dtype=network_global_orient.dtype,
    )
def refine_camera_extrinsics(
    body_joints_cam,
    body_targets_px,
20   initial_translation,
    fisheye_cam,
    iterations,
    learning_rate,
):
25   """Fit a camera-frame delta R,t while local pose, shape, and expression stay fixed."""
    if iterations <= 0:
        raise ValueError("--camera-fit-iters must be positive")
    if learning_rate <= 0:
        raise ValueError("--camera-fit-lr must be positive")
30   device = body_joints_cam.device
    dtype = body_joints_cam.dtype
    indices = torch.as_tensor(CAMERA_FIT_BODY_INDICES, device=device, dtype=torch.long)
    joints = body_joints_cam.index_select(0, indices).detach()
    targets = torch.as_tensor(
35   body_targets_px[CAMERA_FIT_BODY_INDICES], device=device, dtype=dtype
    )
    initial = initial_translation.detach().to(device=device, dtype=dtype)
    identity = torch.eye(3, device=device, dtype=dtype)
    def project(rotation, translation):
40   transformed = joints @ rotation.t() + translation[None, :]
        return fisheye_cam.world2camera_pytorch(transformed)
    with torch.no_grad():
        initial_projection = project(identity, initial)
        initial_error = torch.linalg.vector_norm(
45   initial_projection - targets, dim=1
        ).mean()
    initial_z = max(0.1, float(initial[2].item()))
    depth_starts = sorted(set([initial_z, 0.5, 0.75, 1.0, 1.5, 2.0]))
    best = None
50   for depth_start in depth_starts:

```

```

parameter = torch.tensor(
    [
        0.0,
        0.0,
5       0.0,
        float(initial[0]),
        float(initial[1]),
        float(np.log(depth_start)),
    ],
10    device=device,
    dtype=dtype,
    requires_grad=True,
)
optimizer = torch.optim.Adam([parameter], lr=learning_rate)
15 for _ in range(iterations):
    rotation = axis_angle_to_matrix(parameter[:3])
    translation = torch.stack(
        (parameter[3], parameter[4], torch.exp(parameter[5]))
    )
20    projection = project(rotation, translation)
    # Charbonnier-style robust pixel loss. Dividing by 100 keeps the
    # Adam step scale stable while preserving the minimizer.
    residual = (projection - targets) / 100.0
    loss = torch.sqrt(residual.square() + 1e-4).mean()
25    if not torch.isfinite(loss):
        break
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
30    with torch.no_grad():
        rotation_norm = torch.linalg.vector_norm(parameter[:3])
        if rotation_norm > np.pi:
            parameter[:3].mul_(float(np.pi / rotation_norm))
            parameter[3].clamp_(-3.0, 3.0)
35            parameter[4].clamp_(-3.0, 3.0)
            parameter[5].clamp_(float(np.log(0.1)), float(np.log(5.0)))
    with torch.no_grad():
        rotation = axis_angle_to_matrix(parameter[:3])
        translation = torch.stack(
40            (parameter[3], parameter[4], torch.exp(parameter[5]))
        )
        projection = project(rotation, translation)
        error = torch.linalg.vector_norm(projection - targets, dim=1).mean()
        candidate = (
45            float(error),
            rotation.detach().clone(),
            translation.detach().clone(),
            projection.detach().clone(),
        )
50    if best is None or candidate[0] < best[0]:

```

```

        best = candidate
    if best is None:
        raise RuntimeError("Camera translation refinement produced no finite solution")
    return {
5         "rotation": best[1],
        "translation": best[2],
        "initial_projection": initial_projection.detach(),
        "refined_projection": best[3],
        "initial_mean_error_px": float(initial_error),
10        "refined_mean_error_px": float(best[0]),
        "joint_indices": CAMERA_FIT_BODY_INDICES.copy(),
    }
def refine_camera_translation(
15    body_joints_local,
    body_targets_px,
    initial_translation,
    fisheye_cam,
    iterations,
    learning_rate,
20 ):
    """Fit tx, ty, and positive tz while keeping pose and rotation fixed."""
    if iterations <= 0:
        raise ValueError("--camera-fit-iters must be positive")
    if learning_rate <= 0:
25         raise ValueError("--camera-fit-lr must be positive")
    device = body_joints_local.device
    dtype = body_joints_local.dtype
    indices = torch.as_tensor(CAMERA_FIT_BODY_INDICES, device=device, dtype=torch.long)
    joints = body_joints_local.index_select(0, indices).detach()
30    targets = torch.as_tensor(
        body_targets_px[CAMERA_FIT_BODY_INDICES], device=device, dtype=dtype
    )
    initial = initial_translation.detach().to(device=device, dtype=dtype)
    identity = torch.eye(3, device=device, dtype=dtype)
35    def project(translation):
        return fisheye_cam.world2camera_pytorch(joints + translation[None, :])
    with torch.no_grad():
        initial_projection = project(initial)
        initial_error = torch.linalg.vector_norm(
40         initial_projection - targets, dim=1
        ).mean()
    initial_z = max(0.1, float(initial[2].item()))
    depth_starts = sorted(set([initial_z, 0.5, 0.75, 1.0, 1.5, 2.0]))
    best = None
45    for depth_start in depth_starts:
        parameter = torch.tensor(
            [
                float(initial[0]),
                float(initial[1]),
50                float(np.log(depth_start)),

```

```

    ],
    device=device,
    dtype=dtype,
    requires_grad=True,
5   )
    optimizer = torch.optim.Adam([parameter], lr=learning_rate)
    for _ in range(iterations):
        translation = torch.stack(
10         (parameter[0], parameter[1], torch.exp(parameter[2]))
        )
        projection = project(translation)
        residual = (projection - targets) / 100.0
        loss = torch.sqrt(residual.square() + 1e-4).mean()
        if not torch.isfinite(loss):
15         break
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        with torch.no_grad():
20         parameter[0].clamp_(-3.0, 3.0)
        parameter[1].clamp_(-3.0, 3.0)
        parameter[2].clamp_(float(np.log(0.1)), float(np.log(5.0)))
        with torch.no_grad():
            translation = torch.stack(
25             (parameter[0], parameter[1], torch.exp(parameter[2]))
            )
            projection = project(translation)
            error = torch.linalg.vector_norm(projection - targets, dim=1).mean()
            candidate = (
30             float(error),
            translation.detach().clone(),
            projection.detach().clone(),
            )
            if best is None or candidate[0] < best[0]:
35             best = candidate
        if best is None:
            raise RuntimeError("Camera translation refinement produced no finite solution")
        return {
            "rotation": identity,
40         "translation": best[1],
            "initial_projection": initial_projection.detach(),
            "refined_projection": best[2],
            "initial_mean_error_px": float(initial_error),
            "refined_mean_error_px": float(best[0]),
45         "joint_indices": CAMERA_FIT_BODY_INDICES.copy(),
        }
    def draw_camera_fit_debug(
        image_rgb,
        target_body_px,
50     initial_body_px,

```

```

    refined_body_px,
    initial_error,
    refined_error,
):
5   canvas = image_rgb.copy()
    series = (
        (target_body_px, (255, 0, 0), "PositionNet"),
        (initial_body_px, (0, 0, 255), "network cam"),
        (refined_body_px, (0, 255, 0), "refined cam"),
10   )
    for points, color, _ in series:
        for start, end in CAMERA_FIT_BODY_EDGES:
            if np.isfinite(points[[start, end]]).all():
                cv2.line(
15                 canvas,
                    tuple(np rint(points[start]).astype(int)),
                    tuple(np rint(points[end]).astype(int)),
                    color,
                    4,
20                 cv2.LINE_AA,
                )
            for index in CAMERA_FIT_BODY_INDICES:
                if np.isfinite(points[index]).all():
                    cv2.circle(
25                     canvas,
                        tuple(np rint(points[index]).astype(int)),
                        8,
                        color,
                        -1,
30                     cv2.LINE_AA,
                    )
        labels = (
            ((255, 0, 0), "red: PositionNet target"),
            ((0, 0, 255), f"blue: network cam ({initial_error:.1f} px)"),
35            ((0, 255, 0), f"green: refined cam ({refined_error:.1f} px)"),
        )
        for row, (color, label) in enumerate(labels):
            cv2.putText(
                canvas,
                label,
40                (30, 45 + row * 38),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.85,
                color,
45                3,
                cv2.LINE_AA,
            )
        return canvas
def main():
50     args = parse_args()

```

```

paths_to_check = [args.image_dir, args.fisheye_calibration, args.config_path, args.checkpoint_path]
if not args.no_detector:
    paths_to_check.extend([args.det_config, args.det_checkpoint])
for path in paths_to_check:
5     if not path.exists():
        raise FileNotFoundError(path)
if args.no_detector and args.smplx_input_mode != "full_image":
    raise ValueError("--no-detector requires --smplx-input-mode full_image")
if (
10     args.refine_camera_from_positionnet
    and args.smplx_input_mode
    not in ("full_image", "det_tangent", "det_undistort_contiguous")
):
    raise ValueError(
15         "Camera refinement requires --smplx-input-mode full_image, "
        "det_tangent, or det_undistort_contiguous"
    )
if args.horizontal_crop_pixels < 0:
    raise ValueError("--horizontal-crop-pixels must be non-negative")
20 if args.det_tangent_expand <= 0:
    raise ValueError("--det-tangent-expand must be positive")
if args.det_tangent_patch_size <= 0:
    raise ValueError("--det-tangent-patch-size must be positive")
if not 0.0 <= args.global_undistort_alpha <= 1.0:
25     raise ValueError("--global-undistort-alpha must be in [0, 1]")
if (
    args.smplx_input_mode in ("det_tangent", "det_undistort_contiguous")
    and args.input_crop_mode != "none"
):
30     raise ValueError(
        f"{args.smplx_input_mode} requires --input-crop-mode none"
    )
image_paths = sorted(args.image_dir.glob(args.image_glob))
if args.max_images is not None:
35     image_paths = image_paths[: args.max_images]
if not image_paths:
    raise RuntimeError(f"No images found in {args.image_dir} with glob {args.image_glob}")
output_dir = make_output_dir(args)
overlay_dir = output_dir / "img"
40 wireframe_dir = output_dir / "wireframe"
smplx_dir = output_dir / "smplx"
meta_dir = output_dir / "meta"
det_dir = output_dir / "detections"
debug_dir = output_dir / "projection_debug"
45 mesh_dir = output_dir / "mesh"
vit_patch_dir = output_dir / "vit_patches"
positionnet_dir = output_dir / "positionnet"
undistorted_dir = output_dir / "undistorted"
undistorted_detection_dir = output_dir / "undistorted_detections"
50 joint_compare_dir = output_dir / "joint_compare"

```

```

mesh_joint_compare_dir = output_dir / "mesh_joint_compare"
for path in (overlay_dir, smplx_dir, meta_dir, det_dir, debug_dir):
    path.mkdir(parents=True, exist_ok=True)
if args.smplx_input_mode in ("det_tangent", "det_undistort_contiguous"):
5     vit_patch_dir.mkdir(parents=True, exist_ok=True)
    positionnet_dir.mkdir(parents=True, exist_ok=True)
if args.smplx_input_mode == "det_undistort_contiguous":
    undistorted_dir.mkdir(parents=True, exist_ok=True)
    undistorted_detection_dir.mkdir(parents=True, exist_ok=True)
10    joint_compare_dir.mkdir(parents=True, exist_ok=True)
    mesh_joint_compare_dir.mkdir(parents=True, exist_ok=True)
if args.save_mesh:
    mesh_dir.mkdir(parents=True, exist_ok=True)
if args.save_wireframe_overlay:
15    wireframe_dir.mkdir(parents=True, exist_ok=True)
    demoer = setup_model(args, output_dir)
    FishEyeCameraCalibrated, smpl_x, load_img, process_bbox, generate_patch_image, render_mesh_fishe
ye, save_obj = load_runtime_utils()
    fisheye_cam = FishEyeCameraCalibrated(str(args.fisheye_calibration.resolve()))
20    positionnet_capture = {}
    if args.smplx_input_mode in ("det_tangent", "det_undistort_contiguous"):
        model_core = demoer.model.module if hasattr(demoer.model, "module") else demoer.model
        def capture_positionnet_output(_module, _inputs, outputs):
            positionnet_capture["body_joint_hm"] = outputs[0].detach().cpu()
25            positionnet_capture["body_joint_img"] = outputs[1].detach().cpu()
            model_core.body_position_net.register_forward_hook(capture_positionnet_output)
        detector = None
        if not args.no_detector:
            detector = init_detector(str(args.det_config.resolve()), str(args.det_checkpoint.resolve()), device="cud
30 a:0")
            transform = transforms.ToTensor()
            first = cv2.imread(str(image_paths[0]), cv2.IMREAD_COLOR)
            if first is None:
                raise RuntimeError(f"Failed to read {image_paths[0]}")
35            img_h0, img_w0 = first.shape[:2]
            undistort_model = None
            if args.smplx_input_mode == "det_undistort_contiguous":
                undistort_model = create_global_undistort_model(
                    args.fisheye_calibration,
40                    img_w0,
                    img_h0,
                    args.global_undistort_alpha,
                )
            video_writer = None
45            if args.save_video:
                fourcc = cv2.VideoWriter_fourcc(*"mp4v")
                video_writer = cv2.VideoWriter(str(output_dir / "detected_mesh_overlay_stride10.mp4"), fourcc, 10.
0, (img_w0, img_h0))
            saved_images = []
50            saved_wireframe_images = []

```

```

saved_sampling_grid_images = []
saved_patch_atlas_images = []
saved_positionnet_images = []
saved_camera_fit_images = []
5 saved_joint_compare_images = []
saved_mesh_joint_compare_images = []
saved_undistorted_images = []
saved_undistorted_detection_images = []
saved_contiguous_patch_grid_images = []
10 records = []
projection_consistency_errors = []
total_persons = 0
no_detection = 0
start_time = time.time()
15 progress_desc = "Full-image project" if args.no_detector else "Detect + project"
with tqdm(image_paths, desc=progress_desc, mininterval=10.0) as pbar:
    for image_path in pbar:
        original_img = load_img(str(image_path))
        vis_img = original_img.copy()
20 wireframe_img = original_img.copy() if args.save_wireframe_overlay else None
        img_h, img_w = original_img.shape[:2]
        detector_img = original_img
        if args.smplx_input_mode == "det_undistort_contiguous":
            if (img_w, img_h) != (img_w0, img_h0):
25                 raise ValueError(
                    "All globally undistorted frames must share one size; "
                    f"expected {(img_w0, img_h0)}, got {(img_w, img_h)}"
                )
            detector_img = undistort_full_image(original_img, undistort_model)
30 undistorted_out = undistorted_dir / f"{image_path.stem}_undistorted.jpg"
            cv2.imwrite(str(undistorted_out), rgb_to_bgr_u8(detector_img))
            saved_undistorted_images.append(undistorted_out)
        if args.no_detector:
            raw_boxes = []
35 det_boxes = [np.array([0.0, 0.0, float(img_w - 1), float(img_h - 1), 1.0], dtype=np.float32)]
        else:
            if args.smplx_input_mode == "det_undistort_contiguous":
                detector_input = np.ascontiguousarray(rgb_to_bgr_u8(detector_img))
            else:
40                 detector_input = str(image_path)
            mmdet_results = inference_detector(detector, detector_input)
            raw_boxes = process_mmdet_results(mmdet_results, cat_id=0, multi_person=True)[0]
            filtered = filter_bboxes(raw_boxes, args.score_thr, args.bbox_thr)
            if args.multi_person:
45                 det_boxes = non_max_suppression(filtered, args.iou_thr) if filtered else []
            else:
                det_boxes = filtered[:1]
        if args.smplx_input_mode == "det_undistort_contiguous":
            detector_vis = detector_img.copy()
50         for selected_box in det_boxes:

```

```

dx1, dy1, dx2, dy2, dscore = [
    float(value) for value in selected_box[:5]
]
cv2.rectangle(
5     detector_vis,
    (int(round(dx1)), int(round(dy1))),
    (int(round(dx2)), int(round(dy2))),
    (255, 0, 0),
    3,
10 )
cv2.putText(
    detector_vis,
    f"person {dscore:.2f}",
    (int(round(dx1)), max(25, int(round(dy1)) - 10)),
15     cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (255, 0, 0),
    2,
    cv2.LINE_AA,
20 )
undistorted_detection_out = (
    undistorted_detection_dir
    / f"{image_path.stem}_undistorted_detection.jpg"
)
25 cv2.imwrite(
    str(undistorted_detection_out),
    rgb_to_bgr_u8(detector_vis),
)
saved_undistorted_detection_images.append(
30     undistorted_detection_out
)
det_record = {
    "detector_enabled": not args.no_detector,
    "image_path": str(image_path.resolve()),
35     "raw_person_detections": int(len(raw_boxes)),
    "score_thr": float(args.score_thr),
    "bbox_thr": float(args.bbox_thr),
    "detector_coordinate_space": (
        "global_undistorted"
40         if args.smplx_input_mode == "det_undistort_contiguous"
        else "original_fisheye"
    ),
    "selected_bboxes_xyxy_score": [np.asarray(b).astype(float).tolist() for b in det_boxes],
}
45 (det_dir / f"{image_path.stem}_detections.json").write_text(json.dumps(det_record, indent=2) + "\n")

if not det_boxes:
    no_detection += 1
    vis_img = draw_no_detection(vis_img)
50 out_img = overlay_dir / f"{image_path.stem}_det_mesh_overlay.jpg"

```

```

cv2.imwrite(str(out_img), rgb_to_bgr_u8(vis_img))
saved_images.append(out_img)
if wireframe_img is not None:
    wireframe_out = wireframe_dir / f'{image_path.stem}_det_mesh_wireframe.jpg'
5    cv2.imwrite(str(wireframe_out), rgb_to_bgr_u8(wireframe_img))
    saved_wireframe_images.append(wireframe_out)
if video_writer is not None:
    video_writer.write(rgb_to_bgr_u8(vis_img))
records.append(**det_record, "persons_projected": 0)
10    continue
persons_projected = 0
for bbox_id, det_box in enumerate(det_boxes):
    x1, y1, x2, y2, score = [float(v) for v in det_box[:5]]
    mmdet_box_xywh = np.array([x1, y1, abs(x2 - x1), abs(y2 - y1)], dtype=np.float32)
15    sampling_bbox_xyxy = None
    sampling_grid_pixels = None
    if args.smplx_input_mode == "det_tangent":
        sampling_bbox_xyxy = expand_bbox_xyxy(
            det_box,
20            args.det_tangent_expand,
            img_w,
            img_h,
        )
        sx1, sy1, sx2, sy2 = sampling_bbox_xyxy
25        bbox = np.asarray([sx1, sy1, sx2 - sx1, sy2 - sy1], dtype=np.float32)
    elif args.smplx_input_mode == "det_undistort_contiguous":
        if args.contiguous_roi_mode == "stretch":
            sampling_bbox_xyxy = expand_bbox_xyxy(
                det_box,
30                args.det_tangent_expand,
                img_w,
                img_h,
            )
        else:
35            sampling_bbox_xyxy = expand_bbox_to_aspect(
                det_box,
                args.det_tangent_expand,
                target_width_over_height=12.0 / 16.0,
            )
40            sx1, sy1, sx2, sy2 = sampling_bbox_xyxy
            bbox = np.asarray(
                [sx1, sy1, sx2 - sx1, sy2 - sy1],
                dtype=np.float32,
            )
45            else:
                bbox = process_bbox(mmdet_box_xywh, img_w, img_h)
            if bbox is None:
                continue
            if args.smplx_input_mode == "det_crop":
50                img_crop, img2bb_trans, bb2img_trans = generate_patch_image(

```

```

        original_img, bbox, 1.0, 0.0, False, cfg.input_img_shape
    )
    img = transform(img_crop.astype(np.float32)) / 255.0
    img = img.cuda()[None, :, :, :]
5    meta_info = {}
    # FishEyeCameraCalibrated.world2camera already returns
    # original fisheye-image pixel coordinates. Applying
    # bb2img_trans here double-applies the crop inverse and
    # shifts/scales the mesh overlay.
10   render_proj_trans = None
else:
    img2bb_trans = np.eye(2, 3, dtype=np.float32)
    bb2img_trans = np.eye(2, 3, dtype=np.float32)
    input_img = (
15        detector_img
        if args.smplx_input_mode == "det_undistort_contiguous"
        else original_img
    )
    input_offset = np.array([0.0, 0.0], dtype=np.float32)
20   input_crop_xyxy = [0, 0, int(img_w), int(img_h)]
    if args.input_crop_mode == "center_square":
        side = min(img_h, img_w)
        crop_left = int(round((img_w - side) / 2.0))
        crop_top = int(round((img_h - side) / 2.0))
25        input_img = original_img[crop_top:crop_top + side, crop_left:crop_left + side, :]
        input_offset = np.array([crop_left, crop_top], dtype=np.float32)
        input_crop_xyxy = [crop_left, crop_top, crop_left + side, crop_top + side]
    elif args.input_crop_mode == "training_horizontal":
        crop_left = int(args.horizontal_crop_pixels)
30        crop_right = int(args.horizontal_crop_pixels)
        if crop_left + crop_right >= img_w:
            raise ValueError(
                "Horizontal crop removes the entire image: "
                f"width={img_w}, left={crop_left}, right={crop_right}"
35            )
        input_img = original_img[:, crop_left:img_w - crop_right, :]
        input_offset = np.array([crop_left, 0.0], dtype=np.float32)
        input_crop_xyxy = [crop_left, 0, img_w - crop_right, img_h]
    input_h, input_w = input_img.shape[:2]
40   img = transform(input_img.astype(np.float32)) / 255.0
    img = img.cuda()[None, :, :, :]
    meta_info = {
        "camera_id": torch.zeros((1,), dtype=torch.long, device=img.device),
        "input_img_shape": torch.tensor([input_h, input_w], dtype=torch.float32, device=img.dev
45   ice),
        "input_coord_offset": torch.from_numpy(input_offset[None]).to(device=img.device, dtype=
torch.float32),
    }
    render_proj_trans = None
50   if args.smplx_input_mode in (

```

```

        "det_tangent",
        "det_undistort_contiguous",
    ):
        model_core = demoer.model.module if hasattr(demoer.model, "module") else demoer.model

5    1

        sampler = model_core.encoder.fisheye2sphere
        patch_rows = int(sampler.patch_num_vertical)
        patch_columns = int(sampler.selected_patch_indices.numel() // patch_rows)
        if patch_rows * patch_columns != 192:
10         raise ValueError(
            "Detection-aligned sampling expects the checkpoint's "
            "192-token 16-row x 12-column ViT layout; "
            f"got rows={patch_rows}, columns={patch_columns}"
        )
15     if args.smplx_input_mode == "det_tangent":
        sampling_grid_pixels = create_bbox_tangent_pixel_grid(
            sampler,
            fisheye_cam,
            sampling_bbox_xyxy,
20         patch_rows=patch_rows,
            patch_columns=patch_columns,
            patch_size=args.det_tangent_patch_size,
        )
    else:
25         patch_h, patch_w = [
            int(value)
            for value in sampler.patches_2d_per_camera_pixels.shape[-3:-1]
        ]
        sampling_grid_pixels = create_contiguous_bbox_pixel_grid(
30         sampling_bbox_xyxy,
            patch_rows=patch_rows,
            patch_columns=patch_columns,
            patch_h=patch_h,
            patch_w=patch_w,
35         )
        sampler.patches_2d_per_camera_pixels = torch.from_numpy(
            sampling_grid_pixels[None]
        ).to(device=img.device, dtype=img.dtype)
        sampler.selected_patch_indices = torch.arange(
40         sampling_grid_pixels.shape[0],
            device=img.device,
            dtype=torch.long,
        )
    with torch.no_grad():
45         sampled_vit_patches = sampler(
            img,
            camera_ids=meta_info["camera_id"],
            input_img_shapes=meta_info["input_img_shape"],
            input_coord_offsets=meta_info["input_coord_offset"],
50         )

```

```

patch_prefix = f'{image_path.stem}_{bbox_id:02d}'
np.save(
    vit_patch_dir / f'{patch_prefix}_sampling_grid_pixels.npy',
    sampling_grid_pixels,
5
)
if args.smplx_input_mode == "det_tangent":
    sampling_overlay = draw_tangent_sampling_grid(
        original_img,
        sampling_grid_pixels,
10
        sampling_bbox_xyxy,
    )
else:
    sampling_overlay = draw_contiguous_sampling_grid(
        detector_img,
15
        sampling_grid_pixels,
        sampling_bbox_xyxy,
    )
sampling_grid_out = vit_patch_dir / f'{patch_prefix}_sampling_grid.jpg'
patch_atlas_out = vit_patch_dir / f'{patch_prefix}_patch_atlas.jpg'
20
cv2.imwrite(str(sampling_grid_out), rgb_to_bgr_u8(sampling_overlay))
if args.smplx_input_mode == "det_tangent":
    save_vit_patch_atlas(
        sampled_vit_patches,
        patch_atlas_out,
25
        rows=patch_rows,
        columns=patch_columns,
    )
else:
    save_vit_patch_atlas(
30
        sampled_vit_patches,
        patch_atlas_out,
        rows=patch_rows,
        columns=patch_columns,
        gap=0,
35
    )
patch_grid_out = (
    vit_patch_dir
    / f'{patch_prefix}_patch_atlas_grid.jpg'
)
40
save_vit_patch_atlas(
    sampled_vit_patches,
    patch_grid_out,
    rows=patch_rows,
    columns=patch_columns,
45
    gap=0,
    grid_color_rgb=(255, 0, 0),
)
saved_contiguous_patch_grid_images.append(
    patch_grid_out
50
)

```

```

        saved_sampling_grid_images.append(sampling_grid_out)
        saved_patch_atlas_images.append(patch_atlas_out)
    positionnet_capture.clear()
    with torch.no_grad():
5         out = demoer.model({"img": img, "img_ori": img}, {}, meta_info, "test")
    body_joint_pixels_undistorted = None
    if args.smplx_input_mode in (
        "det_tangent",
        "det_undistort_contiguous",
10    ):
        if not {"body_joint_hm", "body_joint_img"}.issubset(positionnet_capture):
            raise RuntimeError("PositionNet forward hook did not capture its outputs")
        body_joint_hm = positionnet_capture["body_joint_hm"].numpy()[0]
        body_joint_img = positionnet_capture["body_joint_img"].numpy()[0]
15        body_joint_pixels_sampling, patch_center_grid_pixels = (
            positionnet_coords_to_sampling_grid(
                body_joint_img,
                sampling_grid_pixels,
                patch_rows=patch_rows,
20                patch_columns=patch_columns,
            )
        )
        if args.smplx_input_mode == "det_undistort_contiguous":
            body_joint_pixels_undistorted = body_joint_pixels_sampling
            body_joint_pixels = undistorted_pixels_to_original(
25                body_joint_pixels_undistorted,
                undistort_model,
            )
            positionnet_canvas = detector_img
            positionnet_canvas_points = body_joint_pixels_undistorted
30        else:
            body_joint_pixels = body_joint_pixels_sampling
            positionnet_canvas = original_img
            positionnet_canvas_points = body_joint_pixels
35        positionnet_prefix = f"{image_path.stem}_{bbox_id:02d}"
        positionnet_payload = {
            "body_joint_hm": body_joint_hm.astype(np.float32),
            "body_joint_img": body_joint_img.astype(np.float32),
            "body_joint_xy_original_from_grid": body_joint_pixels.astype(np.float32),
40            "patch_center_grid_pixels": patch_center_grid_pixels.astype(np.float32),
            "body_joint_names": np.asarray(smpl_x.pos_joints_name[:25]),
            "sampling_bbox_xyxy": sampling_bbox_xyxy.astype(np.float32),
            "output_hm_shape": np.asarray(cfg.output_hm_shape, dtype=np.int32),
        }
45        if body_joint_pixels_undistorted is not None:
            positionnet_payload[
                "body_joint_xy_global_undistorted"
            ] = body_joint_pixels_undistorted.astype(np.float32)
            positionnet_payload[
50                "global_undistorted_new_k"
            ]

```

```

        ] = undistort_model["new_k"].astype(np.float64)
    np.savez_compressed(
        positionnet_dir / f"{positionnet_prefix}_positionnet.npz",
        **positionnet_payload,
5    )
    positionnet_vis = draw_positionnet_result(
        positionnet_canvas,
        body_joint_hm,
        positionnet_canvas_points,
10    sampling_bbox_xyxy,
    )
    positionnet_vis_out = positionnet_dir / f"{positionnet_prefix}_positionnet.jpg"
    cv2.imwrite(str(positionnet_vis_out), rgb_to_bgr_u8(positionnet_vis))
    saved_positionnet_images.append(positionnet_vis_out)
15    mesh = out["smplx_mesh_cam"].detach().cpu().numpy()[0]
    network_translation = out["cam_trans"].detach()[0]
    render_translation = network_translation
    network_global_orient = out["smplx_root_pose"].detach()[0]
    render_global_orient = network_global_orient.clone()
20    identity_rotation = torch.eye(
        3,
        device=network_global_orient.device,
        dtype=network_global_orient.dtype,
    )
25    global_orient_delta_matrix = identity_rotation.clone()
    camera_rotation_delta = identity_rotation.clone()
    global_orient_delta_degrees = 0.0
    global_orient_mesh_consistency_max_error_m = None
    global_orient_stage_mean_error_px = None
30    body_targets_px = None
    body_projection_initial_px = None
    body_projection_refined_px = None
    body_projection_model_px = None
    projection_consistency_max_error_px = None
35    camera_fit = None
    if args.smplx_input_mode in (
        "full_image",
        "det_tangent",
        "det_undistort_contiguous",
40    ):
        # smplx_joint_cam_pred is root-relative and is intended for
        # 3D evaluation. Rebuild the absolute local joints from the
        # predicted SMPL-X parameters before adding cam_trans; these
        # are the joints used by get_coord to create joint_proj.
45    zero_eye_pose = torch.zeros_like(out["smplx_jaw_pose"])
    with torch.no_grad():
        model_core = demoer.model.module if hasattr(demoer.model, "module") else demoer.model

1    reconstructed_smplx = model_core.smplx_layer(
50    betas=out["smplx_shape"],

```

```

        body_pose=out["smplx_body_pose"],
        global_orient=out["smplx_root_pose"],
        right_hand_pose=out["smplx_rhand_pose"],
        left_hand_pose=out["smplx_lhand_pose"],
5         jaw_pose=out["smplx_jaw_pose"],
        leye_pose=zero_eye_pose,
        reye_pose=zero_eye_pose,
        expression=out["smplx_expr"],
    )
10    body_joints_local = reconstructed_smplx.joints[0, smplx.joint_idx[:25], :]
    body_joints_local_render = body_joints_local
    if args.smplx_input_mode in (
        "det_tangent",
        "det_undistort_contiguous",
15    ):
        body_targets_px = body_joint_pixels
    else:
        joint_img = out["joint_img"].detach().cpu().numpy()[0]
        body_targets_px = heatmap_joints_to_original_pixels(
20         joint_img[:25, :2], (input_h, input_w), input_offset
        )
    with torch.no_grad():
        body_projection_initial = fisheye_cam.world2camera_pytorch(
            body_joints_local + network_translation[None, :]
25         )
        body_projection_initial_px = body_projection_initial.cpu().numpy()
        body_projection_refined_px = body_projection_initial_px.copy()
        body_projection_model_px = heatmap_joints_to_original_pixels(
            out["smplx_joint_proj"].detach().cpu().numpy()[0, :25, :2],
30         (input_h, input_w),
            input_offset,
        )
        projection_consistency_max_error_px = float(
            np.linalg.norm(
35         body_projection_model_px - body_projection_initial_px,
            axis=1,
        ).max()
        )
        projection_consistency_errors.append(projection_consistency_max_error_px)
40    if args.refine_camera_from_positionnet:
        camera_fit_fn = (
            refine_camera_translation
            if args.camera_fit_mode == "translation"
            else refine_camera_extrinsics
45        )
        camera_fit = camera_fit_fn(
            body_joints_local,
            body_targets_px,
            network_translation,
50         fisheye_cam,

```

```

        args.camera_fit_iters,
        args.camera_fit_lr,
    )
    render_rotation = camera_fit["rotation"]
5    render_translation = camera_fit["translation"]
    if args.camera_fit_mode == "translation_global_orient":
        global_orient_stage_mean_error_px = camera_fit[
            "refined_mean_error_px"
        ]
10    global_orient_delta_matrix = render_rotation
    render_global_orient = compose_global_orient(
        render_rotation,
        network_global_orient,
    )
15    cosine = ((torch.trace(render_rotation) - 1.0) * 0.5).clamp(-1.0, 1.0)
    global_orient_delta_degrees = float(
        torch.rad2deg(torch.acos(cosine)).item()
    )
    with torch.no_grad():
20        refined_smplx = model_core.smplx_layer(
            betas=out["smplx_shape"],
            body_pose=out["smplx_body_pose"],
            global_orient=render_global_orient[None, :],
            right_hand_pose=out["smplx_rhand_pose"],
25            left_hand_pose=out["smplx_lhand_pose"],
            jaw_pose=out["smplx_jaw_pose"],
            leye_pose=zero_eye_pose,
            reye_pose=zero_eye_pose,
            expression=out["smplx_expr"],
30        )
    body_joints_local_render = refined_smplx.joints[
        0, smpl_x.joint_idx[:25], :
    ]
    translation_refit = refine_camera_translation(
35        body_joints_local_render,
        body_targets_px,
        render_translation,
        fisheye_cam,
        args.camera_fit_iters,
40        args.camera_fit_lr,
    )
    render_translation = translation_refit["translation"]
    camera_fit["translation"] = render_translation
    camera_fit["refined_projection"] = translation_refit[
45        "refined_projection"
    ]
    camera_fit["refined_mean_error_px"] = translation_refit[
        "refined_mean_error_px"
    ]
50    mesh = (

```

```

        refined_smplx.vertices[0]
        + render_translation[None, :]
    ).detach().cpu().numpy()
    mesh_from_refined_params = mesh.copy()
5    global_orient_mesh_consistency_max_error_m = float(
        np.linalg.norm(
            mesh - mesh_from_refined_params,
            axis=1,
        ).max()
10    )
    elif args.camera_fit_mode == "extrinsics":
        camera_rotation_delta = render_rotation
        body_joints_local_render = (
            body_joints_local @ render_rotation.t()
15    )
    else:
        body_joints_local_render = body_joints_local
    with torch.no_grad():
        body_projection_refined = fisheye_cam.world2camera_pytorch(
20        body_joints_local_render
            + render_translation[None, :]
        )
    body_projection_refined_px = body_projection_refined.cpu().numpy()
    if args.camera_fit_mode != "translation_global_orient":
25        network_translation_np = network_translation.cpu().numpy()
        render_translation_np = render_translation.cpu().numpy()
        mesh = (
            (mesh - network_translation_np[None, :])
            @ render_rotation.cpu().numpy().T
30        + render_translation_np[None, :]
        )
    debug_image = draw_camera_fit_debug(
        original_img,
        body_targets_px,
35        body_projection_initial_px,
        body_projection_refined_px,
        camera_fit["initial_mean_error_px"],
        camera_fit["refined_mean_error_px"],
    )
40    camera_fit_out = debug_dir / f'{image_path.stem}_{bbox_id:02d}_camera_fit.jpg'
    cv2.imwrite(str(camera_fit_out), rgb_to_bgr_u8(debug_image))
    saved_camera_fit_images.append(camera_fit_out)
    if body_targets_px is not None:
        joint_compare_image = draw_positionnet_smplx_2d_comparison(
45        original_img,
            body_targets_px,
            body_projection_refined_px,
        )
    joint_compare_out = (
50    joint_compare_dir

```

```

        / f"{image_path.stem}_{bbox_id:02d}_positionnet_vs_smplx.jpg"
    )
    cv2.imwrite(
        str(joint_compare_out),
5        rgb_to_bgr_u8(joint_compare_image),
    )
    saved_joint_compare_images.append(joint_compare_out)
    smplx_pred = {
        "global_orient": render_global_orient.reshape(-1, 3).detach().cpu().numpy(),
10        "global_orient_network": network_global_orient.reshape(-1, 3).detach().cpu().numpy(),
        "global_orient_delta_matrix": global_orient_delta_matrix.reshape(1, 3, 3).detach().cpu().numpy(),
    y(),
        "body_pose": out["smplx_body_pose"].reshape(-1, 3).detach().cpu().numpy(),
        "left_hand_pose": out["smplx_lhand_pose"].reshape(-1, 3).detach().cpu().numpy(),
15        "right_hand_pose": out["smplx_rhand_pose"].reshape(-1, 3).detach().cpu().numpy(),
        "jaw_pose": out["smplx_jaw_pose"].reshape(-1, 3).detach().cpu().numpy(),
        "leye_pose": np.zeros((1, 3), dtype=np.float32),
        "reye_pose": np.zeros((1, 3), dtype=np.float32),
        "betas": out["smplx_shape"].reshape(-1, 10).detach().cpu().numpy(),
20        "expression": out["smplx_expr"].reshape(-1, 10).detach().cpu().numpy(),
        "transl": render_translation.reshape(-1, 3).detach().cpu().numpy(),
        "transl_network": network_translation.reshape(-1, 3).detach().cpu().numpy(),
        "camera_rotation_delta": camera_rotation_delta.reshape(1, 3, 3).detach().cpu().numpy(),
        "det_bbox_xyxy_score": np.asarray(det_bbox).reshape(1, 5).astype(np.float32),
25        "smplx_joint_cam_root_relative": out["smplx_joint_cam_pred"][0].detach().cpu().numpy().astype(np.float32),
        "smplx_mesh_cam": mesh.astype(np.float32),
    }
    if body_targets_px is not None:
30        body_joints_camera_network = body_joints_local + network_translation[None, :]
        if camera_fit is not None:
            body_joints_camera_render = (
                body_joints_local_render + render_translation[None, :]
            )
35        else:
            body_joints_camera_render = body_joints_camera_network
        smplx_pred["smplx_body_joints_3d_local"] = body_joints_local.detach().cpu().numpy().astype(np.float32)
        if args.camera_fit_mode == "translation_global_orient" and camera_fit is not None:
40            smplx_pred["smplx_body_joints_3d_local_refined_global_orient"] = (
                body_joints_local_render
            ).detach().cpu().numpy().astype(np.float32)
            smplx_pred["smplx_body_joints_3d_network_cam"] = body_joints_camera_network.detach().cpu().numpy().astype(np.float32)
45            smplx_pred["smplx_body_joints_3d_render_cam"] = body_joints_camera_render.detach().cpu().numpy().astype(np.float32)
            smplx_pred["positionnet_body_joints_2d"] = body_targets_px.astype(np.float32)
            if body_joint_pixels_undistorted is not None:
                smplx_pred[
50                    "positionnet_body_joints_2d_global_undistorted"

```

```

    ] = body_joint_pixels_undistorted.astype(np.float32)
    smplx_pred["smplx_body_joints_2d_network_cam"] = body_projection_initial_px.astype(np.f
loat32)
    smplx_pred["smplx_body_joints_2d_render_cam"] = body_projection_refined_px.astype(np.fl
5    oat32)
    smplx_pred["smplx_body_joints_2d_model_output"] = body_projection_model_px.astype(np.
float32)
    smplx_pred["projection_consistency_max_error_px"] = np.asarray(
        projection_consistency_max_error_px, dtype=np.float32
10    )
    np.savez(smplx_dir / f"{image_path.stem}_{bbox_id:02d}_smplx.npz", **smplx_pred)
    if args.save_mesh:
        save_obj(mesh, smpl_x.face, str(mesh_dir / f"{image_path.stem}_{bbox_id:02d}.obj"))
    vis_img = render_mesh_fisheye(
15        vis_img,
        mesh,
        smpl_x.face,
        {"focal": [1.0, 1.0], "princpt": [0.0, 0.0]},
        fisheye_cam,
20        mesh_as_vertices=args.show_verts,
        mode=args.mesh_mode,
        proj_trans=render_proj_trans,
    )
    if wireframe_img is not None:
25        wireframe_img = render_mesh_fisheye(
            wireframe_img,
            mesh,
            smpl_x.face,
            {"focal": [1.0, 1.0], "princpt": [0.0, 0.0]},
30            fisheye_cam,
            mesh_as_vertices=False,
            mode="wireframe",
            proj_trans=render_proj_trans,
        )
35        if body_targets_px is not None:
            mesh_joint_compare_image = draw_positionnet_smplx_2d_comparison(
                wireframe_img,
                body_targets_px,
                body_projection_refined_px,
40            )
            mesh_joint_compare_out = (
                mesh_joint_compare_dir
                / f"{image_path.stem}_{bbox_id:02d}_mesh_positionnet_smplx.jpg"
            )
45            cv2.imwrite(
                str(mesh_joint_compare_out),
                rgb_to_bgr_u8(mesh_joint_compare_image),
            )
            saved_mesh_joint_compare_images.append(mesh_joint_compare_out)
50        if args.show_bbox and not args.no_detector:

```

```

if args.smplx_input_mode == "det_undistort_contiguous":
    det_corners_undistorted = np.asarray(
        [[x1, y1], [x2, y1], [x2, y2], [x1, y2]],
        dtype=np.float32,
5    )
    det_corners_original = undistorted_pixels_to_original(
        det_corners_undistorted,
        undistort_model,
    )
10    det_corners_int = np rint(
        det_corners_original
    ).astype(np.int32)
    cv2.polylines(
        vis_img,
15    [det_corners_int],
        isClosed=True,
        color=(255, 0, 0),
        thickness=3,
        lineType=cv2.LINE_AA,
20    )
    label_point = tuple(det_corners_int[0])
else:
    pt1 = (int(round(x1)), int(round(y1)))
    pt2 = (int(round(x2)), int(round(y2)))
25    cv2.rectangle(vis_img, pt1, pt2, (255, 0, 0), 3)
    label_point = pt1
    cv2.putText(
        vis_img,
        f"person {score:.2f}",
30    (label_point[0], max(25, label_point[1] - 10)),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (255, 0, 0),
        2,
35    cv2.LINE_AA,
    )
meta = {
    "image_path": str(image_path.resolve()),
    "bbox_id": int(bbox_id),
40    "det_bbox_xyxy_score": np.asarray(det_box).astype(float).tolist(),
    "bbox_processed_xywh": bbox.tolist(),
    "img2bb_trans": img2bb_trans.tolist(),
    "bb2img_trans": bb2img_trans.tolist(),
    "fisheye_calibration": str(args.fisheye_calibration.resolve()),
45    "checkpoint_path": str(args.checkpoint_path.resolve()),
    "config_path": str(args.config_path.resolve()),
    "smplx_input_mode": args.smplx_input_mode,
    "detector_coordinate_space": (
        "global_undistorted"
50    if args.smplx_input_mode == "det_undistort_contiguous"

```

```

        else "original_fisheye"
    ),
    "input_crop_mode": args.input_crop_mode,
    "input_crop_xyxy": input_crop_xyxy if args.smplx_input_mode != "det_crop" else None,
5    "input_coord_offset": input_offset.astype(float).tolist() if args.smplx_input_mode != "det_crop"
    " else [0.0, 0.0],
    "det_tangent_sampling_bbox_xyxy": (
        sampling_bbox_xyxy.astype(float).tolist()
        if sampling_bbox_xyxy is not None
10    else None
    ),
    "sampling_bbox_coordinate_space": (
        "global_undistorted"
        if args.smplx_input_mode == "det_undistort_contiguous"
15    else (
        "original_fisheye"
        if sampling_bbox_xyxy is not None
        else None
    )
20    ),
    "det_tangent_patch_grid_shape": (
        list(sampling_grid_pixels.shape)
        if sampling_grid_pixels is not None
        else None
25    ),
    "det_tangent_patch_size": (
        float(args.det_tangent_patch_size)
        if args.smplx_input_mode == "det_tangent"
        else None
30    ),
    "positionnet_2d_mapping": (
        "bilinear_tangent_patch_centers"
        if args.smplx_input_mode == "det_tangent"
        else (
35    "continuous_undistorted_grid_then_brown_conrady_to_original"
        if args.smplx_input_mode == "det_undistort_contiguous"
        else "heatmap_to_full_image"
    )
    ),
    "global_undistort_alpha": (
40    float(args.global_undistort_alpha)
        if args.smplx_input_mode == "det_undistort_contiguous"
        else None
    ),
    "contiguous_roi_mode": (
45    args.contiguous_roi_mode
        if args.smplx_input_mode == "det_undistort_contiguous"
        else None
    ),
50    "global_undistorted_new_k": (

```

```

        undistort_model["new_k"].astype(float).tolist()
        if args.smplx_input_mode == "det_undistort_contiguous"
        else None
    ),
5    "camera_translation_network": network_translation.detach().cpu().numpy().astype(float).tolist
    (,
        "camera_translation_render": render_translation.detach().cpu().numpy().astype(float).tolist(),
        "global_orient_network": network_global_orient.detach().cpu().numpy().astype(float).tolist(),
        "global_orient_render": render_global_orient.detach().cpu().numpy().astype(float).tolist(),
10    "global_orient_delta_matrix": global_orient_delta_matrix.detach().cpu().numpy().astype(float)
        .tolist(),
        "global_orient_delta_degrees": float(global_orient_delta_degrees),
        "global_orient_mesh_consistency_max_error_m": (
            global_orient_mesh_consistency_max_error_m
15    ),
        "global_orient_stage_mean_error_px": (
            global_orient_stage_mean_error_px
        ),
        "camera_rotation_delta": camera_rotation_delta.detach().cpu().numpy().astype(float).tolist(),
20    "camera_refined_from_positionnet": bool(camera_fit is not None),
        "camera_fit_mode": args.camera_fit_mode if camera_fit is not None else None,
        "projection_consistency_max_error_px": projection_consistency_max_error_px,
    }
    if camera_fit is not None:
25        meta["camera_fit_joint_indices"] = camera_fit["joint_indices"].astype(int).tolist()
        meta["camera_fit_initial_mean_error_px"] = camera_fit["initial_mean_error_px"]
        meta["camera_fit_refined_mean_error_px"] = camera_fit["refined_mean_error_px"]
        (meta_dir / f"{image_path.stem}_{bbox_id:02d}.json").write_text(json.dumps(meta, indent=2) +
        "\n")
30    persons_projected += 1
    total_persons += 1
    torch.cuda.empty_cache()
    out_img = overlay_dir / f"{image_path.stem}_det_mesh_overlay.jpg"
    cv2.imwrite(str(out_img), rgb_to_bgr_u8(vis_img))
35    saved_images.append(out_img)
    if wireframe_img is not None:
        wireframe_out = wireframe_dir / f"{image_path.stem}_det_mesh_wireframe.jpg"
        cv2.imwrite(str(wireframe_out), rgb_to_bgr_u8(wireframe_img))
        saved_wireframe_images.append(wireframe_out)
40    if video_writer is not None:
        video_writer.write(rgb_to_bgr_u8(vis_img))
        records.append(**det_record, "persons_projected": int(persons_projected))
    if video_writer is not None:
        video_writer.release()
45    save_contact_sheet(saved_images, output_dir / "contact_sheet.jpg", max_images=args.contact_sheet_li
        mit)
    if saved_wireframe_images:
        save_contact_sheet(
            saved_wireframe_images,
50    output_dir / "contact_sheet_wireframe.jpg",

```

```
        max_images=args.contact_sheet_limit,
    )
    if saved_sampling_grid_images:
        save_contact_sheet(
5            saved_sampling_grid_images,
            output_dir / "contact_sheet_vit_sampling_grid.jpg",
            max_images=args.contact_sheet_limit,
        )
    if saved_patch_atlas_images:
10        save_contact_sheet(
            saved_patch_atlas_images,
            output_dir / "contact_sheet_vit_patch_atlas.jpg",
            max_images=args.contact_sheet_limit,
        )
    if saved_contiguous_patch_grid_images:
15        save_contact_sheet(
            saved_contiguous_patch_grid_images,
            output_dir / "contact_sheet_vit_patch_atlas_grid.jpg",
            max_images=args.contact_sheet_limit,
20        )
    if saved_undistorted_images:
        save_contact_sheet(
            saved_undistorted_images,
            output_dir / "contact_sheet_global_undistorted.jpg",
25        max_images=args.contact_sheet_limit,
        )
    if saved_undistorted_detection_images:
        save_contact_sheet(
            saved_undistorted_detection_images,
30        output_dir / "contact_sheet_global_undistorted_detection.jpg",
            max_images=args.contact_sheet_limit,
        )
    if saved_positionnet_images:
        save_contact_sheet(
35        saved_positionnet_images,
            output_dir / "contact_sheet_positionnet.jpg",
            max_images=args.contact_sheet_limit,
        )
    if saved_camera_fit_images:
40        save_contact_sheet(
            saved_camera_fit_images,
            output_dir / "contact_sheet_camera_fit.jpg",
            max_images=args.contact_sheet_limit,
        )
    if saved_joint_compare_images:
45        save_contact_sheet(
            saved_joint_compare_images,
            output_dir / "contact_sheet_positionnet_vs_smplx_2d.jpg",
            max_images=args.contact_sheet_limit,
50        )
```

```

if saved_mesh_joint_compare_images:
    save_contact_sheet(
        saved_mesh_joint_compare_images,
        output_dir / "contact_sheet_mesh_positionnet_smplx_2d.jpg",
5        max_images=args.contact_sheet_limit,
    )
summary = {
    "image_dir": str(args.image_dir.resolve()),
    "image_glob": args.image_glob,
10    "processed_images": int(len(image_paths)),
    "images_without_detection": int(no_detection),
    "projected_persons": int(total_persons),
    "multi_person": bool(args.multi_person),
    "detector_enabled": not args.no_detector,
15    "score_thr": float(args.score_thr),
    "bbox_thr": float(args.bbox_thr),
    "smplx_input_mode": args.smplx_input_mode,
    "det_tangent_expand": float(args.det_tangent_expand),
    "det_tangent_patch_size": float(args.det_tangent_patch_size),
20    "global_undistort_alpha": float(args.global_undistort_alpha),
    "contiguous_roi_mode": (
        args.contiguous_roi_mode
        if args.smplx_input_mode == "det_undistort_contiguous"
        else None
25    ),
    "global_undistorted_source_k": (
        undistort_model["source_k"].astype(float).tolist()
        if undistort_model is not None
        else None
30    ),
    "global_undistorted_new_k": (
        undistort_model["new_k"].astype(float).tolist()
        if undistort_model is not None
        else None
35    ),
    "global_undistorted_distortion": (
        undistort_model["distortion"].astype(float).tolist()
        if undistort_model is not None
        else None
40    ),
    "global_undistorted_valid_roi": (
        list(undistort_model["valid_roi"])
        if undistort_model is not None
        else None
45    ),
    "input_crop_mode": args.input_crop_mode,
    "horizontal_crop_pixels": int(args.horizontal_crop_pixels),
    "camera_refined_from_positionnet": bool(args.refine_camera_from_positionnet),
    "camera_fit_mode": args.camera_fit_mode,
50    "camera_fit_iters": int(args.camera_fit_iters),

```

```

        "camera_fit_lr": float(args.camera_fit_lr),
        "save_wireframe_overlay": bool(args.save_wireframe_overlay),
        "projection_consistency_max_error_px": (
            float(max(projection_consistency_errors))
5            if projection_consistency_errors
            else None
        ),
        "output_dir": str(output_dir.resolve()),
        "fisheye_calibration": str(args.fisheye_calibration.resolve()),
10        "checkpoint_path": str(args.checkpoint_path.resolve()),
        "det_config": str(args.det_config.resolve()),
        "det_checkpoint": str(args.det_checkpoint.resolve()),
        "elapsed_sec": float(time.time() - start_time),
    }
15    (output_dir / "manifest.json").write_text(json.dumps(summary, indent=2) + "\n")
    (output_dir / "records.json").write_text(json.dumps(records, indent=2) + "\n")
    print(json.dumps(summary, indent=2))
    sys.stdout.flush()
    sys.stderr.flush()
20    os._exit(0)
if __name__ == "__main__":
    main()

```

文件名: egosmplx_pipeline/sapiens_pose.py

功能: Single-person Sapiens2 inference using the installed official pipeline.

```

25    #!/usr/bin/env python3
    """Single-person Sapiens2 inference using the installed official pipeline.
    An explicit box or the full image is used; no manual keypoint annotations are
    read. Output keypoints are in original image pixels, with uncalibrated scores.
    """
30    import argparse
    import hashlib
    import json
    import sys
    import time
35    from pathlib import Path
    def main():
        p = argparse.ArgumentParser(description=__doc__)
        p.add_argument('--repo-root', type=Path, required=True, help='Sapiens2 checkout')
        p.add_argument('--dependencies', type=Path)
40        p.add_argument('--checkpoint', type=Path, required=True)
        p.add_argument('--checkpoint-sha256', required=True)
        p.add_argument('--image', type=Path, required=True)
        p.add_argument('--output-dir', type=Path, required=True)
        p.add_argument('--size', choices=['1b', '5b'], default='1b')
45        p.add_argument('--bbox', nargs=4, type=float, help='Original image xyxy; default full image')
        a = p.parse_args()
        root = a.repo_root.resolve()
        repo = root
        sys.path.insert(0, str(repo))
50        if a.dependencies:

```

```

        sys.path.insert(0, str(a.dependencies))
import cv2
import numpy as np
import torch
5   from safetensors.torch import load_file
    from sapiens.engine.config import Config
    from sapiens.engine.datasets import Compose
    from sapiens.pose.datasets import parse_pose_metainfo
    from sapiens.pose.datasets.codecs.udp_heatmap import UDPHeatmap
10   from sapiens.registry import MODELS
    im = cv2.imread(str(a.image))
    if im is None:
        raise ValueError(f'Unreadable image: {a.image}')
    h, w = im.shape[:2]
15   bbox = a.bbox or [0, 0, w, h]
    if not (0 <= bbox[0] < bbox[2] <= w and 0 <= bbox[1] < bbox[3] <= h):
        raise ValueError('bbox must lie inside the original image')
    a.output_dir.mkdir(parents=True, exist_ok=False)
    weight = a.checkpoint
20   digest = hashlib.sha256()
    with weight.open('rb') as f:
        for block in iter(lambda: f.read(8 * 1024**2), b''):
            digest.update(block)
    if digest.hexdigest() != a.checkpoint_sha256:
25   raise ValueError('Pose checkpoint SHA-256 mismatch')
    config = repo / f'sapiens/pose/configs/keypoints308/shutterstock_goliath_3po/sapiens2_{a.size}_keypoi
nts308_shutterstock_goliath_3po-1024x768.py'
    cfg = Config.fromfile(config)
    cfg.model['backbone'].pop('init_cfg', None)
30   print('Building official pose model', flush=True)
    model = MODELS.build(cfg.model)
    state = load_file(str(weight), device='cpu')
    model.load_state_dict(state, strict=True)
    tensor_count = len(state)
35   del state
    model = model.cuda().eval()
    preprocessor = MODELS.build(cfg.data_preprocessor).cuda()
    pipeline = Compose(cfg.test_pipeline)
    meta = parse_pose_metainfo(dict(from_file=str(repo / 'sapiens/pose/configs/_base_/keypoints308.py')))
40   names = [meta['keypoint_id2name'][i] for i in range(308)]
    codec = UDPHeatmap(**{k: v for k, v in cfg.codec.items() if k != 'type'})
    data = preprocessor(pipeline(dict(img=im, bbox=np.array([bbox], dtype=np.float32), bbox_score=np.on
es(1, dtype=np.float32))))
    inputs, sample = data['inputs'], data['data_samples']
45   torch.cuda.synchronize()
    start = time.time()
    with torch.inference_mode(), torch.autocast('cuda', dtype=torch.bfloat16):
        heatmaps = model(inputs)
        if cfg.val_cfg.get('flip_test', False):
50   flipped = model(inputs.flip(-1)).flip(-1)[:, meta['flip_indices']]

```

```

        heatmaps = (heatmaps + flipped) / 2
        heatmaps = heatmaps.float().cpu().numpy()
        xy, scores = codec.decode(heatmaps[0])
        xy = xy / sample['meta']['input_size'] * sample['meta']['bbox_scale'] + sample['meta']['bbox_center'] - 0.5
5    * sample['meta']['bbox_scale']
        xy, scores = xy[0], scores[0]
        assert xy.shape == (308, 2) and np.isfinite(xy).all() and np.isfinite(scores).all()
        joints = [dict(index=i, name=n, x=float(xy[i, 0]), y=float(xy[i, 1]), score=float(scores[i]), inside_image=
bool(0 <= xy[i, 0] < w and 0 <= xy[i, 1] < h)) for i, n in enumerate(names)]
10    payload = dict(model=f'Sapiens2-{a.size.upper()}', image_path=str(a.image.resolve()),
        image_sha256=hashlib.sha256(a.image.read_bytes()).hexdigest(),
        image_width=w, image_height=h, joint_set='sociopticon308',
        coordinate_space='original_image_pixels', bbox_xyxy=bbox,
        visibility='unknown; heatmap scores are not calibrated visibility',
15    keypoints=joints, inference_seconds=time.time() - start)
    (a.output_dir / 'keypoints.json').write_text(json.dumps(payload, indent=2) + '\n')
    np.savez_compressed(a.output_dir / 'heatmaps.npz', heatmap=heatmaps[0])
    body = im.copy()
    for j in joints:
20        if j['name'] in {'nose', 'left_shoulder', 'right_shoulder', 'left_elbow', 'right_elbow', 'left_wrist', 'right_wri
st', 'left_hip', 'right_hip', 'left_knee', 'right_knee', 'left_ankle', 'right_ankle'} and j['inside_image']:
            pt = (round(j['x']), round(j['y']))
            color = (0, 220, 0) if j['score'] >= 0.3 else (0, 150, 255)
            cv2.circle(body, pt, 5, color, -1, cv2.LINE_AA)
25    cv2.putText(body, f'{j["name"]} {j["score"]:.2f}', (pt[0] + 5, pt[1] - 6), cv2.FONT_HERSHEY_SIM
PLEX, .4, color, 1, cv2.LINE_AA)
    cv2.imwrite(str(a.output_dir / 'body_keypoints.jpg'), body)
    manifest = dict(checkpoint=str(weight), checkpoint_sha256=digest.hexdigest(),
        configured_sha256_verified=True, strict_load=True, checkpoint_tensors=tensor_count,
30    config=str(config), python=sys.executable, torch=torch.__version__,
        input_size_hw=[1024, 768], precision='bf16',
        flip_test=cfg.val_cfg.get('flip_test', False), manual_labels_used=False,
        gpu=torch.cuda.get_device_name(), peak_gpu_memory_bytes=torch.cuda.max_memory_alloc
ated())
35    (a.output_dir / 'manifest.json').write_text(json.dumps(manifest, indent=2) + '\n')
    print(json.dumps(payload, indent=2))
    if __name__ == '__main__':
        main()
文件名: egosmplx_pipeline/sapiens_seg.py
40 功能: 项目程序模块。
    from pathlib import Path
    import sys, json, time, hashlib
    import argparse
    parser = argparse.ArgumentParser(description='Strict Sapiens2 29-class segmentation')
45    parser.add_argument('--repo-root', type=Path, required=True)
    parser.add_argument('--dependencies', type=Path)
    parser.add_argument('--checkpoint', type=Path, required=True)
    parser.add_argument('--checkpoint-sha256', required=True)
    parser.add_argument('--manifest', type=Path, required=True)
50    parser.add_argument('--output-dir', type=Path, required=True)

```

```

args=parser.parse_args()
REPO=args.repo_root;OUT=args.output_dir
OUT.mkdir(parents=True,exist_ok=False)
sys.path.insert(0,str(REPO))
5 if args.dependencies:sys.path.insert(0,str(args.dependencies))
import cv2,numpy as np,torch
import torch.nn.functional as F
from sapiens.engine.config import Config
from sapiens.engine.datasets import Compose
10 from sapiens.registry import MODELS
from sapiens.dense.visualizers import SegVisualizer
from sapiens.dense.datasets import DOME_CLASSES_29
from safetensors.torch import load_file
def write(p,x):p.write_text(json.dumps(x,indent=2,ensure_ascii=False))
15 def sha(p):
    h=hashlib.sha256()
    with p.open('rb') as f:
        for b in iter(lambda:f.read(16*1024**2),b''):h.update(b)
    return h.hexdigest()
20 def main():
    cfgpath=REPO/'sapiens/dense/configs/seg/shutterstock_goliath/sapiens2_1b_seg_shutterstock_goliath-102
4x768.py';weight=args.checkpoint
    digest=sha(weight);assert digest==args.checkpoint_sha256, 'Segmentation checkpoint SHA-256 mismatch
'
25    cfg=Config.fromfile(cfgpath);cfg.model['backbone'].pop('init_cfg',None)
    print('Building model',flush=True);model=MODELS.build(cfg.model);state=load_file(str(weight));model.
load_state_dict(state,strict=True);count=len(state);del state
    model=model.cuda().eval();prep=MODELS.build(cfg.data_preprocessor).cuda();pipe=Compose(cfg.test_
pipeline)
30    viz=SegVisualizer(with_labels=False);palette=viz.class_palette[:29,:-1].copy()
    write(OUT/'classes.json',{int(k):v for k,v in DOME_CLASSES_29.items()})
    rows=json.loads(args.manifest.read_text())['frames'];records=[]
    for r in rows:
        t=time.time();d=OUT/'frames'/r['id'];d.mkdir(parents=True,exist_ok=True);im=cv2.imread(r['image']);ass
35    ert im is not None
        data=prep(pipe(dict(img=im)));inputs=data['inputs']
        with torch.inference_mode(),torch.autocast('cuda',dtype=torch.bfloat16):logits=model(inputs)
        logits=F.interpolate(logits.float(),size=im.shape[:2],mode='bilinear',align_corners=False);prob=logits.soft
max(dim=1)[0].cpu().numpy();labels=prob.argmax(0).astype(np.uint8);confidence=prob.max(0)
40    assert prob.shape==(29,*im.shape[:2]) and np.isfinite(prob).all() and np.max(np.abs(prob.sum(0)-1))<1e-
5
        np.save(d/'labels.npy',labels);cv2.imwrite(str(d/'labels.png'),labels);np.savez_compressed(d/'probabilities.
npz',probabilities=prob.astype(np.float16),class_ids=np.arange(29),coordinate_space=np.array('original_im
age_pixels'))
45    cv2.imwrite(str(d/'original.jpg'),im);cv2.imwrite(str(d/'all_parts.jpg'),viz._visualize_segmentation(im,labe
ls));cv2.imwrite(str(d/'part_colors.png'),palette[labels])
        focus=im.copy();region=np.isin(labels,[6,7,11,15,16,20]);focus[region]=(im[region]*.5+palette[labels[reg
ion]]*.5).astype(np.uint8)
        boundary=im.copy();stats={}
50    for k in [6,7,11,15,16,20]:

```

```

mask=(labels==k).astype(np.uint8);contours,_=cv2.findContours(mask,cv2.RETR_EXTERNAL,cv2.CH
AIN_APPROX_SIMPLE);color=tuple(int(x) for x in palette[k]);cv2.drawContours(boundary,contours,-1,c
olor,2)
cv2.imwrite(str(d/f'mask_{k:02d}.png'),mask*255)
5   ys,xs=np.where(mask);stats[k]=dict(name=DOME_CLASSES_29[k]['name'],pixels=int(mask.sum()),me
an_softmax=float(confidence[mask>0].mean()) if mask.any() else None,high_confidence_pixels=int(((mas
k>0)&(confidence>=.8)).sum()),bbox_xyxy=[int(xs.min()),int(ys.min()),int(xs.max()+1),int(ys.max()+1)] if
len(xs) else None)
cv2.imwrite(str(d/'arms_hands.jpg'),focus);cv2.imwrite(str(d/'boundaries.jpg'),boundary);cv2.imwrite(str(
10 d/'confidence.jpg'),cv2.applyColorMap((confidence*255).astype(np.uint8),cv2.COLORMAP_VIRIDIS))
rec=dict(id=r['id'],camera=r.get('camera','unknown'),image=r['image'],image_sha256=sha(Path(r['image']
),original_hw=list(im.shape[:2]),network_input_shape=list(inputs.shape),inference_seconds=time.time()-t,p
arts=stats,softmax_is_not_calibrated_accuracy=True)
write(d/'stats.json',rec);records.append(rec);print('DONE',r['id'],round(time.time()-t,2),{x:stats[x]['pixels']
15 for x in stats},flush=True)
write(OUT/'manifest.json',dict(frames=records,checkpoint=str(weight),checkpoint_sha256=digest,configu
red_sha256_verified=True,strict_load=True,tensor_count=count,config=str(cfgpath),config_sha256=sha(cf
gpath),python=sys.executable,torch=torch.__version__,precision='bfloat16 autocast; float32 interpolation/s
oftmax',preprocessing='official test pipeline: 1024x768 direct resize keep_ratio=False; BGR to RGB; offici
20 al normalization',postprocessing='bilinear logits resize to original H,W align_corners=False; softmax; argm
ax',manual_annotations_used=False,mesh_optimization_run=False))
if __name__=='__main__':main()
文件名: egosmplx_pipeline/wilor_predict.py
功能: Run WiLoR hand detection/reconstruction on a configurable image manifest.
25 #!/usr/bin/env python3
"""Run WiLoR hand detection/reconstruction on a configurable image manifest."""
import argparse
import json
from pathlib import Path
30 import cv2
import numpy as np
import torch
from scipy.spatial.transform import Rotation
from wilor.datasets.vitdet_dataset import ViTDetDataset
35 from wilor.models import load_wilor
from wilor.utils import recursive_to
from wilor.utils.camera import cam_crop_to_full
from wilor.utils.yolo_loader import load_yolo_detector
def parse_args():
40     ap = argparse.ArgumentParser(description=__doc__)
     ap.add_argument("--manifest", type=Path, required=True)
     ap.add_argument("--output-dir", type=Path, required=True)
     ap.add_argument("--checkpoint", type=Path, required=True)
     ap.add_argument("--config", type=Path, required=True)
45     ap.add_argument("--detector", type=Path, required=True)
     ap.add_argument("--confidence", type=float, default=0.20)
     ap.add_argument("--rescale-factor", type=float, default=2.0)
     return ap.parse_args()
def project(points, translation, focal_length, image_size_wh):
50     width, height = map(float, image_size_wh)

```

```

xyz = points + translation[None]
uv = xyz[:, :2] / np.maximum(xyz[:, 2:3], 1e-6)
uv *= float(focal_length)
uv += np.asarray([width / 2.0, height / 2.0], np.float32)
5   return uv
def write_obj(path, vertices, faces):
    with path.open("w", encoding="utf-8") as handle:
        for vertex in vertices:
            handle.write(f'v {vertex[0]} {vertex[1]} {vertex[2]}\n')
10    for face in faces:
        handle.write(f'f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n')
def main():
    args = parse_args()
    if args.output_dir.exists():
15        raise FileExistsError(args.output_dir)
    for name in ("npz", "json", "mesh", "overlay"):
        (args.output_dir / name).mkdir(parents=True, exist_ok=True)
    rows = json.loads(args.manifest.read_text())['frames']
    images = [Path(r['image']) for r in rows]
20    if len({p.stem for p in images}) != len(images):
        raise ValueError("WiLoR input stems must be unique; runner stages images by frame id")
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model, model_cfg = load_wilor(str(args.checkpoint), str(args.config))
    detector = load_yolo_detector(str(args.detector))
25    model = model.to(device).eval()
    detector = detector.to(device)
    faces = np.asarray(model.mano.faces, np.int32)
    report = {"method": "WiLoR detector + MANO regression", "frames": []}
    for image_path in images:
30        image = cv2.imread(str(image_path), cv2.IMREAD_COLOR)
        result = detector(image, conf=args.confidence, verbose=False)[0]
        boxes = result.bboxes.xyxy.detach().cpu().numpy().astype(np.float32)
        scores = result.bboxes.conf.detach().cpu().numpy().astype(np.float32)
        right = result.bboxes.cls.detach().cpu().numpy().astype(np.float32)
35        frame = {"image": str(image_path), "detections": []}
        overlay = image.copy()
        focal_value = float(model_cfg.EXTRA.FOCAL_LENGTH / model_cfg.MODEL.IMAGE_SIZE * ma
x(image.shape[:2]))
        if len(boxes) == 0:
40            np.savez_compressed(args.output_dir / 'npz' / f'{image_path.stem}_wilor.npz',
                                detector_confidence=np.empty(0, np.float32), is_right=np.empty(0, np.float32),
                                joints_2d=np.empty((0, 21, 2), np.float32), mano_faces=faces,
                                image_size_wh=np.array(image.shape[1::-1]), focal_length_px=np.array(focal_value),
                                image_sha256=np.array(__import__('hashlib').sha256(image_path.read_bytes()).hexdigest()
45            gest()))
            cv2.imwrite(str(args.output_dir / 'overlay' / f'{image_path.stem}_wilor.jpg'), overlay)
            report['frames'].append(frame)
            continue
    dataset = ViTDetDataset(model_cfg, image, boxes, right, rescale_factor=args.rescale_factor)
50    loader = torch.utils.data.DataLoader(dataset, batch_size=16, shuffle=False, num_workers=0)

```

```

arrays = {key: [] for key in (
    "bbox_xyxy", "detector_confidence", "is_right", "global_orient_aa",
    "hand_pose_aa", "betas", "cam_t", "joints_3d_local", "joints_2d",
    "vertices_3d_local",
5   )}
offset = 0
for batch in loader:
    batch = recursive_to(batch, device)
    with torch.no_grad():
10      output = model(batch)
    pred_mano = output["pred_mano_params"]
    pred_cam = output["pred_cam"].clone()
    pred_cam[:, 1] *= 2 * batch["right"] - 1
    image_size = batch["img_size"].float()
15    focal = model_cfg.EXTRA.FOCAL_LENGTH / model_cfg.MODEL.IMAGE_SIZE * image_size.
    max()
    translations = cam_crop_to_full(
        pred_cam, batch["box_center"].float(), batch["box_size"].float(),
        image_size, focal,
20    ).detach().cpu().numpy()
    vertices_batch = output["pred_vertices"].detach().cpu().numpy()
    joints_batch = output["pred_keypoints_3d"].detach().cpu().numpy()
    hand_rot = pred_mano["hand_pose"].detach().cpu().numpy()
    global_rot = pred_mano["global_orient"].detach().cpu().numpy()
25    betas = pred_mano["betas"].detach().cpu().numpy()
    right_batch = batch["right"].detach().cpu().numpy()
    size_batch = image_size.detach().cpu().numpy()
    for index in range(len(vertices_batch)):
        source_index = offset + index
30        is_right = bool(right_batch[index] > 0.5)
        vertices = vertices_batch[index].copy()
        joints = joints_batch[index].copy()
        mirror = 1.0 if is_right else -1.0
        vertices[:, 0] *= mirror
35        joints[:, 0] *= mirror
        uv = project(joints, translations[index], float(focal), size_batch[index])
        global_aa = Rotation.from_matrix(global_rot[index, 0]).as_rotvec().astype(np.float32)
        hand_aa = Rotation.from_matrix(hand_rot[index]).as_rotvec().astype(np.float32)
        arrays["bbox_xyxy"].append(boxes[source_index])
40        arrays["detector_confidence"].append(scores[source_index])
        arrays["is_right"].append(float(is_right))
        arrays["global_orient_aa"].append(global_aa)
        arrays["hand_pose_aa"].append(hand_aa)
        arrays["betas"].append(betas[index])
45        arrays["cam_t"].append(translations[index])
        arrays["joints_3d_local"].append(joints)
        arrays["joints_2d"].append(uv)
        arrays["vertices_3d_local"].append(vertices)
        rec = {
50            "index": source_index,

```

```

        "is_right": is_right,
        "confidence": float(scores[source_index]),
        "bbox_xyxy": boxes[source_index].tolist(),
        "joints_2d": uv.tolist(),
5         "global_orient_aa": global_aa.tolist(),
        "hand_pose_aa": hand_aa.tolist(),
        "betas": betas[index].tolist(),
        "cam_t": translations[index].tolist(),
    }
10     frame["detections"].append(rec)
    color = (0, 220, 255) if is_right else (255, 180, 0)
    p1 = tuple(np rint(boxes[source_index, :2]).astype(int))
    p2 = tuple(np rint(boxes[source_index, 2:]).astype(int))
    cv2.rectangle(overlay, p1, p2, color, 2)
15     cv2.putText(overlay, f'{'R' if is_right else 'L'} {scores[source_index]:.2f}', p1,
        cv2.FONT_HERSHEY_SIMPLEX, 0.65, color, 2, cv2.LINE_AA)
    for point in uv:
        if np.isfinite(point).all():
            cv2.circle(overlay, tuple(np rint(point).astype(int)), 2, color, -1, cv2.LINE_AA)
20     write_obj(
        args.output_dir / "mesh" / f'{image_path.stem}_hand{source_index}_{'R' if is_right else 'L'}.
obj",
        vertices + translations[index][None], faces,
    )
25     offset += len(vertices_batch)
    np.savez_compressed(
        args.output_dir / "npz" / f'{image_path.stem}_wilor.npz",
        **{key: np.asarray(value) for key, value in arrays.items()},
        mano_faces=faces,
30     image_size_wh=np.array(image.shape[1::-1]), focal_length_px=np.array(focal_value),
        image_sha256=np.array(__import__('hashlib').sha256(image_path.read_bytes()).hexdigest()),
    )
    (args.output_dir / "json" / f'{image_path.stem}_wilor.json").write_text(
        json.dumps(frame, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
35     )
    cv2.imwrite(str(args.output_dir / "overlay" / f'{image_path.stem}_wilor.jpg"), overlay)
    report["frames"].append(frame)
    print(image_path.name, len(frame["detections"]))
    (args.output_dir / "wilor_report.json").write_text(
40     json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
    )
    if __name__ == "__main__":
        main()
    文件名: egosmplx_pipeline/body_common.py
45     功能: Joint mappings, bounded pose increments, and rigid rotation math. No annotation reader.
        """Joint mappings, bounded pose increments, and rigid rotation math. No annotation reader."""
    import numpy as np
    import torch
    JOINT_MAPPING = {
50     "nose": "Nose",

```

```

    "left_shoulder": "L_Shoulder",
    "right_shoulder": "R_Shoulder",
    "left_elbow": "L_Elbow",
    "right_elbow": "R_Elbow",
5    "left_wrist": "L_Wrist",
    "right_wrist": "R_Wrist",
    "left_hip": "L_Hip",
    "right_hip": "R_Hip",
    "left_knee": "L_Knee",
10   "right_knee": "R_Knee",
    "left_ankle": "L_Ankle",
    "right_ankle": "R_Ankle",
}
BODY_POSE_NAMES = (
15   "L_Hip", "R_Hip", "Spine_1", "L_Knee", "R_Knee", "Spine_2",
    "L_Ankle", "R_Ankle", "Spine_3", "L_Foot", "R_Foot", "Neck",
    "L_Collar", "R_Collar", "Head", "L_Shoulder", "R_Shoulder",
    "L_Elbow", "R_Elbow", "L_Wrist", "R_Wrist",
)
20   MAX_DELTA_DEG = {
        "L_Hip": 18.0, "R_Hip": 18.0,
        "Spine_1": 10.0, "Spine_2": 10.0, "Spine_3": 12.0,
        "L_Knee": 22.0, "R_Knee": 22.0,
        "L_Ankle": 14.0, "R_Ankle": 14.0,
25   "Neck": 12.0, "Head": 12.0,
        "L_Collar": 15.0, "R_Collar": 15.0,
        "L_Shoulder": 28.0, "R_Shoulder": 28.0,
        "L_Elbow": 25.0, "R_Elbow": 25.0,
        "L_Wrist": 15.0, "R_Wrist": 15.0,
30   }
def save_obj(vertices, faces, path):
    with path.open("w", encoding="utf-8") as output:
        for vertex in vertices:
            output.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
35   for face in faces:
            output.write(f"f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n")
def axis_angle_rotation(axis_angle):
    x, y, z = axis_angle.unbind()
    zero = torch.zeros_like(x)
40   skew = torch.stack((zero, -z, y, z, zero, -x, -y, x, zero)).reshape(3, 3)
    theta = torch.linalg.norm(axis_angle)
    first = torch.sinc(theta / np.pi)
    second = 0.5 * torch.sinc(theta / (2.0 * np.pi)) ** 2
    eye = torch.eye(3, dtype=axis_angle.dtype, device=axis_angle.device)
45   return eye + first * skew + second * (skew @ skew)
文件名: egosmplx_pipeline/body_rigid.py
功能: Fit only SMPL-X global orientation and translation to Sapiens2 2D points.
#!/usr/bin/env python3
"""Fit only SMPL-X global orientation and translation to Sapiens2 2D points.
50   Reuses the existing calibrated-fisheye rigid-fit helpers. Only automatic Sapiens2 targets are accepted.
```

The body rotates about the SMPL-X pelvis; saved parameters are regenerated and checked against the optimized geometry before any result is delivered.

"""

```

import argparse
5 import hashlib
import json
import os
from pathlib import Path
import cv2
10 import numpy as np
import torch
import torch.nn.functional as F
def main():
    p = argparse.ArgumentParser(description=__doc__)
15 for name in ['repo-root', 'source-npz', 'calibration', 'keypoints', 'image', 'output-dir']:
        p.add_argument('--' + name, type=Path, required=True)
    p.add_argument('--score-threshold', type=float, default=0.3)
    p.add_argument('--steps', type=int, default=1800)
    p.add_argument('--max-orient-delta-deg', type=float, default=60.)
20 p.add_argument('--device', choices=['cpu', 'cuda'], default='cpu', help='Rigid fitting is small; CPU avoids
old CUDA numerical differences')
    a = p.parse_args()
    if not 0 <= a.score_threshold <= 1 or a.steps < 1:
        raise ValueError('Invalid score threshold or step count')
25 os.environ['EGOSMPLX_REPO_ROOT'] = str(a.repo_root.resolve())
    from egosmplx_pipeline.bootstrap import setup
    setup(a.repo_root)
    from egosmplx_pipeline.body_common import axis_angle_rotation, save_obj, JOINT_MAPPING
    from FishEyeCalibrated import FishEyeCameraCalibrated
30 from utils.human_models import smpl_x
    from utils.vis import render_mesh_fisheye
    image = cv2.imread(str(a.image))
    if image is None:
        raise ValueError('Unreadable image')
35 h, w = image.shape[:2]
    payload = json.loads(a.keypoints.read_text())
    if payload['coordinate_space'] != 'original_image_pixels':
        raise ValueError('Sapiens2 targets must use original image pixels')
    if [payload['image_width'], payload['image_height']] != [w, h]:
40 raise ValueError('Target dimensions differ from image')
    if payload['image_sha256'] != hashlib.sha256(a.image.read_bytes()).hexdigest():
        raise ValueError('Target image hash mismatch')
    camera = FishEyeCameraCalibrated(str(a.calibration.resolve()))
    if tuple(map(int, camera.img_size)) != (w, h):
45 raise ValueError('Calibration dimensions differ from image')
    raw = {k: v.copy() for k, v in np.load(a.source_npz).items()}
    fixed_keys = ['body_pose', 'betas', 'left_hand_pose', 'right_hand_pose', 'jaw_pose', 'leye_pose', 'reye_pos
e', 'expression']
    for key in fixed_keys + ['global_orient', 'transl', 'smplx_mesh_cam', 'smplx_body_joints_3d_local']:
50 if key not in raw or not np.isfinite(raw[key]).all():

```

```

        raise ValueError('Missing or nonfinite source field: ' + key)
    dev = torch.device(a.device)
    # Rigid composition and parameter regeneration require full FP32 mantissas;
    # TF32 matrix products can introduce millimeter-scale discrepancies.
5   torch.backends.cuda.matmul.allow_tf32 = False
    torch.backends.cudnn.allow_tf32 = False
    t = lambda x: torch.as_tensor(x, dtype=torch.float32, device=dev)
    layer = smpl_x.layer['neutral'].to(dev).eval()
    fixed = {key: t(raw[key]).reshape(1, -1) for key in fixed_keys}
10   cam0 = t(raw['transl']).reshape(3)
    glob0 = t(raw['global_orient']).reshape(1, 3)
    with torch.no_grad():
        initial = layer(global_orient=glob0, **fixed)
        local = initial.joints[0, smpl_x.joint_idx[:25]].detach()
15   vertices = initial.vertices[0].detach()
        pivot = initial.joints[0, 0].detach()
        mesh_error = float((vertices + cam0 - t(raw['smplx_mesh_cam'])).abs().max())
        joints_error = float((local - t(raw['smplx_body_joints_3d_local'])).abs().max())
    print('Source reconstruction errors (m):', mesh_error, joints_error, flush=True)
20   # Legacy CUDA inference and full-precision CPU LBS differ by submillimeter
    # rounding. Audit this separately from the much tighter final round trip.
    if max(mesh_error, joints_error) > 1e-3:
        raise ValueError(f'Source parameter reconstruction mismatch: mesh={mesh_error}, joints={joints_err
or} m')
25   # Selection is determined solely by model scores and in-image coordinates.
    lookup = {j['name']: j for j in payload['keypoints']}
    if len(lookup) != len(payload['keypoints']):
        raise ValueError('Duplicate keypoint names')
    selected, excluded = [], []
30   for name, model_name in JOINT_MAPPING.items():
        j = lookup.get(name)
        valid = j is not None and np.isfinite([j['x'], j['y'], j['score']]).all()
        valid = valid and 0 <= j['x'] < w and 0 <= j['y'] < h and j['score'] >= a.score_threshold
        if valid:
35   selected.append(dict(name=name, model_name=model_name, index=smpl_x.joints_name.index(m
odel_name), x=j['x'], y=j['y'], score=j['score']))
        else:
            excluded.append(dict(name=name, prediction=j, reason='missing, nonfinite, outside image, or belo
w score threshold'))
40   if len(selected) < 4:
        raise ValueError(f'Only {len(selected)} usable automatic targets; need at least four')
    target = t([j['x'], j['y']] for j in selected)
    if np.linalg.svd(target.cpu().numpy() - target.cpu().numpy().mean(0), compute_uv=False)[1] < 10:
        raise ValueError('Automatic targets have insufficient 2D spatial spread')
45   indices = torch.tensor([j['index'] for j in selected], device=dev)
    weights = t([np.clip(j['score'], 0, 1)**2 for j in selected])
    cam = cam0.clone().requires_grad_(True)
    delta = torch.zeros(3, device=dev, requires_grad=True)
    scale = t([w, h])
50   def geometry():

```

```

rotation = axis_angle_rotation(delta)
joints = (local - pivot) @ rotation.t() + pivot + cam
mesh = (vertices - pivot) @ rotation.t() + pivot + cam
pixels = camera.world2camera_pytorch(joints)
5   return joints, mesh, pixels
with torch.no_grad():
    _, _, before = geometry()
    before = before.clone()
opt = torch.optim.Adam([{'params': [cam], 'lr': .005}, {'params': [delta], 'lr': .002}])
10  best_loss, best_state = float('inf'), None
    history = []
    for step in range(a.steps + 1):
        opt.zero_grad()
        joints, mesh, pixels = geometry()
15  residual = (pixels[indices] - target) / scale
        robust = F.smooth_l1_loss(residual, torch.zeros_like(residual), beta=.005, reduction='none').mean(1)
        reprojection = (robust * weights).sum() / weights.sum()
        prior_t = (((cam - cam0) / t([.25, .25, .50]))**2).mean()
        loss = reprojection + 1e-4 * prior_t + 2e-4 * delta.square().mean() + F.relu(.02 - mesh[:, 2].min()).squ
20  are()
        if not torch.isfinite(loss):
            raise RuntimeError('Nonfinite optimization objective')
        value = float(loss.detach())
        if value < best_loss:
25  best_loss, best_state = value, (cam.detach().clone(), delta.detach().clone())
        if step % 100 == 0 or step == a.steps:
            history.append(dict(step=step, loss=value, reprojection=float(reprojection.detach())))
        if step == a.steps:
            break
30  loss.backward()
        if not torch.isfinite(cam.grad).all() or not torch.isfinite(delta.grad).all():
            raise RuntimeError('Nonfinite rigid-fit gradient')
        opt.step()
        with torch.no_grad():
35  norm = delta.norm()
            limit = np.deg2rad(a.max_orient_delta_deg)
            if norm > limit:
                delta.mul_(limit / norm)
                cam[2].clamp_(.05, 3.)
40  with torch.no_grad():
            cam.copy_(best_state[0])
            delta.copy_(best_state[1])
            joints, mesh, after = geometry()
            delta_rotation = axis_angle_rotation(delta).cpu().numpy()
45  new_rotation = delta_rotation @ cv2.Rodrigues(raw['global_orient'].astype(np.float64).reshape(3))[0]
            new_global = cv2.Rodrigues(new_rotation.astype(np.float64))[0].astype(np.float32).reshape(1, 3)
            regenerated = layer(global_orient=t(new_global), **fixed)
            rebuilt_mesh = regenerated.vertices[0] + cam
            rebuilt_local = regenerated.joints[0, smpl_x.joint_idx[:25]]
50  rebuilt_pixels = camera.world2camera_pytorch(rebuilt_local + cam)

```

```

    regen_mesh_error = float((rebuilt_mesh - mesh).abs().max())
    regen_pixel_error = float((rebuilt_pixels - after).abs().max())
    if regen_mesh_error > 1e-4 or regen_pixel_error > .05:
        print('REGEN_DIAGNOSTIC', dict(
5            difference_mean=(rebuilt_mesh - mesh).mean(0).cpu().tolist(),
            difference_std=(rebuilt_mesh - mesh).std(0).cpu().tolist(),
            pivot=pivot.cpu().tolist(), regenerated_pivot=regenerated.joints[0, 0].cpu().tolist(),
            rotation_roundtrip_error=float(np.max(np.abs(cv2.Rodrigues(new_global.astype(np.float64).reshape(3))[0] - new_rotation))),
10            rotation_orthogonality_error=float(np.max(np.abs(new_rotation.T @ new_rotation - np.eye(3))))
        ,
            delta=delta.cpu().tolist(), global_new=new_global.tolist(), flush=True)
        raise RuntimeError(f'Regenerated geometry mismatch: {regen_mesh_error} m, {regen_pixel_error} px')
15    cpu = lambda x: x.detach().cpu().numpy()
    saved = {k: v.copy() for k, v in raw.items()}
    saved.update(global_orient=new_global,
        transl=cpu(cam).reshape(1, 3), smplx_mesh_cam=cpu(rebuilt_mesh),
        smplx_body_joints_3d_local=cpu(rebuilt_local),
20        smplx_body_joints_3d_render_cam=cpu(rebuilt_local + cam),
        smplx_body_joints_2d_render_cam=cpu(rebuilt_pixels),
        sapiens2_target_joint_pixels=cpu(target), sapiens2_selected_joint_indices=cpu(indices),
        sapiens2_target_scores=np.array([j['score'] for j in selected], dtype=np.float32),
        global_orient_delta_rotation=delta_rotation)
25    full_joints = regenerated.joints[0, smplx_joint_idx]
    saved['smplx_joint_cam_root_relative'] = cpu(full_joints - full_joints[:1])
    if not all(np.array_equal(saved[k], raw[k]) for k in fixed_keys):
        raise RuntimeError('A fixed parameter was changed')
    def errors(pred, truth):
30        d = np.linalg.norm(pred - truth, axis=1)
        return dict(rmse_px=float(np.sqrt(np.mean(d**2))), mean_px=float(d.mean()), median_px=float(np.median(d)))
    report = dict(source_npz=str(a.source_npz.resolve()), image=str(a.image.resolve()), calibration=str(a.calibration.resolve()),
35        sapiens2_keypoints=str(a.keypoints.resolve()), optimized_parameters=['global_orient', 'cam_trans'],
        fixed_parameters=fixed_keys, fixed_parameters_bitwise_equal=True, network_was_rerun=False,
        manual_labels_used_for_optimization=False, score_threshold=a.score_threshold,
40        score_weight='clip(score,0,1)^2; scores are not calibrated visibility probabilities',
        selected=selected, excluded=excluded, initial_cam_trans_m=cpu(cam0).tolist(),
        optimized_cam_trans_m=cpu(cam).tolist(), initial_global_orient=raw['global_orient'].tolist(),
        optimized_global_orient=new_global.tolist(), global_orient_delta_deg=float(np.rad2deg(np.linalg.norm(cpu(delta)))))
45        source_reconstruction_max_abs_m=dict(mesh=mesh_error, joints=joints_error),
        source_reconstruction_tolerance_m=1e-3, optimization_device=str(dev),
        saved_parameter_regeneration_max_abs_m=regen_mesh_error,
        saved_parameter_regeneration_max_abs_px=regen_pixel_error,
        source_npz_sha256=hashlib.sha256(a.source_npz.read_bytes()).hexdigest(),
50        raw_to_sapiens2=errors(cpu(before[indices]), cpu(target)),

```

```

        optimized_to_sapiens2=errors(cpu(rebuilt_pixels[indices]), cpu(target)), history=history,
        minimum_vertex_z_m=float(rebuilt_mesh[:, 2].min()),
        geometry_convention='SMPL-X vertices/joints include global orientation; add transl; rotate abo
ut SMPL-X pelvis',
5         preserved_network_fields='Fields containing network or model_output and positionnet are origi
nal inference diagnostics')
    a.output_dir.mkdir(parents=True, exist_ok=False)
    np.savez_compressed(a.output_dir / 'optimized_params.npz', **saved)
    save_obj(cpu(rebuilt_mesh), smpl_x.face, a.output_dir / 'optimized_mesh.obj')
10    overlays = []
    for name, verts in [(('raw', raw['smplx_mesh_cam']), ('optimized', cpu(rebuilt_mesh))):
        overlay = render_mesh_fisheye(image, verts, smpl_x.face, {}, camera, mode='wireframe', mesh_color
=(255, 255, 255), alpha=.78, proj_trans=None)
        cv2.imwrite(str(a.output_dir / f'{name}_wireframe.jpg'), overlay)
15    panel = cv2.resize(overlay, (640, round(h * 640 / w)))
    cv2.rectangle(panel, (0, 0), (640, 38), (25, 25, 25), -1)
    cv2.putText(panel, 'Raw EgoSMPLX' if name == 'raw' else 'Sapiens2 -> global orient + translation', (1
2, 25), cv2.FONT_HERSHEY_SIMPLEX, .62, (255, 255, 255), 1, cv2.LINE_AA)
    overlays.append(panel)
20    cv2.imwrite(str(a.output_dir / 'comparison.jpg'), np.concatenate(overlays, axis=1))
    diagnostic = image.copy()
    for j, initial_xy, final_xy in zip(selected, cpu(before[indices]), cpu(rebuilt_pixels[indices])):
        target_xy = (round(j['x']), round(j['y']))
        cv2.line(diagnostic, target_xy, tuple(np rint(final_xy).astype(int)), (0, 220, 220), 1, cv2.LINE_AA)
25    for xy, color in [(initial_xy, (255, 100, 0)), (final_xy, (0, 0, 255)), (target_xy, (0, 220, 0))]:
        cv2.circle(diagnostic, tuple(np rint(xy).astype(int)), 5, color, -1, cv2.LINE_AA)
        cv2.putText(diagnostic, j['name'], (target_xy[0] + 7, target_xy[1] - 7), cv2.FONT_HERSHEY_SIMPL
EX, .4, (0, 220, 0), 1, cv2.LINE_AA)
        cv2.putText(diagnostic, 'BLUE raw | GREEN Sapiens2 | RED optimized', (15, 30), cv2.FONT_HERSHEY
30    EY_SIMPLEX, .65, (255, 255, 255), 2, cv2.LINE_AA)
    cv2.imwrite(str(a.output_dir / 'keypoint_diagnostic.jpg'), diagnostic)
    (a.output_dir / 'optimization_report.json').write_text(json.dumps(report, indent=2) + '\n')
    print(json.dumps(report, indent=2))
    if __name__ == '__main__':
35        main()
    文件名: egosmplx_pipeline/body_pose.py
    功能: Optimize 19 body joints and small rigid changes using automatic points.
    #!/usr/bin/env python3
    """Optimize 19 body joints and small rigid changes using automatic points.
    Shape, hands, feet and facial parameters stay fixed. No local surface deformation.
    """
    import argparse
    import hashlib
    import json
45    import os
    from pathlib import Path
    import time
    import cv2
    import numpy as np
50    import torch

```

```

import torch.nn.functional as F
def main():
    p = argparse.ArgumentParser(description=__doc__)
    for key in ['repo-root', 'source-npz', 'image', 'calibration', 'automatic-keypoints', 'output-dir']:
5        p.add_argument('--' + key, type=Path, required=True)
    p.add_argument('--steps', type=int, default=1600)
    a = p.parse_args()
    os.environ['EGOSMPLX_REPO_ROOT'] = str(a.repo_root.resolve())
    from egosmplx_pipeline.bootstrap import setup
10    setup(a.repo_root)
    from egosmplx_pipeline.body_common import BODY_POSE_NAMES, MAX_DELTA_DEG, JOINT_
    MAPPING, save_obj
    from FishEyeCalibrated import FishEyeCameraCalibrated
    from utils.human_models import smpl_x
15    from utils.vis import render_mesh_fisheye
    a.output_dir.mkdir(parents=True, exist_ok=False)
    image = cv2.imread(str(a.image))
    if image is None:
        raise ValueError('Unreadable image')
20    h, w = image.shape[:2]
    camera = FishEyeCameraCalibrated(str(a.calibration))
    assert tuple(map(int, camera.img_size)) == (w, h)
    raw = {k: v.copy() for k, v in np.load(a.source_npz).items()}
    auto = json.loads(a.automatic_keypoints.read_text())
25    assert auto['coordinate_space'] == 'original_image_pixels'
    assert [auto['image_width'], auto['image_height']] == [w, h]
    assert auto['image_sha256'] == hashlib.sha256(a.image.read_bytes()).hexdigest()
    lookup = {j['name']: j for j in auto['keypoints']}
    # Filter automatic observations before optimization.
30    names = [n for n in JOINT_MAPPING if n in lookup and np.isfinite([lookup[n][k] for k in ['x', 'y', 'score'
]])].all()
        and lookup[n]['score'] >= .3 and 0 <= lookup[n]['x'] < w and 0 <= lookup[n]['y'] < h]
    if len(names) < 4:
        raise ValueError('Too few usable automatic targets')
35    t = lambda x: torch.as_tensor(x, dtype=torch.float32)
    indices = torch.tensor([smpl_x.joints_name.index(JOINT_MAPPING[n]) for n in names])
    auto_xy = np.array([[lookup[n]['x'], lookup[n]['y']] for n in names], dtype=np.float32)
    layer = smpl_x.layer['neutral'].cpu().eval()
    layer.requires_grad_(False)
40    fixed_keys = ['betas', 'left_hand_pose', 'right_hand_pose', 'jaw_pose', 'leye_pose', 'reye_pose', 'expression
']
    fixed = {k: t(raw[k]).reshape(1, -1) for k in fixed_keys}
    body0, glob0, cam0 = t(raw['body_pose']).reshape(21, 3), t(raw['global_orient']).reshape(3), t(raw['transl'
]).reshape(3)
45    active = list(MAX_DELTA_DEG)
    active_indices = torch.tensor([BODY_POSE_NAMES.index(n) for n in active])
    caps = t([np.deg2rad(MAX_DELTA_DEG[n]) for n in active])
    wrists = [smpl_x.joints_name.index('L_Wrist'), smpl_x.joints_name.index('R_Wrist')]
    def forward(body, glob, cam):
50        out = layer(body_pose=body.reshape(1, -1), global_orient=glob.reshape(1, 3), **fixed)

```

```

    local = out.joints[0, smpl_x.joint_idx]
    mesh = out.vertices[0] + cam
    joints = local + cam
    pixels = camera.world2camera_pytorch(joints)
5     return mesh, local, joints, pixels
with torch.no_grad():
    initial = tuple(v.clone() for v in forward(body0, glob0, cam0))
    initial_wrists = initial[2][wrists].norm(dim=1)
    wrist_floor = initial_wrists * .85
10    source_error = float((initial[0] - t(raw['smplx_mesh_cam'])).abs().max())
    assert source_error < 1e-5
def cpu(x):
    return x.detach().numpy()
def error(pred, target):
15    d = np.linalg.norm(pred - target, axis=1)
    return dict(rmse_px=float(np.sqrt(np.mean(d*d))), mean_px=float(d.mean()), median_px=float(np.me
dian(d)), per_joint_px={n: float(v) for n, v in zip(names, d)})
def angle_between(x, y):
    rx, ry = cv2.Rodrigues(np.asarray(x, dtype=np.float64))[0], cv2.Rodrigues(np.asarray(y, dtype=np.flo
20 at64))[0]
    return float(np.rad2deg(np.arccos(np.clip((np.trace(rx @ ry.T)-1)/2, -1, 1))))
settings = dict(steps=a.steps, device='cpu', target_weighting='uniform automatic targets',
    body_lr=.0015, camera_lr=.0008, orient_lr=.0005,
    pose_prior_weight=.0008, camera_prior_weight=.0002, orient_prior_weight=.0004,
25    smooth_l1_beta=.003, max_global_axis_angle_increment_deg=8.,
    body_axis_angle_increment_limits_deg=MAX_DELTA_DEG,
    wrist_distance_floor_fraction=.85, wrist_guard_weight=.002,
    wrist_distance_floor_m=cpu(wrist_floor).tolist(), minimum_vertex_z_m=.02,
    depth_barrier_onset_m=.03, depth_barrier_scale_m=.02, depth_barrier_weight=.02,
30    best_candidate_rule='lowest total objective among candidates satisfying depth guards; manual
evaluation never used')
def fit(label, target_xy):
    start = time.time()
    target = t(target_xy)
35    delta = torch.zeros((len(active), 3), requires_grad=True)
    gd = torch.zeros(3, requires_grad=True)
    cam = cam0.clone().requires_grad_(True)
    opt = torch.optim.Adam([{'params': [delta], 'lr': .0015}, {'params': [gd], 'lr': .0005}, {'params': [cam], '
lr': .0008}])
40    best_loss, best = float('inf'), None
    history = []
    for step in range(a.steps + 1):
        opt.zero_grad()
        body = body0.clone()
45    body[active_indices] = body[active_indices] + delta
        mesh, local, joints, pixels = forward(body, glob0 + gd, cam)
        rep = F.smooth_l1_loss((pixels[indices] - target) / t([w, h]), torch.zeros_like(target), beta=.003)
        pp = (delta / caps[:, None]).square().mean()
        cp = ((cam - cam0) / t([.10, .10, .18])).square().mean()
50    gp = (gd / np.deg2rad(8.)).square().mean()

```

```

wrist_dist = joints[wrists].norm(dim=1)
guard = F.relu((wrist_floor - wrist_dist) / .05).square().mean()
# Start the barrier before the hard near plane. The legacy penalty
# in raw meters was too weak relative to normalized pixel residuals.
5 depth = F.relu((.03 - mesh[:, 2].min()) / .02).square()
loss = rep + .0008*pp + .0002*cp + .0004*gp + .002*guard + .02*depth
if not torch.isfinite(loss):
    raise RuntimeError('Nonfinite objective')
feasible = bool((wrist_dist >= wrist_floor - 1e-6).all() and mesh[:, 2].min() >= .02 - 1e-6)
10 value = float(loss.detach())
if feasible and value < best_loss:
    best_loss = value
    best = (delta.detach().clone(), gd.detach().clone(), cam.detach().clone(), step)
if step % 100 == 0 or step == a.steps:
15 item = dict(step=step, loss=value, reprojection=float(rep.detach()), feasible=feasible,
               minimum_z_m=float(mesh[:, 2].min().detach()))
    history.append(item)
    print(label, item, flush=True)
if step == a.steps:
20 break
loss.backward()
if not all(torch.isfinite(x.grad).all() for x in [delta, gd, cam]):
    raise RuntimeError('Nonfinite optimization gradients')
opt.step()
25 with torch.no_grad():
    delta.mul_(torch.minimum(torch.ones_like(caps), caps / delta.norm(dim=1).clamp_min(1e-8))[:,
None])
    gd.mul_(torch.clamp(t(np.deg2rad(8.)) / gd.norm().clamp_min(1e-8), max=1.))
    cam[2].clamp_(.05, 3.)
30 if best is None:
    raise RuntimeError('No feasible refinement candidate')
with torch.no_grad():
    bd, gg, cc, best_step = best
    final_body = body0.clone()
35 final_body[active_indices] += bd
    final_global = glob0 + gg
    mesh, local, joints, pixels = forward(final_body, final_global, cc)
    dst = a.output_dir / label
    dst.mkdir()
40 saved = {k: v.copy() for k, v in raw.items()}
    saved.update(body_pose_rigid_input=raw['body_pose'], global_orient_rigid_input=raw['global_orient
'], transl_rigid_input=raw['transl'],
                body_pose=cpu(final_body).reshape(raw['body_pose'].shape), global_orient=cpu(final_global
).reshape(1, 3), transl=cpu(cc).reshape(1, 3),
45 smplx_mesh_cam=cpu(mesh), smplx_body_joints_3d_local=cpu(local[:25]), smplx_body_joi
nts_3d_render_cam=cpu(joints[:25]),
    smplx_body_joints_2d_render_cam=cpu(pixels[:25]), smplx_joint_cam_root_relative=cpu(lo
cal-local[:1]),
    stage2_target_joint_pixels=target_xy, stage2_selected_joint_indices=cpu(indices))
50 assert all(np.array_equal(saved[k], raw[k]) for k in fixed_keys)

```

```

np.savez_compressed(dst / 'refined_params.npz', **saved)
# Independently reload the actual artifact and regenerate its geometry.
reloaded = np.load(dst / 'refined_params.npz')
assert all(np.isfinite(reloaded[k]).all() for k in reloaded.files if np.issubdtype(reloaded[k].dtype, np.nu
5 mber))
    with torch.no_grad():
        rm, _, rp = forward(t(reloaded['body_pose']).reshape(21, 3), t(reloaded['global_orient']).reshape(
3), t(reloaded['transl']).reshape(3))
        reload_mesh_error = float(np.abs(cpu(rm)-reloaded['smplx_mesh_cam']).max())
10 reload_pixel_error = float(np.abs(cpu(rp[:25])-reloaded['smplx_body_joints_2d_render_cam']).max())
        assert reload_mesh_error < 1e-5 and reload_pixel_error < .01
        save_obj(cpu(mesh), smpl_x.face, dst / 'refined_mesh.obj')
        overlay = render_mesh_fisheye(image, cpu(mesh), smpl_x.face, {}, camera, mode='wireframe', mesh
_color=(255, 255, 255), alpha=.78, proj_trans=None)
15 cv2.imwrite(str(dst / 'refined_wireframe.jpg'), overlay)
        diag = image.copy()
        for n, target_pt, pred in zip(names, target_xy, cpu(pixels[indices])):
            tx, px = tuple(np rint(target_pt).astype(int)), tuple(np rint(pred).astype(int))
            cv2.line(diag, tx, px, (0, 220, 220), 1, cv2.LINE_AA)
20 cv2.circle(diag, tx, 6, (0, 220, 0), -1, cv2.LINE_AA)
            cv2.circle(diag, px, 4, (0, 0, 255), -1, cv2.LINE_AA)
            cv2.putText(diag, n, (tx[0]+6, tx[1]-6), cv2.FONT_HERSHEY_SIMPLEX, .4, (0, 180, 0), 1, cv2.LI
NE_AA)
        cv2.imwrite(str(dst / 'keypoint_diagnostic.jpg'), diag)
25 increments = np.rad2deg(cpu(bd.norm(dim=1)))
        actual_angles = {n: angle_between(cpu(final_body[i]), cpu(body0[i])) for n, i in zip(active, active_ind
ices)}
        report = dict(branch=label, target_source='Sapiens2',
            manual_labels_used_for_optimization=False,
30 target_fit_before=error(cpu(initial[3][indices]), target_xy), target_fit_after=error(cpu(pixels[i
ndices]), target_xy),
            best_step=best_step, best_objective=best_loss, elapsed_seconds=time.time()-start,
            pose_increment_deg={n: float(v) for n, v in zip(active, increments)}, pose_geodesic_change
_deg=actual_angles,
35 joints_at_increment_limit=[n for n, v in zip(active, increments) if v >= MAX_DELTA_DEG[
n]-.05],
            global_geodesic_change_deg=angle_between(cpu(final_global), cpu(glob0)),
            cam_trans_m=cpu(cc).tolist(), cam_trans_change_m=cpu(cc-cam0).tolist(),
            wrist_camera_distance_m=cpu(joints[wrists].norm(dim=1)).tolist(),
40 initial_wrist_camera_distance_m=cpu(initial_wrists).tolist(), minimum_vertex_z_m=float(m
esh[:, 2].min()),
            fixed_fields_bitwise_equal=fixed_keys, reload_mesh_error_m=reload_mesh_error, reload_pi
xel_error_px=reload_pixel_error,
            history=history)
45 (dst / 'refinement_report.json').write_text(json.dumps(report, indent=2)+'\n')
        return report, cpu(pixels[indices]), cpu(mesh), overlay, cpu(final_body)
        automatic = fit('automatic', auto_xy)
        if True:
            rows = {
50 'rigid_shared_input': {'against_sapiens2': error(cpu(initial[3][indices]), auto_xy)},

```

```

        'automatic_stage2': {'against_sapiens2': error(automatic[1], auto_xy)},
    }
    summary = dict(source_npz=str(a.source_npz), source_sha256=hashlib.sha256(a.source_npz.read_byt
5      es()).hexdigest(),
        automatic_keypoints=str(a.automatic_keypoints), image=str(a.image), calibration=str(a.calib
ration),
        comparison='Automatic Sapiens2 stage-two refinement from the supplied rigid initialization'
    ,
        names=names, settings=settings, source_mesh_reload_error_m=source_error, results=rows,
10      automatic_diagnostics=automatic[0])
    (a.output_dir / 'comparison_summary.json').write_text(json.dumps(summary, indent=2)+'\n')
    rigid_overlay = render_mesh_fisheye(image, cpu(initial[0]), smpl_x.face, {}, camera, mode='wirefra
me', mesh_color=(255, 255, 255), alpha=.78, proj_trans=None)
    panels = []
15      for title, overlay, key in [('Rigid input', rigid_overlay, 'rigid_shared_input'), ('Stage 2: Sapiens2 targets
', automatic[3], 'automatic_stage2')]:
        panel = cv2.resize(overlay, (640, round(h*640/w)))
        cv2.rectangle(panel, (0, 0), (640, 62), (25, 25, 25), -1)
        cv2.putText(panel, title, (12, 25), cv2.FONT_HERSHEY_SIMPLEX, .65, (255, 255, 255), 1, cv2.L
20      INE_AA)
        cv2.putText(panel, f'Sapiens2 RMSE: {rows[key]['against_sapiens2']['rmse_px']:.2f} px', (12, 49),
        cv2.FONT_HERSHEY_SIMPLEX, .55, (210, 210, 210), 1, cv2.LINE_AA)
        panels.append(panel)
        cv2.imwrite(str(a.output_dir / 'stage2_comparison.jpg'), np.concatenate(panels, axis=1))
25      print('AUTOMATIC_ONLY', json.dumps(rows, indent=2), flush=True)
    return
    if __name__ == '__main__':
        main()
    文件名: egosmplx_pipeline/camera.py
30      功能: 项目程序模块。
    import json
    import numpy as np
    import torch
    from copy import deepcopy
35      class FishEyeCameraCalibrated:
        def __init__(self, calibration_file_path, use_gpu=False):
            with open(calibration_file_path) as f:
                calibration_data = json.load(f)
            self.img_size = np.array(calibration_data['size']) # w, h
40      self.fisheye_polynomial = np.array(calibration_data['polynomialC2W'])
            self.fisheye_inverse_polynomial = np.array(calibration_data['polynomialW2C'])
            if 'intrinsic' in calibration_data:
                self.intrinsic = np.array(calibration_data['intrinsic'])
                self.img_center = np.array([
45      self.intrinsic[0][2], self.intrinsic[1][2]
                ])
            elif 'image_center' in calibration_data:
                self.img_center = np.asarray(
                    calibration_data['image_center'], dtype=np.float64
50      )

```

```

        self.intrinsic = np.eye(4, dtype=np.float64)
        self.intrinsic[0, 2] = self.img_center[0]
        self.intrinsic[1, 2] = self.img_center[1]
    else:
5         raise KeyError(
            "Calibration must contain either 'intrinsic' or 'image_center'"
        )
    affine = calibration_data.get('affine')
    if affine is None:
10         affine = [
            calibration_data.get('c', 1.0),
            calibration_data.get('d', 0.0),
            calibration_data.get('e', 0.0),
        ]
15     self.affine = np.asarray(affine, dtype=np.float64).reshape(3)
    self.c, self.d, self.e = self.affine
    self.affine_det = self.c - self.d * self.e
    if not np.isfinite(self.affine).all() or abs(self.affine_det) < 1e-12:
        raise ValueError('Calibration affine parameters are invalid')
20     self.camera_axis_transform = np.asarray(
        calibration_data.get('camera_axis_transform', np.eye(3)),
        dtype=np.float64,
    ).reshape(3, 3)
    if not np.isfinite(self.camera_axis_transform).all():
25         raise ValueError("camera_axis_transform must contain finite values")
    if not np.allclose(
        self.camera_axis_transform @ self.camera_axis_transform.T,
        np.eye(3),
        atol=1e-6,
30     ):
        raise ValueError("camera_axis_transform must be an orthonormal rotation")
    self.camera_axis_transform_inv = self.camera_axis_transform.T
    self.use_gpu = use_gpu
    # self.img_center = np.array([self.img_size[0] / 2, self.img_size[1] / 2])
35     def camera2world(self, point: np.ndarray, depth: np.ndarray):
        """
        point: np.ndarray of 2D points on image (n * 2)
        depth: np.ndarray of depth of every 2D points (n)
        """
40         depth = depth.astype(np.float32)
        point_centered = point.astype(np.float32) - self.img_center
        distorted_x = point_centered[:, 0]
        distorted_y = point_centered[:, 1]
        x = (distorted_x - self.d * distorted_y) / self.affine_det
45         y = (-self.e * distorted_x + self.c * distorted_y) / self.affine_det
        distance_from_center = np.sqrt(np.square(x) + np.square(y))
        z = np.polyval(p=self.fisheye_polynomial[::-1], x=distance_from_center)
        point_3d = np.array([x, y, -z]) # 3, n
        norm = np.linalg.norm(point_3d, axis=0)
50         point_3d = point_3d / norm * depth

```

```

    point_3d = point_3d.transpose()
    return point_3d @ self.camera_axis_transform_inv.T
def camera2world_ray(self, point: np.ndarray):
    """
5      calculate the ray direction from the image points
      point: np.ndarray of 2D points on image (n * 2)
      depth: np.ndarray of depth of every 2D points (n)
    """

    point_centered = point.astype(np.float32) - self.img_center
10     distorted_x = point_centered[:, 0]
    distorted_y = point_centered[:, 1]
    x = (distorted_x - self.d * distorted_y) / self.affine_det
    y = (-self.e * distorted_x + self.c * distorted_y) / self.affine_det
    distance_from_center = np.sqrt(np.square(x) + np.square(y))
15     z = np.polyval(p=self.fisheye_polynomial[::-1], x=distance_from_center)
    point_3d = np.array([x, y, -z]) # 3, n
    norm = np.linalg.norm(point_3d, axis=0)
    point_3d = point_3d / norm
    point_3d = point_3d.transpose()
20     return point_3d @ self.camera_axis_transform_inv.T
def getPolyVal(self, p, x):
    curVal = torch.zeros_like(x)
    for curValIndex in range(len(p) - 1):
        curVal = (curVal + p[curValIndex]) * x
25     return curVal + p[len(p) - 1]
def camera2world_pytorch(self, point: torch.Tensor, depth: torch.Tensor):
    """
      point: np.ndarray of 2D points on image (n * 2)
      depth: np.ndarray of depth of every 2D points (n)
30     """

    img_center = torch.from_numpy(self.img_center).float().to(point.device)
    point_centered = point.float() - img_center
    distorted_x = point_centered[:, 0]
    distorted_y = point_centered[:, 1]
35     c = torch.as_tensor(self.c, dtype=point.dtype, device=point.device)
    d = torch.as_tensor(self.d, dtype=point.dtype, device=point.device)
    e = torch.as_tensor(self.e, dtype=point.dtype, device=point.device)
    det = torch.as_tensor(self.affine_det, dtype=point.dtype, device=point.device)
    x = (distorted_x - d * distorted_y) / det
40     y = (-e * distorted_x + c * distorted_y) / det
    distance_from_center = torch.sqrt(torch.square(x) + torch.square(y))
    z = self.getPolyVal(p=self.fisheye_polynomial[::-1], x=distance_from_center)
    point_3d = torch.stack([x, y, -z]).float() # 3, n
    norm = torch.norm(point_3d, dim=0)
45     point_3d = point_3d / norm * depth.float()
    point_3d = point_3d.t()
    axis_transform_inv = torch.as_tensor(
        self.camera_axis_transform_inv,
        dtype=point_3d.dtype,
50     device=point_3d.device,

```

```

    )
    return point_3d @ axis_transform_inv.t()
def camera2world_ray_pytorch(self, point: torch.Tensor):
    """
5       point: np.ndarray of 2D points on image (n * 2)
       depth: np.ndarray of depth of every 2D points (n)
    """
    img_center = torch.as_tensor(
        self.img_center, dtype=point.dtype, device=point.device
10    )
    point_centered = point.float() - img_center
    distorted_x = point_centered[:, 0]
    distorted_y = point_centered[:, 1]
    c = torch.as_tensor(self.c, dtype=point.dtype, device=point.device)
15    d = torch.as_tensor(self.d, dtype=point.dtype, device=point.device)
    e = torch.as_tensor(self.e, dtype=point.dtype, device=point.device)
    det = torch.as_tensor(self.affine_det, dtype=point.dtype, device=point.device)
    x = (distorted_x - d * distorted_y) / det
    y = (-e * distorted_x + c * distorted_y) / det
20    distance_from_center = torch.sqrt(torch.square(x) + torch.square(y))
    z = self.getPolyVal(p=self.fisheye_polynomial[:, :-1], x=distance_from_center)
    point_3d = torch.stack([x, y, -z]).float() # 3, n
    norm = torch.norm(point_3d, dim=0)
    point_3d = point_3d / norm
25    point_3d = point_3d.t()
    axis_transform_inv = torch.as_tensor(
        self.camera_axis_transform_inv,
        dtype=point_3d.dtype,
        device=point_3d.device,
30    )
    return point_3d @ axis_transform_inv.t()
def world2camera(self, point3D):
    point3D = np.asarray(deepcopy(point3D))
    point3D = point3D @ self.camera_axis_transform.T
35    point3D[:, 2] = point3D[:, 2] * -1
    point3D = point3D.T
    xc, yc = self.img_center[0], self.img_center[1]
    point2D = []
    norm = np.linalg.norm(point3D[:, 2], axis=0)
40    if (norm != 0).all():
        theta = np.arctan(point3D[2] / norm)
        invnorm = 1.0 / norm
        t = theta
        rho = self.fisheye_inverse_polynomial[0]
45    t_i = 1.0
    for i in range(1, len(self.fisheye_inverse_polynomial)):
        t_i *= t
        rho += t_i * self.fisheye_inverse_polynomial[i]
    x = point3D[0] * invnorm * rho
50    y = point3D[1] * invnorm * rho

```

```

        point2D.append(self.c * x + self.d * y + xc)
        point2D.append(self.e * x + y + yc)
    else:
        point2D.append(xc)
5       point2D.append(yc)
        raise Exception("norm is zero!")
    return np.asarray(point2D).T
def world2camera_with_depth(self, point3D):
    point3D_cloned = deepcopy(point3D)
10    point2D = self.world2camera(point3D_cloned)
    depth = np.linalg.norm(point3D, axis=-1)
    return point2D, depth
def world2camera_pytorch_with_depth(self, point3D):
    point2D = self.world2camera_pytorch(point3D)
15    depth = torch.norm(point3D, dim=-1)
    return point2D, depth
def world2camera_pytorch(self, point3d_original: torch.Tensor, normalize=False):
    """
    Args:
20     point3d_original: point
        normalize: normalize to -1, 1
    Returns:
    """
    fisheye_inv_polynomial = self.fisheye_inverse_polynomial
25    axis_transform = torch.as_tensor(
        self.camera_axis_transform,
        dtype=point3d_original.dtype,
        device=point3d_original.device,
    )
30    point3d = point3d_original.clone() @ axis_transform.t()
    point3d[:, 2] = point3d[:, 2] * -1
    point3d = point3d.transpose(0, 1)
    xc, yc = self.img_center[0], self.img_center[1]
    xc = torch.Tensor([xc]).float().to(point3d.device)
35    yc = torch.Tensor([yc]).float().to(point3d.device)
    point2d = torch.empty((2, point3d.shape[-1])).to(point3d.device)
    norm = torch.norm(point3d[2:], dim=0)
    if (norm != 0).all():
        theta = torch.atan(point3d[2] / norm)
40        invnorm = 1.0 / norm
        t = theta
        rho = fisheye_inv_polynomial[0]
        t_i = 1.0
        for i in range(1, len(fisheye_inv_polynomial)):
45            t_i *= t
            rho += t_i * fisheye_inv_polynomial[i]
        x = point3d[0] * invnorm * rho
        y = point3d[1] * invnorm * rho
        c = torch.as_tensor(self.c, dtype=x.dtype, device=x.device)
50        d = torch.as_tensor(self.d, dtype=x.dtype, device=x.device)

```

```

        e = torch.as_tensor(self.e, dtype=x.dtype, device=x.device)
        point2d[0] = c * x + d * y + xc
        point2d[1] = e * x + y + yc
    else:
5         point2d[0] = xc
        point2d[1] = yc
        raise Exception("norm is zero!")
    # if normalize, the point result will be -1 to 1 from
    if normalize is True:
10         image_w, image_h = self.img_size[0], self.img_size[1]
        assert image_w > image_h
        point2d[0] = point2d[0] - (image_w - image_h) // 2 # to square image
        point2d = point2d / (image_h - 1) * 2 # to [0, 2]
        point2d -= 1
15         return point2d.transpose(0, 1)
def undistort(self, point_2Ds):
    """
    undistort the input 2d points in fisheye camera
    """
20     point_length = point_2Ds.shape[0]
    depths = np.ones(shape=point_length)
    point_3Ds = self.camera2world(point_2Ds, depths)
    point_3Ds = point_3Ds @ self.camera_axis_transform.T
    # point_3Ds_homo = np.ones((point_length, 4))
25     # point_3Ds_homo[:, :3] = point_3Ds
    projected_2d_points = (self.intrinsic[:3, :3] @ point_3Ds.T).T
    projected_2d_points = projected_2d_points[:, :2] / projected_2d_points[:, 2:]
    return projected_2d_points
文件名: egosmplx_pipeline/projection.py
30 功能: FP64 Horner evaluation of the calibrated fisheye polynomial with autograd.
    """FP64 Horner evaluation of the calibrated fisheye polynomial with autograd."""
    import torch
    def stable_world2camera(self, points, normalize=False):
        dtype = points.dtype
35         axes = torch.as_tensor(self.camera_axis_transform, dtype=torch.float64, device=points.device)
        xyz = points.to(torch.float64) @ axes.t()
        radius = torch.linalg.vector_norm(xyz[:, :2], dim=1)
        if not bool((radius != 0).all()):
            raise ValueError('norm is zero!')
40         theta = torch.atan(-xyz[:, 2] / radius)
        coefficients = self.fisheye_inverse_polynomial
        rho = torch.full_like(theta, float(coefficients[-1]))
        for coefficient in reversed(coefficients[:-1]):
            rho = rho * theta + float(coefficient)
45         x, y = xyz[:, 0] * rho / radius, xyz[:, 1] * rho / radius
        u = float(self.c) * x + float(self.d) * y + float(self.img_center[0])
        v = float(self.e) * x + y + float(self.img_center[1])
        pixels = torch.stack((u, v), dim=1)
        if normalize:
50         width, height = self.img_size

```

```

    assert width > height
    pixels = (pixels - pixels.new_tensor([(width-height)/2, 0])) / (height-1) * 2 - 1
    return pixels.to(dtype)

```

文件名: egosmplx_pipeline/matching.py

5 功能: One-to-one automatic hand assignment, frozen before fusion.

"""One-to-one automatic hand assignment, frozen before fusion."""

import itertools

import numpy as np

SIDES = ["left", "right"]

10 def assign(w, lookup, initial_uv, width=1280, height=720):

"""Global one-to-one assignment, using only automatic wrist evidence."""

costs={}; details={}

for si,s in enumerate(SIDES):

a=lookup.get(s+'_wrist', dict(score=0,x=0,y=0))

15 use= a['score']>=.3 and np.isfinite([a['x'],a['y']]).all() and 0<=a['x']<width and 0<=a['y']<height

anchor=np.array([a['x'],a['y']]) if use else initial_uv[12+si]

for i,c in enumerate(w['detector_confidence']):

uv=w['joints_2d'][i];dist=float(np.linalg.norm(uv[0]-anchor))

agree=bool(w['is_right'][i]>.5)==(s=='right')

20 valid=np.isfinite(uv).all(axis=1)&(uv[:,0]>=0)&(uv[:,0]<width)&(uv[:,1]>=0)&(uv[:,1]<height)

if c>=.25 and dist<=150 and valid.sum()>=12:

costs[si,i]=dist+(0 if agree else 45)+20*(1-float(c))

details[si,i]=dict(candidate=i,confidence=float(c),wrist_distance_px=dist,detector_handedness_a

grees=agree,anchor_source='Sapiens2' if use else 'automatic_SMPLX',valid_keypoints=int(valid.sum()))

25 options=[[-1]+[i for a,i in costs if a==s] for s in range(2)]

combos=[c for c in itertools.product(*options) if c[0]<0 or c[1]<0 or c[0]!=c[1]]

best=min(combos,key=lambda c:sum(180 if i<0 else costs[s,i] for s,i in enumerate(c)))

return {s:details[k,i] for k,(s,i) in enumerate(zip(SIDES,best)) if i>=0}

文件名: egosmplx_pipeline/lift.py

30 功能: Preserve MANO projection through calibrated rays and radial distance.

"""Preserve MANO projection through calibrated rays and radial distance."""

import numpy as np

def native_projection(points_camera, focal, center):

normalized = points_camera[:, :2] / points_camera[:, 2:3].clip(1e-8)

35 return focal * normalized + np.asarray(center, dtype=np.float64)

def lift_mano_to_fisheye(camera, wilor, candidate, body_wrist, body_wrist_uv):

vertices_camera_native = (

np.asarray(wilor["vertices_3d_local"][candidate], dtype=np.float64)

+ np.asarray(wilor["cam_t"][candidate], dtype=np.float64)[None]

40)

joints_camera_native = (

np.asarray(wilor["joints_3d_local"][candidate], dtype=np.float64)

+ np.asarray(wilor["cam_t"][candidate], dtype=np.float64)[None]

)

45 focal = float(wilor['focal_length_px'])

center = np.asarray(wilor['image_size_wh'], dtype=np.float64) / 2

vertex_uv = native_projection(vertices_camera_native, focal, center)

joint_uv = native_projection(joints_camera_native, focal, center)

Keep the body wrist fixed and translate the complete hand projection.

50 # This residual can be large when the body estimate is inaccurate.

```

uv_shift = np.asarray(body_wrist_uv, dtype=np.float64) - joint_uv[0]
vertex_uv += uv_shift[None]
joint_uv += uv_shift[None]
native_root_depth = float(np.linalg.norm(joints_camera_native[0]))
5 body_root_depth = float(np.linalg.norm(body_wrist))
vertex_relative_depth = np.linalg.norm(vertices_camera_native, axis=1) - native_root_depth
joint_relative_depth = np.linalg.norm(joints_camera_native, axis=1) - native_root_depth
vertex_depth = np.clip(body_root_depth + vertex_relative_depth, 0.05, None)
joint_depth = np.clip(body_root_depth + joint_relative_depth, 0.05, None)
10 vertices_fisheye = camera.camera2world(vertex_uv, vertex_depth)
joints_fisheye = camera.camera2world(joint_uv, joint_depth)
return {
    "vertices": vertices_fisheye.astype(np.float32),
    "joints": joints_fisheye.astype(np.float32),
15 "vertex_uv": vertex_uv.astype(np.float32),
    "joint_uv": joint_uv.astype(np.float32),
    "uv_shift": uv_shift.astype(np.float32),
    "body_wrist_depth": body_root_depth,
    "native_wilor_root_depth": native_root_depth,
20 }

```

文件名: egosmplx_pipeline/topology.py

功能: Mesh boundary and breadth-first ring distances.

"""Mesh boundary and breadth-first ring distances."""

```

from collections import Counter, deque
25 import numpy as np
def boundary_edges(faces):
    counts = Counter(
        tuple(sorted((int(first), int(second))))
        for face in np.asarray(faces, dtype=np.int64)
30     for first, second in ((face[0], face[1]), (face[1], face[2]), (face[2], face[0]))
    )
    return {edge for edge, count in counts.items() if count == 1}
def graph_distance_from_boundary(faces, boundary_vertices, vertex_count):
    adjacency = [set() for _ in range(vertex_count)]
35     for face in np.asarray(faces, dtype=np.int64):
        for first, second in ((face[0], face[1]), (face[1], face[2]), (face[2], face[0])):
            adjacency[int(first)].add(int(second))
            adjacency[int(second)].add(int(first))
    sentinel = np.iinfo(np.int32).max
40     distance = np.full(vertex_count, sentinel, dtype=np.int32)
    queue = deque()
    for vertex in boundary_vertices:
        distance[int(vertex)] = 0
        queue.append(int(vertex))
45     while queue:
        first = queue.popleft()
        for second in adjacency[first]:
            if distance[second] > distance[first] + 1:
                distance[second] = distance[first] + 1
50         queue.append(second)

```

```

    if np.any(distance == sentinel):
        raise RuntimeError("MANO topology contains vertices disconnected from wrist boundary")
    return distance

```

文件名: egosmplx_pipeline/fusion.py

```

5  功能: Seven-ring baseline used before the archived wrist/contour/surface refinement.
    """Seven-ring baseline used before the archived wrist/contour/surface refinement."""
    from collections import Counter
    import numpy as np
    import cv2
10  import torch
    from egosmplx_pipeline.io import arr, rmse
    from egosmplx_pipeline.topology import boundary_edges, graph_distance_from_boundary
    from egosmplx_pipeline.lift import lift_mano_to_fisheye
    from egosmplx_pipeline.render import render
15  SIDES = ['left', 'right']
    PARAMS = ['betas', 'jaw_pose', 'leye_pose', 'reye_pose', 'expression', 'body_pose', 'global_orient', 'left_han
d_pose', 'right_hand_pose']
    def initialize(model):
        global smpl_x, LAYER, FACES, JI
20  smpl_x = model
        LAYER = model.layer['neutral'].cpu().eval()
        LAYER.requires_grad_(False)
        FACES = np.asarray(model.face, dtype=np.int32)
        JI = {n: i for i, n in enumerate(model.joints_name)}
25  def geometry(p):
        with torch.no_grad():
            r=LAYER(**{k:torch.as_tensor(p[k],dtype=torch.float32).reshape(1,-1) for k in PARAMS})
            return (arr(r.vertices[0])+p['transl'].reshape(3),
                    arr(r.joints[0,smpl_x.joint_idx]+p['transl'].reshape(3)))
30  def edge_counts(f):
        return Counter(tuple(sorted((int(a),int(b)))) for tri in f for a,b in zip(tri,np.roll(tri,-1)))
    def oriented_edges(f):
        return Counter((int(a),int(b)) for tri in f for a,b in zip(tri,np.roll(tri,-1)))
    def stitch(p,w,matches,cam):
35  original,j=geometry(p); uv=cam.world2camera(j.astype(np.float64))
        vertices=original.copy(); keep=np.ones(len(FACES),bool); additions=[]; extra={}; reports={}
        mf=np.asarray(w['mano_faces'],dtype=np.int32)
        boundary=boundary_edges(mf); bv=np.array(sorted({i for e in boundary for i in e}))
        assert len(bv)==len(boundary)==16
40  rings=graph_distance_from_boundary(mf,bv,778)
        t=np.clip(rings.astype(np.float32)/7,0,1); blend=t*t*(3-2*t)
        for side,a in matches.items():
            ids=np.asarray(smpl_x.hand_vertex_idx[side+' _hand'],dtype=np.int64)
            internal=np.isin(FACES,ids).all(axis=1)
45  inverse=np.full(len(vertices),-1,dtype=int); inverse[ids]=np.arange(778)
            assert boundary_edges(inverse[FACES[internal]])==boundary
            keep &= ~internal
            k=12+SIDES.index(side)
            lifted=lift_mano_to_fisheye(cam,w,a['candidate'],j[k],uv[k])
50  target=lifted['vertices'].copy()

```

```

# Preserve pixel rays while enforcing the original positive near-plane guard.
assert np.all(target[:,2]>0), 'Target ray behind the camera: reject rather than silently draw it'
adjusted=target[:,2]<.020001
target*=np.maximum(1,.020001/target[:,2])[None]
5 vertices[ids]=(1-blend[:,None])*original[ids]+blend[:,None]*target
assert np.array_equal(vertices[ids[bv]],original[ids[bv]])
faces=ids[mf]
# Native MANO winding may disagree on the reflected hand. Match the body seam.
original_edge=oriented_edges(FACES[internal]); new_edge=oriented_edges(faces)
10 a0,b0=ids[list(next(iter(boundary)))]
reversed_winding=original_edge[(int(a0),int(b0))]!=new_edge[(int(a0),int(b0))]
if reversed_winding: faces=faces[:,[0,2,1]]
additions.append(faces)
extra.update({side+' _local_to_mesh':ids,side+' _lifted_vertices':target,
15 side+' _blend_weights':blend,side+' _target_vertex_uv':lifted['vertex_uv'],
side+' _wrist_loop_local':bv,side+' _candidate':np.array(a['candidate'])})
projected=cam.world2camera(vertices[ids].astype(np.float64))
interior=blend==1
reports[side]=dict(candidate=a['candidate'],wrist_shift_px=lifted['uv_shift'].tolist(),
20 wrist_shift_norm_px=float(np.linalg.norm(lifted['uv_shift'])),
blend_rings=7,wrist_boundary_vertex_count=16,boundary_max_change_m=0.,
reversed_winding=bool(reversed_winding),near_plane_adjusted_vertices=int(adjusted.su
m()),
unblended_vertex_projection_rmse_px=rmse(projected[interior],lifted['vertex_uv'])[interior
25 ])
faces=np.concatenate([FACES[keep],*additions]).astype(np.int32)
counts=edge_counts(faces); directions=oriented_edges(faces)
for side in matches:
ids=extra[side+' _local_to_mesh']
30 for a,b in boundary:
u,v=int(ids[a]),int(ids[b]); assert counts[tuple(sorted((u,v)))]==2
assert directions[u,v]==directions[v,u]==1
# Check all MANO interior edges too, including corrected winding.
for a,b in edge_counts(mf):
35 u,v=int(ids[a]),int(ids[b]); assert counts[tuple(sorted((u,v)))]==2
assert directions[u,v]==directions[v,u]==1
reports[side]['seam_and_hand_edges_two_opposite_faces']=True
hand_joints=np.stack([smp1_x.orig_hand_regressor[s]@vertices for s in SIDES]).astype(np.float32)
assert np.isfinite(vertices).all() and vertices[:,2].min()>=.02-1e-6
40 untouched=np.ones(len(vertices),bool)
for s in matches: untouched[extra[s+' _local_to_mesh']]=False
assert np.array_equal(vertices[untouched],original[untouched])
return dict(vertices_cam=vertices,faces=faces,hand_joints_cam=hand_joints,body_joints_cam=j,
method=np.array('automatic_projection_preserving_MANO_wrist_seam'),**extra),reports
45 def draw_geometry(image,cam,mesh):
image=render(image,mesh['vertices_cam'].astype(np.float64),mesh['faces'],cam,.12)
body=cam.world2camera(mesh['body_joints_cam'].astype(np.float64))
def segment(points,a,b,color):
50 q=points[[a,b]]
if np.isfinite(q).all() and np.abs(q).max()<100000:

```

```

        cv2.line(image,tuple(np rint(q[0]).astype(int)),tuple(np rint(q[1]).astype(int)),color,2,cv2.LINE_AA
    )
    for a,b in [('Neck','L_Shoulder'),('Neck','R_Shoulder'),('L_Shoulder','L_Elbow'),('L_Elbow','L_Wrist'),('
R_Shoulder','R_Elbow'),('R_Elbow','R_Wrist'),('L_Shoulder','L_Hip'),('R_Shoulder','R_Hip'),('L_Hip','R_H
5 ip')]:
        segment(body,JI[a],JI[b],(0,185,235))
    for k,s in enumerate(SIDES):
        points=cam.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
        for start in [1,5,9,13,17]:
10         chain=[0]+list(range(start,start+4))
            color=((225,85,30) if s=='left' else (30,45,225)) if s+'_local_to_mesh' in mesh else (150,150,150)
            for a,b in zip(chain[:-1],chain[1:]):segment(points,a,b,color)
    return image
文件名: egosmplx_pipeline/render.py
15 功能: Clipped fisheye mesh overlays and Wavefront OBJ export.
    """Clipped fisheye mesh overlays and Wavefront OBJ export."""
    import numpy as np
    import cv2
    def read_obj(path):
20         vertices, faces = [], []
        with path.open("r", encoding="utf-8") as handle:
            for line in handle:
                if line.startswith("v "):
                    vertices.append([float(v) for v in line.split()[1:4]])
25                 elif line.startswith("f "):
                    faces.append([int(v.split("/")[0]) - 1 for v in line.split()[1:4]])
        return np.asarray(vertices, np.float64), np.asarray(faces, np.int32)
    def render(image, vertices, faces, camera, max_edge_fraction):
        uv = camera.world2camera(vertices)
30         axes = np.asarray(getattr(camera, "camera_axis_transform", np.eye(3)), np.float64)
        camera_vertices = vertices @ axes.T
        valid = (
            np.isfinite(camera_vertices).all(axis=1)
            & np.isfinite(uv).all(axis=1)
35             & (camera_vertices[:, 2] > 1e-5)
        )
        height, width = image.shape[:2]
        max_edge = max(40.0, max_edge_fraction * np.hypot(width, height))
        overlay = image.copy()
40         for tri in faces:
            if not valid[tri].all():
                continue
            points = uv[tri]
            if (
45                 points[:, 0].max() < 0 or points[:, 0].min() >= width
                or points[:, 1].max() < 0 or points[:, 1].min() >= height
            ):
                continue
            edges = points[[1, 2, 0]] - points
50             if np.linalg.norm(edges, axis=1).max() > max_edge:

```

```

        continue
    cv2.polylines(
        overlay, [np rint(points).astype(np.int32)], True,
        (255, 255, 255), 1, cv2.LINE_AA,
5    )
    return cv2.addWeighted(image, 0.22, overlay, 0.78, 0)
def write_obj(path, vertices, faces):
    with path.open("w", encoding="utf-8") as handle:
        for vertex in vertices:
10            handle.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
        for face in faces:
            handle.write(f"f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n")
文件名: egosmplx_pipeline/report.py
功能: Static, shareable three-stage comparison without remote asset dependencies.
15 """Static, shareable three-stage comparison without remote asset dependencies."""
import html
from pathlib import Path
import cv2
import numpy as np
20 from egosmplx_pipeline.io import write
from egosmplx_pipeline.profiles import CURRENT, LEGACY
def build(output, records):
    output = Path(output)
    tiles, sections = [], []
25    profiles = {r.get('pipeline_profile', LEGACY) for r in records}
    if len(profiles) != 1:
        raise ValueError('Cannot mix different pipeline profiles in one report')
    profile = next(iter(profiles))
    stages = [('raw', 'Raw EgoSMPLX'), ('body', 'Sapiens2 two-stage body fit')]
30    if profile == CURRENT:
        stages.append(('guided_body', 'MANO-guided body'))
        stages.append(('fused', 'Guided body + original seam' if profile == CURRENT else 'MANO + contour +
harmonic surface'))
    for row in records:
35        identifier = row['id']
        panels = []
        for stage, title in stages:
            if stage == 'guided_body' and not row.get('mano_guided_body'):
                stage, title = 'body', 'Body (no matched hand; guidance skipped)'
40            image = cv2.imread(str(output / 'frames' / identifier / stage / 'wireframe.jpg'))
            if image is None:
                raise ValueError('Missing stage overlay: ' + identifier + '/' + stage)
            panel = cv2.resize(image, (640, 360))
            cv2.rectangle(panel, (0, 0), (640, 32), (20, 20, 20), -1)
45            cv2.putText(panel, title, (12, 23), cv2.FONT_HERSHEY_SIMPLEX, .55, (255, 255, 255), 1, cv2.L
INE_AA)
            panels.append(panel)
        tile = np.concatenate(panels, axis=1)
        image_name = identifier + '_comparison.jpg'
50        cv2.imwrite(str(output / image_name), tile)

```

```

tiles.append(cv2.resize(tile, (480*len(stages), 270)))
sections.append('<section><h2>' + html.escape(identifier) + '</h2><p>Matched hands: ' + str(row['ma
tched_hands']) +
    '; local surface check: ' + ('passed' if row.get('local_surface_accepted') is True else 'crossings
5 remain' if row.get('local_surface_accepted') is False else 'not audited/skipped') + '</p><img loading="lazy"
src="" + image_name + ""></section>')
cv2.imwrite(str(output / 'contact_sheet.jpg'), np.concatenate(tiles, axis=0))
body = [x for r in records for x in r['body_error_px']]
hands = [x for r in records for x in r['hand_error_px']]
10 summary = dict(frame_count=len(records), matched_hands=sum(r['matched_hands'] for r in records),
    body_target_count=len(body), hand_target_count=len(hands),
    body_rmse_px=float(np.sqrt(np.mean(np.square(body)))),
    hand_rmse_px=float(np.sqrt(np.mean(np.square(hands)))) if hands else None,
    all_reload_checks_passed=all(r['passed'] for r in records),
15 manual_annotations_used=False, local_surface_optimizer=any(r.get('local_surface_optimizer')
for r in records),
    pipeline_profile=profile,
    local_surface_pass_count=sum(r.get('local_surface_accepted') is True for r in records),
    local_surface_audited_count=sum(r.get('local_surface_accepted') is not None for r in records),
20 reference='Sapiens2 body points / WiLoR hand points; not human ground truth')
write(output / 'summary.json', summary)
text = '<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport" content="
width=device-width">' + \
    '<title>EgoSmplx 分阶段对比</title><style>body{background:#161b22;color:#eee;font:16px sans-
25 serif;margin:24px}img{width:100%}section{margin:32px 0}</style>' + \
    '<h1>EgoSmplx V1.0 分阶段输出</h1><p>自动目标拟合误差，不是人工真值精度。重载通过
和局部表面检查分别报告，穿插未通过的结果仍保留供诊断。</p>' + '.join(sections) + '</html>'
(output / 'index.html').write_text(text, encoding='utf-8')
文件名: egosmplx_pipeline/package.py
30 功能: Package a verified run, its three-stage results and reconstruction inputs.
"""Package a verified run, its three-stage results and reconstruction inputs.
This exports a new bundle; it does not overwrite or impersonate the historical
session_hand6 ZIP. Network weights and segmentation probability caches stay out.
"""
35 import argparse
import hashlib
import json
from pathlib import Path
import zipfile
40 from egosmplx_pipeline.io import read, sha
from egosmplx_pipeline.configuration import PACKAGE_ROOT
from egosmplx_pipeline.profiles import SUPPORTED
def package(run, destination):
    run, destination = Path(run).resolve(), Path(destination).resolve()
45 if read(run / 'run_state.json')['state'] != 'complete':
    raise ValueError('Only complete, verified runs can be packaged')
if destination.exists():
    raise FileExistsError(destination)
sources = {}
50 for path in sorted((run / 'frames').glob('*/*validation.json')):

```

```

    report = read(path)
    if not report.get('passed') or report.get('pipeline_profile') not in SUPPORTED or report.get('pipeline_p
rofile') != read(run / 'summary.json')['pipeline_profile']:
        raise ValueError('Frame is not a verified reference-profile result')
5    frame = path.parent
    for name, digest in report['artifact_sha256'].items():
        candidate = (frame / name).resolve()
        if frame not in candidate.parents or sha(candidate) != digest:
            raise ValueError('Frame artifact changed: ' + name)
10    identifier = report['id']
    if frame.name != identifier:
        raise ValueError('Frame ID mismatch')
    row = read(run / 'jobs' / (identifier + '.json'))['row']
    if sha(row['image']) != report['input_image_sha256'] or sha(row['calibration']) != report['calibration_s
15    ha256']:
        raise ValueError('Original image or calibration changed')
    prefix = 'frames/' + identifier + '/'
    sources[prefix + 'input' + Path(row['image']).suffix.lower()] = Path(row['image'])
    sources[prefix + 'calibration.json'] = Path(row['calibration'])
20    sources[prefix + 'automatic_observations/sapiens2_keypoints.json'] = Path(row['predictions']['sapiens'
])
    for stage in ['raw', 'body', 'guided_body', 'fused']:
        if stage == 'guided_body' and not (frame / stage).exists():
            continue
25    for asset in sorted((frame / stage).iterdir()):
        if asset.is_file():
            sources[prefix + stage + '/' + asset.name] = asset
    sources[prefix + 'validation.json'] = path
    if len(list((run / 'frames').glob('*validation.json'))) != read(run / 'summary.json')['frame_count']:
30    raise ValueError('Run frame count differs from summary')
    for name in ['index.html', 'contact_sheet.jpg']:
        sources[prefix + name] = run / name
    for path in run.glob('*_comparison.jpg'):
        sources[prefix + path.name] = path
35    sources['summary.json'] = run / 'summary.json'
    sources['REFERENCE_PIPELINE_zh.md'] = PACKAGE_ROOT / 'docs/REFERENCE_PIPELINE_zh.
md'
    sources['CURRENT_PIPELINE_zh.md'] = PACKAGE_ROOT / 'docs/CURRENT_PIPELINE_zh.md'
    for path in sorted((PACKAGE_ROOT / 'egosmplx_pipeline').rglob('*py')):
40    sources['scripts/' + str(path.relative_to(PACKAGE_ROOT))] = path
    destination.parent.mkdir(parents=True, exist_ok=True)
    manifest = {}
    # Exclusive creation prevents an existing historical bundle from being replaced.
    with zipfile.ZipFile(destination, 'x', zipfile.ZIP_DEFLATED, compresslevel=6) as archive:
45    for name, path in sorted(sources.items()):
        data = path.read_bytes()
        archive.writestr(name, data)
        manifest[name] = dict(bytes=len(data), sha256=hashlib.sha256(data).hexdigest())
        archive.writestr('ARCHIVE_MANIFEST.json', json.dumps(manifest, indent=2))
50    with zipfile.ZipFile(destination) as archive:

```

```

        if archive.testzip() is not None:
            raise RuntimeError('Exported ZIP failed CRC validation')
        return dict(path=str(destination), sha256=sha(destination), manifest_members=len(manifest),
                    contains_weights=False, contains_full_segmentation_probabilities=False)
5  def main():
        parser = argparse.ArgumentParser(description=__doc__)
        parser.add_argument('--run', type=Path, required=True)
        parser.add_argument('--archive', type=Path, required=True)
        args = parser.parse_args()
10  print(json.dumps(package(args.run, args.archive), indent=2))
    if __name__ == '__main__':
        main()
    文件名: egosmplx_pipeline/profiles.py
    功能: Stable names for the current algorithm and the reproducible archived route.
15  """Stable names for the current algorithm and the reproducible archived route."""
    CURRENT = 'mano_guided_original_seam_20260929_v1'
    LEGACY = 'session_hand6_full_pipeline_20260926_v1'
    SUPPORTED = (CURRENT, LEGACY)
    def selected(config):
20  profile = config.get('pipeline_profile', CURRENT)
        if profile not in SUPPORTED:
            raise ValueError('Unsupported pipeline_profile: ' + str(profile))
        return profile
    文件名: egosmplx_pipeline/reference/__init__.py
25  功能: Exact algorithm profile of the session_hand6 result bundle dated 2026-09-26.
    """Exact algorithm profile of the session_hand6 result bundle dated 2026-09-26."""
    文件名: egosmplx_pipeline/reference/contour_refine.py
    功能: Fixed-shape SMPL-X wrist and projected seam compatibility refinement.
    """Fixed-shape SMPL-X wrist and projected seam compatibility refinement."""
30  from pathlib import Path
    import os
    import sys, copy, argparse, time
    import numpy as np
    import torch
35  import torch.nn.functional as F
    OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour'
    BASE=OUT.parent/'baseline'
    from egosmplx_pipeline.reference.support import (read, write, load, arr, geometry, constraints, PARAMS, FIX
    ED, SIDES, JI, MAX_DELTA_DEG, BODY_POSE_NAMES, JOINT_MAPPING, smpl_x, FishEyeCameraCa
40  librated)
    from egosmplx_pipeline.reference.support import native_projection
    from egosmplx_pipeline.reference.silhouette import targets, prepare, loss as silhouette_loss, metrics as silhou
    ette_metrics, PREV
    def main():
45  ap=argparse.ArgumentParser();ap.add_argument('--steps', type=int, default=500);ap.add_argument('--indice
    s', type=int, nargs='*');ap.add_argument('--active-projection', action='store_true');a=ap.parse_args()
        torch.backends.cuda.matmul.allow_tf32=False;torch.backends.cudnn.allow_tf32=False
        torch.set_num_threads(2);dev='cuda' if torch.cuda.is_available() else 'cpu';layer=copy.deepcopy(smpl_x.la
        yer['neutral']).to(dev).eval();layer.requires_grad_(False)
50  t=lambda x:torch.as_tensor(x, dtype=torch.float32, device=dev)

```

```

rows=read(BASE/'automatic_input_manifest.json')['frames']
for ii,row in enumerate(rows):
    if a.indices and ii not in a.indices:continue
    dst=OUT/('active_frames' if a.active_projection else 'frames')/row['condition']/row['camera'];dst.mkdir(pa
5 rents=True,exist_ok=True)
    if (dst/'refinement.json').exists():continue
    bd=BASE/'frames'/row['condition']/row['camera'];p=load(PREV/'final_frames'/row['condition']/row['cam
era']/'body_params.npz');source=load(row['source_npz']);rigid=load(row['rigid_npz']);w=load(row['wilor_n
pz']);oldmesh=load(bd/'hybrid_mesh.npz');matches=read(bd/'recipe.json')['matches'];cam=FishEyeCameraC
10 alibrated(row['calibration'])
    lookup={q['name']:q for q in read(row['sapiens_json']]['keypoints']}
    active=list(MAX_DELTA_DEG);ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device=
e=dev);caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]))
    b0=t(rigid['body_pose']).reshape(21,3);g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl']).reshape
15 (3)
    init_path=OUT/'frames'/row['condition']/row['camera']/'body_params.npz'
    initp=p
    delta=(t(initp['body_pose']).reshape(21,3)-b0)[ai].clone().requires_grad_();gd=(t(initp['global_orient']).res
hape(3)-g0).clone().requires_grad_();tr=t(initp['transl']).reshape(3).clone().requires_grad_()
20 names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]['y
']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
    bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([[lookup[n]['x'],lookup[n]['y
']] for n in names]);bw=t([min(1,lookup[n]['score'])**2 for n in names]);bw/=bw.mean()
    ix=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);goals=t([w['joints_2d'][m['candidate'],
25 0] for s,m in matches.items()])
    _srcj=geometry(source);_rigj=geometry(rigid);floor=t(np.maximum(np.linalg.norm(srcj[[12,13]],axis=1
),np.linalg.norm(rigj[[12,13]],axis=1))*.85)
    bodylimit=float(np.sqrt(np.mean(np.sum((cam.world2camera(srcj.astype(np.float64))[bi.cpu().numpy()]-
arr(bt))**2,axis=1))))+5
30 sv,sj=geometry(p);initial_werr=np.linalg.norm(cam.world2camera(sj.astype(np.float64))[ix.cpu().numpy(
)]-arr(goals),axis=1)
    loopids=[];loopgoals=[]
    for s,m in matches.items():
        ids=oldmesh[s+' _local_to_mesh'];bv=oldmesh[s+' _wrist_loop_local'];loopids.append(torch.tensor(ids[bv
35 ],device=dev))
        uv=cam.world2camera(w['vertices_3d_local'][m['candidate']]+w['cam_t'][m['candidate']]);uv=uv[bv];loop
goals.append(t(uv-uv.mean(0)))
    sil_data=prepare(targets(row,p,matches,w,cam,dst),dev)
    sil_before=silhouette_metrics(sv,cam,sil_data)
40 def params():
    pp={k:t(p[k]).reshape(1,-1) for k in PARAMS};body=b0.clone();body[ai]+=delta;pp.update(body_pose=
body.reshape(1,-1),global_orient=(g0+gd).reshape(1,3));return pp
    def forward():
        o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;uv=cam.world2camera_pytorc
45 h(j);return v,j,uv
    def feasible(v,j,uv):
        err=(uv[ix]-goals).norm(dim=1)
        return bool(v[:,2].min()>=.020001 and (j[[12,13]].norm(dim=1)>=floor+1e-6).all() and torch.sqrt((uv[bi
-bt).square().sum(1).mean())<=bodylimit-1e-4 and (err<=(initial_werr)+3).all())
50 def objective(v,j,uv):

```

```

    body=(F.smooth_l1_loss((uv[bi]-bt)/t([1280,720]),torch.zeros_like(bt),beta=.003,reduction='none').mean(1)*bw).mean()
    wrists=F.smooth_l1_loss((uv[ix]-goals)/t([1280,720]),torch.zeros_like(goals),beta=.003)
    seam=[]
5   for ids,target in zip(loopids,loopgoals):
        q=cam.world2camera_pytorch(v[ids]);q=q-q.mean(0);radius=q.square().sum(1).mean().sqrt();rr=target.square().sum(1).mean().sqrt();seam.append(torch.log(radius/rr.clamp_min(1)).square())
        shape=torch.stack(seam).mean()
        priors=.0008*(delta/caps[:,None]).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean()+.000
10   4*(gd/np.deg2rad(8)).square().mean()
        depth=.02*F.relu((.03-v[:,2].min())/.02).square();guard=.002*F.relu((floor-j[[12,13]].norm(dim=1))/0.05).square().mean()
        return body+4*wrists+.0008*shape+priors+depth+guard+.08*silhouette_loss(v,cam,sil_data)
        opt=torch.optim.Adam([{'params':[delta],'lr':.0015},{'params':[gd],'lr':.0005},{'params':[tr],'lr':.0008}])
15   with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.detach().clone() for x in [delta,gd,tr]]
        rejected=0;start=time.time()
        for step in range(a.steps):
            opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward()
20   previous=[x.detach().clone() for x in [delta,gd,tr]];opt.step()
            if a.active_projection:
                # Project infeasible Adam proposals back onto nonlinear constraint surfaces.
                for project_step in range(12):
                    with torch.no_grad():
25   delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
                    pv,pj,pu=forward()
                    br=(pu[bi]-bt).square().sum(1).mean().sqrt()
                    violations=torch.cat([(br-bodylimit+.02).reshape(1)/5,(.02001-pv[:,2].min()).reshape(1)/.02,(floor+1e-5-pj[[12,13]].norm(dim=1))/0.05,((pu[ix]-goals).norm(dim=1)-t(initial_werr)-2.99)/5])
30   g=violations.max()
                    if float(g)<=0:break
                    grads=torch.autograd.grad(g,[delta,gd,tr]);scales=[.0015,.0005,.0008]
                    denom=sum((grad.square().sum()*scale**2) for grad,scale in zip(grads,scales)).clamp_min(1e-12)
35   with torch.no_grad():
                        for x,grad,scale in zip([delta,gd,tr],grads,scales):x.sub_(1.05*g.detach()*grad*scale**2/denom)
                    with torch.no_grad():
                        delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
40   ok=False
                        for trial in range(10):
                            v,j,uv=forward()
                            if feasible(v,j,uv):ok=True;break
                            for x,prev in zip([delta,gd,tr],previous):x.copy_((x+prev)*.5)
45   if not ok:
                            rejected+=1
                            for x,prev in zip([delta,gd,tr],previous):x.copy_(prev)
                            v,j,uv=forward()
                            value=float(objective(v,j,uv))
50   if ok and value<bestloss:bestloss=value;best=[x.detach().clone() for x in [delta,gd,tr]]

```

```

    if step%100==0:print(row['id'],step,'wrist',arr((uv[ix]-goals).norm(dim=1)).round(2).tolist(),'rejects',rejec
ted,flush=True)
    with torch.no_grad():
        for x,b in zip([delta,gd,tr],best):x.copy_(b)
5      pp=params();saved={k:x.copy() for k,x in p.items()}
        for k,x in pp.items():saved[k]=arr(x).reshape(p[k].shape)
        saved['transl']=arr(tr).reshape(p['transl'].shape)
        candidate={k:x.copy() for k,x in saved.items()};cpu_alpha=1.
        for trial in range(22):
10       for k in ['body_pose','global_orient','transl']:saved[k]=p[k]+cpu_alpha*(candidate[k]-p[k])
            try:acceptance=constraints(saved,source,rigid,cam,lookup);break
            except AssertionError:cpu_alpha*=.9
        else:
            saved={k:x.copy() for k,x in p.items()};cpu_alpha=0.;acceptance=constraints(saved,source,rigid,cam,loo
15      kup)
            vv,jj=geometry(saved);local=jj-saved['transl'].reshape(3);pixels=cam.world2camera(jj.astype(np.float64))
            .astype(np.float32)
            saved.update(smplx_mesh_cam=vv,smplx_joint_cam=jj,smplx_joint_img=pixels,smplx_body_joints_3d_
            local=local[:25],smplx_body_joints_3d_render_cam=jj[:25],smplx_body_joints_2d_render_cam=pixels[:25
20      ],smplx_joint_cam_root_relative=jj-jj[:1])
            if 'smplx_body_joints_3d_cam' in saved:saved['smplx_body_joints_3d_cam']=jj[:25]
            if 'smplx_body_joints_2d' in saved:saved['smplx_body_joints_2d']=pixels[:25]
            for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(saved[k],p[k])
            # Fixed beta kinematic limb lengths (selected joints) are conserved under pose.
25      lengths=lambda j:np.array([np.linalg.norm(j[a]-j[b]) for a,b in [(8,10),(10,12),(9,11),(11,13)])])
            boneerr=float(np.max(np.abs(lengths(jj)-lengths(sj))));assert boneerr<1e-5
            np.savez_compressed(dst/'body_params.npz',**saved)
            report=dict(id=row['id'],active_constraint_projection=a.active_projection,steps=a.steps,cpu_feasibility_st
            ep_fraction=cpu_alpha,elapsed_seconds=time.time()-start>manual_inputs=False,original_hard_bounds_pres
30      erved=True,body_shape_fixed=True,bone_length_max_change_m=boneerr,wrist_before_px=initial_werr.t
            olist(),wrist_after_px=np.linalg.norm(cam.world2camera(jj.astype(np.float64))[ix.cpu().numpy()]-arr(goals
            ),axis=1).tolist(),matches=matches,loss=bestloss,rejected_steps=rejected,acceptance=acceptance)
            report.update(silhouette_before=sil_before,silhouette_after=silhouette_metrics(vv,cam,sil_data),silhouett
            e_weight=.08,previous_identity_preserved=True)
35      write(dst/'refinement.json',report);print('DONE',row['id'],report['wrist_after_px'],flush=True)
    if __name__=='__main__':main()
文件名: egosmplx_pipeline/reference/contour_refine_strong.py
功能: Fixed-shape SMPL-X wrist and projected seam compatibility refinement.
    """Fixed-shape SMPL-X wrist and projected seam compatibility refinement."""
40      from pathlib import Path
            import os
            import sys,copy,argparse,time
            import numpy as np
            import torch
45      import torch.nn.functional as F
            OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour'
            BASE=OUT.parent/'baseline'
            from egosmplx_pipeline.reference.support import (read,write,load,arr,geometry,constraints,PARAMS,FIX
            ED,SIDES,JI,MAX_DELTA_DEG,BODY_POSE_NAMES,JOINT_MAPPING,smpl_x,FishEyeCameraCa
50      librated)

```

```

from egosmplx_pipeline.reference.support import native_projection
from egosmplx_pipeline.reference.silhouette import targets,prepare,loss as silhouette_loss,metrics as silhouette_metrics,PREV
def main():
5   ap=argparse.ArgumentParser();ap.add_argument('--steps',type=int,default=500);ap.add_argument('--indices',type=int,nargs='*');ap.add_argument('--active-projection',action='store_true');a=ap.parse_args()
    torch.backends.cuda.matmul.allow_tf32=False;torch.backends.cudnn.allow_tf32=False
    torch.set_num_threads(2);dev='cuda' if torch.cuda.is_available() else 'cpu';layer=copy.deepcopy(smpl_x.layer['neutral']).to(dev).eval();layer.requires_grad_(False)
10   t=lambda x:torch.as_tensor(x, dtype=torch.float32, device=dev)
    rows=read(BASE/'automatic_input_manifest.json')['frames']
    for ii,row in enumerate(rows):
        if a.indices and ii not in a.indices:continue
        dst=OUT/'strong_frames'/row['condition']/row['camera'];dst.mkdir(parents=True,exist_ok=True)
15   if (dst/'refinement.json').exists():continue
        bd=BASE/'frames'/row['condition']/row['camera'];p=load(PREV/'final_frames'/row['condition']/row['camera']/'body_params.npz');source=load(row['source.npz']);rigid=load(row['rigid.npz']);w=load(row['wilor.npz']);oldmesh=load(bd/'hybrid_mesh.npz');matches=read(bd/'recipe.json')['matches'];cam=FishEyeCameraCalibrated(row['calibration'])
20   lookup={q['name']:q for q in read(row['sapiens.json']]['keypoints']}
        active=list(MAX_DELTA_DEG);ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device=dev);caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]))
        b0=t(rigid['body_pose']).reshape(21,3);g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl']).reshape(3)
25   init_path=OUT/'frames'/row['condition']/row['camera']/'body_params.npz'
        initp=p
        delta=(t(initp['body_pose']).reshape(21,3)-b0)[ai].clone().requires_grad_();gd=(t(initp['global_orient']).reshape(3)-g0).clone().requires_grad_();tr=(t(initp['transl']).reshape(3).clone().requires_grad_())
        names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
30   bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([[lookup[n]['x'],lookup[n]['y']] for n in names]);bw=t([min(1,lookup[n]['score'])**2 for n in names]);bw/=bw.mean()
        ix=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);goals=t([w['joints_2d'][m['candidate'],0] for s,m in matches.items()])
35   _,srcj=geometry(source);_,rigj=geometry(rigid);floor=t(np.maximum(np.linalg.norm(srcj[[12,13]],axis=1),np.linalg.norm(rigi[[12,13]],axis=1))*0.85)
        bodylimit=float(np.sqrt(np.mean(np.sum((cam.world2camera(srcj.astype(np.float64))[bi.cpu().numpy()-arr(bt))**2,axis=1])))+5
        sv,sj=geometry(p);initial_werr=np.linalg.norm(cam.world2camera(sj.astype(np.float64))[ix.cpu().numpy()-arr(goals),axis=1])
40   loopids=[];loopgoals=[]
        for s,m in matches.items():
            ids=oldmesh[s+' _local_to_mesh'];bv=oldmesh[s+' _wrist_loop_local'];loopids.append(torch.tensor(ids[bv],device=dev))
45   uv=native_projection(w['vertices_3d_local'][m['candidate']] + w['cam_t'][m['candidate']]);uv=uv[bv];loopgoals.append(t(uv-uv.mean(0)))
        sil_data=targets(row,p,matches,w,cam,dst)
        for sd in sil_data:
            side=sd["side"];sd["ids"]=np.unique(np.r_[sd["ids"],oldmesh[side+" _local_to_mesh"][oldmesh[side+" _wrist_loop_local"]]])
50   wrist_loop_local"]]])

```

```

sil_data=prepare(sil_data,dev)
sil_before=silhouette_metrics(sv,cam,sil_data)
def params():
    pp={k:t(p[k]).reshape(1,-1) for k in PARAMS};body=b0.clone();body[ai]+=delta;pp.update(body_pose=
5 body.reshape(1,-1),global_orient=(g0+gd).reshape(1,3));return pp
    def forward():
        o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;uv=cam.world2camera_pytorc
h(j);return v,j,uv
    def feasible(v,j,uv):
10 err=(uv[ix]-goals).norm(dim=1)
        return bool(v[:,2].min())>=.020001 and (j[[12,13]].norm(dim=1)>=floor+1e-6).all() and torch.sqrt((uv[bi]
-bt).square().sum(1).mean())<=bodylimit-1e-4 and (err<=t(initial_werr)+3).all())
    def objective(v,j,uv):
        body=(F.smooth_l1_loss((uv[bi]-bt)/t([1280,720]),torch.zeros_like(bt),beta=.003,reduction='none').mean
15 (1)*bw).mean()
        wrists=F.smooth_l1_loss((uv[ix]-goals)/t([1280,720]),torch.zeros_like(goals),beta=.003)
        seam=[]
        for ids,target in zip(loopids,loopgoals):
            q=cam.world2camera_pytorch(v[ids]);q=q-q.mean(0);radius=q.square().sum(1).mean().sqrt();rr=target.s
20 quare().sum(1).mean().sqrt();seam.append(torch.log(radius/rr.clamp_min(1)).square())
            shape=torch.stack(seam).mean()
            priors=.0008*(delta/caps[:,None]).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean()+.000
4*(gd/np.deg2rad(8)).square().mean()
            depth=.02*F.relu((.03-v[:,2].min())/t([.10,.10,.18])).square();guard=.002*F.relu((floor-j[[12,13]].norm(dim=1))/t([.10,.10,.18])).square().mean()
25 square().mean()
            return body+4*wrists+.0008*shape+priors+depth+guard+.4*silhouette_loss(v,cam,sil_data)
        opt=torch.optim.Adam([{'params':[delta],'lr':.0015},{'params':[gd],'lr':.0005},{'params':[tr],'lr':.0008}])
        with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.detach().clone() for x in [d
elta,gd,tr]]
30 rejected=0;start=time.time()
        for step in range(a.steps):
            opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward()
            previous=[x.detach().clone() for x in [delta,gd,tr]];opt.step()
            if a.active_projection:
35 # Project infeasible Adam proposals back onto nonlinear constraint surfaces.
                for project_step in range(12):
                    with torch.no_grad():
                        delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);g
d.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
40 pv,pj,pu=forward()
                        br=(pu[bi]-bt).square().sum(1).mean().sqrt()
                        violations=torch.cat([(br-bodylimit+.02).reshape(1)/5,(.02001-pv[:,2].min()).reshape(1)/.02,(floor+1e-5-
pj[[12,13]].norm(dim=1))/t([.10,.10,.18])).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean()+.000
45 g=violations.max()
                        if float(g)<=0:break
                        grads=torch.autograd.grad(g,[delta,gd,tr]);scales=[.0015,.0005,.0008]
                        denom=sum((grad.square().sum()*scale**2) for grad,scale in zip(grads,scales)).clamp_min(1e-12)
                        with torch.no_grad():
                            for x,grad,scale in zip([delta,gd,tr],grads,scales):x.sub_(1.05*g.detach()*grad*scale**2/denom)
50 with torch.no_grad():

```

```

    delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);gd
    .mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
    ok=False
    for trial in range(10):
5      v,j,uv=forward()
      if feasible(v,j,uv):ok=True;break
      for x,prev in zip([delta,gd,tr],previous):x.copy_((x+prev)*.5)
      if not ok:
        rejected+=1
10     for x,prev in zip([delta,gd,tr],previous):x.copy_(prev)
        v,j,uv=forward()
        value=float(objective(v,j,uv))
        if ok and value<bestloss:bestloss=value;best=[x.detach().clone() for x in [delta,gd,tr]]
        if step%100==0:print(row['id'],step,'wrist',arr((uv[ix]-goals).norm(dim=1)).round(2).tolist(),'rejects',rejec
15 ted,flush=True)
        with torch.no_grad():
          for x,b in zip([delta,gd,tr],best):x.copy_(b)
          pp=params();saved={k:x.copy() for k,x in p.items()}
          for k,x in pp.items():saved[k]=arr(x).reshape(p[k].shape)
20         saved['transl']=arr(tr).reshape(p['transl'].shape)
          candidate={k:x.copy() for k,x in saved.items()};cpu_alpha=1.
          for trial in range(22):
            for k in ['body_pose','global_orient','transl']:saved[k]=p[k]+cpu_alpha*(candidate[k]-p[k])
            try:acceptance=constraints(saved,source,rigid,cam,lookup);break
25         except AssertionError:cpu_alpha*=.9
        else:
          saved={k:x.copy() for k,x in p.items()};cpu_alpha=0.;acceptance=constraints(saved,source,rigid,cam,loo
          kup)
          vv,jj=geometry(saved);local=jj-saved['transl'].reshape(3);pixels=cam.world2camera(jj.astype(np.float64))
30         .astype(np.float32)
          saved.update(smplx_mesh_cam=vv,smplx_joint_cam=jj,smplx_joint_img=pixels,smplx_body_joints_3d
          local=local[:25],smplx_body_joints_3d_render_cam=jj[:25],smplx_body_joints_2d_render_cam=pixels[:25
          ],smplx_joint_cam_root_relative=jj-jj[:1])
          if 'smplx_body_joints_3d_cam' in saved:saved['smplx_body_joints_3d_cam']=jj[:25]
35         if 'smplx_body_joints_2d' in saved:saved['smplx_body_joints_2d']=pixels[:25]
          for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(saved[k],p[k])
          # Fixed beta kinematic limb lengths (selected joints) are conserved under pose.
          lengths=lambda j:np.array([np.linalg.norm(j[a]-j[b]) for a,b in [(8,10),(10,12),(9,11),(11,13)])])
          boneerr=float(np.max(np.abs(lengths(jj)-lengths(sj))));assert boneerr<1e-5
40         np.savez_compressed(dst/'body_params.npz',**saved)
          report=dict(id=row['id'],active_constraint_projection=a.active_projection,steps=a.steps,cpu_feasibility_st
          ep_fraction=cpu_alpha,elapsed_seconds=time.time()-start>manual_inputs=False,original_hard_bounds_pres
          erved=True,body_shape_fixed=True,bone_length_max_change_m=boneerr,wrist_before_px=initial_werr.t
          olist(),wrist_after_px=np.linalg.norm(cam.world2camera(jj.astype(np.float64))[ix.cpu().numpy()]-arr(goals
45         ),axis=1).tolist(),matches=matches,loss=bestloss,rejected_steps=rejected,acceptance=acceptance)
          report.update(silhouette_before=sil_before,silhouette_after=silhouette_metrics(vv,cam,sil_data),silhouett
          e_weight=.4,previous_identity_preserved=True)
          write(dst/'refinement.json',report);print('DONE',row['id'],report['wrist_after_px'],flush=True)
          if __name__=='__main__':main()
50 文件名: egosmplx_pipeline/reference/contour_seam.py

```

功能: Standard-shape bounded wrist loop plus harmonic displacement on both sides.

""""Standard-shape bounded wrist loop plus harmonic displacement on both sides.""""

from pathlib import Path

import os

5 import sys, argparse

from collections import deque

import numpy as np, cv2

from scipy.sparse import coo_matrix

from scipy.sparse.linalg import spsolve

10 from scipy.spatial.transform import Rotation

OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour';BASE=OUT.parent/'baseline'

from egosmplx_pipeline.reference.support import (read, write, load, geometry, stitch, draw_geometry, FACES

, SIDES, smpl_x, FishEyeCameraCalibrated, write_obj, read_obj, constraints)

def adjacency(faces, n):

15 a=[set() for _ in range(n)]

for face in faces:

for x, y in zip(face, np.roll(face, -1)): a[int(x)].add(int(y)); a[int(y)].add(int(x))

return a

def distance(a, seeds):

20 d=np.full(len(a), 9999, dtype=int); q=deque()

for v in seeds: d[v]=0; q.append(int(v))

while q:

v=q.popleft()

for u in a[v]:

25 if d[u]>d[v]+1: d[u]=d[v]+1; q.append(u)

return d

def harmonic(a, known, values, free):

""""Dirichlet boundary displacements; everything outside free/known is fixed.""""

n=len(a); disp=np.zeros((n, 3)); disp[known]=values; idx=np.full(n, -1, int); idx[free]=np.arange(len(free)); rr=[

30]; cc=[]; vv=[]; rhs=np.zeros((len(free), 3))

for r, v in enumerate(free):

rr.append(r); cc.append(r); vv.append(len(a[v]))

for u in a[v]:

if idx[u]>=0: rr.append(r); cc.append(idx[u]); vv.append(-1)

35 else: rhs[r]+=disp[u]

if len(free): disp[free]=spsolve(coo_matrix((vv, (rr, cc)), shape=(len(free), len(free))).tocsr(), rhs)

return disp

def normals(v, f): return np.cross(v[f[:, 1]]-v[f[:, 0]], v[f[:, 2]]-v[f[:, 0]])

def edges(f): return np.unique(np.sort(np.concatenate([f[:, [0, 1]], f[:, [1, 2]], f[:, [2, 0]]]), axis=1), axis=0)

40 def hand_metrics(v, target, f):

e=edges(f); rat=np.linalg.norm(v[e[:, 0]]-v[e[:, 1]], axis=1)/np.maximum(1e-10, np.linalg.norm(target[e[:, 0]]-target[e[:, 1]], axis=1))

n=normals(v, f); nt=normals(target, f)

return dict(edge_log_rms=float(np.sqrt(np.mean(np.log(np.maximum(rat, 1e-8))**2))), edge_ratio_p01=flo

45 at(np.percentile(rat, 1)), edge_ratio_p99=float(np.percentile(rat, 99)), normal_reversals_vs_target=int((np.sum

(n*nt, axis=1)<0).sum()), minimum_double_area=float(np.linalg.norm(n, axis=1).min()))

def build(p, w, matches, cam, sil_data=None):

from egosmplx_pipeline.reference.silhouette import metrics

sil_data=sil_data or []

50 base, reports=stitch(p, w, matches, cam); original, j=geometry(p); v=base['vertices_cam'].copy(); mf=np.asarra

```

y(w['mano_faces']);ha=adjacency(mf,778);report={}
neutral={k:x.copy() for k,x in p.items()}
for k in ['body_pose','global_orient','left_hand_pose','right_hand_pose']:neutral[k]=np.zeros_like(neutral[k])
5   nv,_=geometry(neutral)
   for s,a in matches.items():
       ids=base[s+'_local_to_mesh'];bv=base[s+'_wrist_loop_local'];global_b=ids[bv];target=base[s+'_lifted_vertices'];posed_loop=original[global_b]
       dx=posed_loop-posed_loop.mean(0);dy=target[bv]-target[bv].mean(0)
10   u,_vt=np.linalg.svd(dx.T@dy);rotation=u@vt
       if np.linalg.det(rotation)<0:u[:,-1]*=-1;rotation=u@vt
       rotvec=Rotation.from_matrix(rotation.T).as_rotvec();rn=np.linalg.norm(rotvec);rotvec*=min(1,np.deg2rad(30)/max(rn,1e-8));rotation=Rotation.from_rotvec(rotvec).as_matrix().T
       standard_radius=float(np.linalg.norm(nv[global_b]-nv[global_b].mean(0),axis=1).mean());posed_radius=
15   float(np.linalg.norm(dx,axis=1).mean());target_radius=float(np.linalg.norm(dy,axis=1).mean())
       scale=np.clip(target_radius/standard_radius,.9,1.1)*standard_radius/posed_radius
       center_move=target[bv].mean(0)-posed_loop.mean(0);center_move*=min(1,.015/max(np.linalg.norm(center_move),1e-10))
       goal_loop=dx@rotation*scale+posed_loop.mean(0)+center_move
20   internal=np.isin(FACES,ids).all(axis=1);bodyfaces=FACES[~internal];ba=adjacency(bodyfaces,len(v));bd=distance(ba,global_b);hd=distance(ha,bv)
       freebody=np.where((bd>0)&(bd<10))[0];freehand=np.where((hd>0)&(hd<7))[0]
       touched_faces=bodyfaces[np.isin(bodyfaces,np.concatenate([global_b,freebody])).any(axis=1)];be=edges(touched_faces);bn=normals(original,touched_faces);bl=np.linalg.norm(original[be[:,0]]-original[be[:,1]],axis=1)
25   before=hand_metrics(v[ids],target,mf);candidates=[]
       loop_candidates=[('toward_mano_'+str(alpha),posed_loop+alpha*(goal_loop-posed_loop)) for alpha in [0.,.25,.5,.75,1.]]
       for axisidx in range(3):
30   for deg in [-30.,-15.,15.,30.]:
           rv=np.zeros(3);rv[axisidx]=np.deg2rad(deg);rot=Rotation.from_rotvec(rv).as_matrix().T
           for radius_scale in [.9,1.]:
               loop_candidates.append(('axis%d_%g_scale%g'%(axisidx,deg,radius_scale),dx@rot*(radius_scale*standard_radius/posed_radius)+posed_loop.mean(0)+.5*center_move))
35   sidedata=[d for d in sil_data if d['side']==s]
       def loop_penalty(vertices):
           if not sidedata:return 0.
           uv=cam.world2camera(vertices[global_b].astype(np.float64)).astype(np.float32)
           dd=cv2.remap(sidedata[0]['outside'],uv[:,0,None],uv[:,1,None],cv2.INTER_LINEAR,borderMode=cv2.
40   BORDER_REPLICATE).ravel()
           return float(np.mean(dd**2)/10000)
           original_sil=metrics(v,cam,sidedata).get(s,dict(score=0.))
           original_sil['wrist_loop_outside_mse_normalized']=loop_penalty(v)
           original_sil['score']+=original_sil['wrist_loop_outside_mse_normalized']
45   for candidate_name,newloop in loop_candidates:
           alpha=candidate_name
           bdsp=harmonic(ba,global_b,newloop-posed_loop,freebody)
           hdsp=harmonic(ha,bv,newloop-target[bv],freehand)
           proposal=v.copy();proposal[freebody]=original[freebody]+bdsp[freebody];proposal[ids]=target+hdsp
50   ratio=np.linalg.norm(proposal[be[:,0]]-proposal[be[:,1]],axis=1)/bl

```

```

nn=normals(proposal,touched_faces);areas=np.linalg.norm(nn,axis=1)/np.maximum(1e-12,np.linalg.norm(bn,axis=1))
lr=float(np.linalg.norm(proposal[global_b]-proposal[global_b].mean(0),axis=1).mean()/standard_radius)
ok=bool(np.isfinite(proposal).all() and proposal[:,2].min()>=.02 and ratio.min()>=.70 and ratio.max()<=1
5 .40 and areas.min()>=.4 and areas.max()<=2.2 and (np.sum(nn*bn,axis=1)>0).all() and .85<=lr<=1.15)
hm=hand_metrics(proposal[ids],target,mf)
if hm['normal_reversals_vs_target']>before['normal_reversals_vs_target']:ok=False
# Separate geometric acceptance from 2D visual accuracy; no manual input.
px=cam.world2camera((smpl_x.orig_hand_regressor[s]@proposal).astype(np.float64));goal=w['joints_2
10 d'][a['candidate']];err=float(np.sqrt(np.mean(np.sum((px-goal)**2,axis=1))))
oldpx=cam.world2camera((smpl_x.orig_hand_regressor[s]@v).astype(np.float64));olderr=float(np.sqrt(np.mean(np.sum((oldpx-goal)**2,axis=1))))
if err>olderr+3:ok=False
sm=metrics(proposal,cam,sidedata).get(s,dict(score=0.))
15 sm['wrist_loop_outside_mse_normalized']=loop_penalty(proposal)
sm['score']+=sm['wrist_loop_outside_mse_normalized']
score=hm['edge_log_rms']+.002*err+.01*hm['normal_reversals_vs_target']+5*sm['score']
diag=dict(alpha=alpha,accepted=ok,hand_metrics=hm,hand_rmse_px=err,forearm_edge_ratio_min=float(ratio.min()),forearm_edge_ratio_max=float(ratio.max()),forearm_area_ratio_min=float(areas.min()),forearm
20 m_area_ratio_max=float(areas.max()),loop_radius_over_standard=lr,score=score,silhouette=sm)
candidates.append((diag,proposal))
originalpx=cam.world2camera((smpl_x.orig_hand_regressor[s]@v).astype(np.float64));originalerr=float(np.sqrt(np.mean(np.sum((originalpx-w['joints_2d'][a['candidate']])**2,axis=1))))
oldscore=before['edge_log_rms']+.002*originalerr+.01*before['normal_reversals_vs_target']+5*original_
25 sil['score']
valid=[c for c in candidates if c[0]['accepted'] and c[0]['score']<oldscore]
if valid:
    chosen,v=min(valid,key=lambda c:c[0]['score']);used=True
else:chosen=dict(alpha=None,hand_metrics=before,hand_rmse_px=originalerr,score=oldscore,silhouette=original_sil);used=False
30 report[s]=dict(changed=used,standard_wrist_radius_m=standard_radius,posed_wrist_radius_m=posed_radius,lifted_mano_wrist_radius_m=target_radius,baseline_hand_metrics=before,baseline_hand_rmse_px=originalerr,selected=chosen,candidates=[d for d,_ in candidates],wrist_displacement_max_m=float(np.linalg.norm(v[global_b]-original[global_b],axis=1).max()),forearm_support_vertices=int(len(freebody)),hand_transition_vertices=int(len(freehand)))
35 base['vertices_cam']=v.astype(np.float32);base['hand_joints_cam']=np.stack([smpl_x.orig_hand_regressor[s]@v for s in SIDES]).astype(np.float32)
for s in matches:
    base[s+'_legacy_blend_weights']=base.pop(s+'_blend_weights')
40 base['method']=np.array('fixed_shape_body_and_bounded_standard_wrist_harmonic_seam')
return base,report
文件名: egosmplx_pipeline/reference/finalize.py
功能: 项目程序模块。
from pathlib import Path
45 import os
import sys,argparse,shutil
import numpy as np,cv2
from egosmplx_pipeline.reference.silhouette import OUT,PREV,targets,metrics,read,write,load,geometry,
SIDES,FishEyeCameraCalibrated,smpl_x
50 from egosmplx_pipeline.reference.contour_seam import build,write_obj,read_obj,constraints

```

```

from egosmplx_pipeline.reference.wrist_seam import build as previous_build, draw_geometry
BASE=OUT.parent/'baseline'
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--indices',type=int,nargs='*');ap.add_argument('--verify',
5    action='store_true');a=ap.parse_args()
    for i,row in enumerate(read(BASE/'automatic_input_manifest.json')['frames']):
        if a.indices and i not in a.indices:continue
        src=OUT/'selected_frames'/row['condition']/row['camera'];dst=OUT/'final_frames'/row['condition']/row['c
amera'];dst.mkdir(parents=True,exist_ok=True)
10    if not (src/'refinement.json').exists():continue
        if (dst/'acceptance.json').exists() and not a.verify:continue
        p=load(src/'body_params.npz');w=load(row['wilor_npz']);matches=read(BASE/'frames'/row['condition']/r
ow['camera']/'recipe.json')['matches'];cam=FishEyeCameraCalibrated(row['calibration']);lookup={q['name']
:q for q in read(row['sapiens_json'])['keypoints']}
15    # Freeze exact target strips from pre-optimization geometry for fair before/after comparison.
        oldp=load(PREV/'final_frames'/row['condition']/row['camera']/'body_params.npz');data=targets(row,oldp,
matches,w,cam,dst)
        mesh,report=build(p,w,matches,cam,data);oldmesh=load(PREV/'final_frames'/row['condition']/row['came
ra']/'hybrid_mesh.npz');bodyonly,_=previous_build(p,w,matches,cam)
20    check=constraints(p,load(row['source_npz']),load(row['rigid_npz']),cam,lookup);check.update(identities_
unchanged=True>manual_annotations_used=False,topology_preserved=bool(np.array_equal(mesh['faces'],o
ldmesh['faces'])),minimum_mesh_depth_m=float(mesh['vertices_cam'][:,2].min()))
        assert check['topology_preserved'] and check['minimum_mesh_depth_m']>=.02
        rec=dict(id=row['id'],condition=row['condition'],camera=row['camera'],hands=report,silhouette_baseline=
25    metrics(oldmesh['vertices_cam'],cam,data),silhouette_body_only=metrics(bodyonly['vertices_cam'],cam,da
ta),silhouette_final=metrics(mesh['vertices_cam'],cam,data))
        if a.verify:
            saved=load(dst/'hybrid_mesh.npz');assert set(saved)==set(mesh)
            for k in mesh:assert np.array_equal(saved[k],mesh[k]),k
30    v,f=read_obj(dst/'hybrid_mesh.obj');assert np.array_equal(f,mesh['faces']) and np.max(np.abs(v-mesh['ve
rtices_cam']))<1e-7
            check.update(independent_reload=True,vertex_reload_max_abs_m=0.)
            else:
                np.savez_compressed(dst/'hybrid_mesh.npz',**mesh);np.savez_compressed(dst/'body_only_mesh.npz',*
35    *bodyonly);shutil.copyfile(src/'body_params.npz',dst/'body_params.npz');shutil.copyfile(src/'refinement.jso
n',dst/'refinement.json');write_obj(dst/'hybrid_mesh.obj',mesh['vertices_cam'],mesh['faces']);write(dst/'geom
etry_report.json',rec)
                im=cv2.imread(row['image']);cv2.imwrite(str(dst/'wireframe_skeleton.jpg'),draw_geometry(im,cam,mesh
));cv2.imwrite(str(dst/'body_only_wireframe.jpg'),draw_geometry(im,cam,bodyonly))
40    write(dst/'acceptance.json',check);print('VERIFIED' if a.verify else 'FINAL',row['id'],rec['silhouette_final
'],flush=True)
        if __name__=='__main__':main()
文件名: egosmplx_pipeline/reference/guided_body.py
功能: MANO-guided continuation of the wrist/contour body; original seam stays separate.
45    """MANO-guided continuation of the wrist/contour body; original seam stays separate.
        Numerical optimization migrated from the verified 2026-09-29 eight-frame run.
        The manifest is built from this run's automatic observations, never manual data.
        """
        from pathlib import Path
50    import os,argparse,copy,time,hashlib

```

```

import numpy as np,cv2,torch
import torch.nn.functional as TF
from egosmplx_pipeline.reference.support import (read,write,load,arr,rmse,geometry,constraints,PARAMS
,FIXED,SIDES,JI,MAX_DELTA_DEG,BODY_POSE_NAMES,JOINT_MAPPING,smpl_x,FishEyeCame
5 raCalibrated,FACES,write_obj,render,stitch,draw_geometry,native_projection)
from egosmplx_pipeline.reference.silhouette import prepare,loss as sil_loss,metrics as sil_metrics
ROOT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])
OUT=ROOT/'guided'
PREV=ROOT/'contour'
10 HAND_NAMES={s:[p+' Wrist']+[p+'_'+f+'_'+str(j) for f in ['Thumb','Index','Middle','Ring','Pinky'] for j i
n range(1,5)] for s,p in [('left','L'),('right','R')]}
def get_rows():
    rows=read(ROOT/'baseline/automatic_input_manifest.json')['frames']
    result=[]
15     for original in rows:
        row=original.copy();rel=Path(row['condition']/row['camera'];previous=PREV/'final_frames'/rel
        row.update(initial_body_npz=str(previous/'body_params.npz'),initial_mesh_npz=str(previous/'hybrid_me
sh.npz'),recipe_json=str(ROOT/'baseline/frames'/rel/'recipe.json'),labels_npy=str(ROOT/'segmentation/fram
es'/rel/'labels.npy'),silhouette_targets_json=str(previous/'silhouette_targets.json'))
20     row['snapshot_sha256']={key:hashlib.sha256(Path(row[key]).read_bytes()).hexdigest() for key in ['image'
,'calibration','source_npz','rigid_npz','sapiens_json','wilor_npz','initial_body_npz','initial_mesh_npz','recipe_
json','labels_npy','silhouette_targets_json']}
    result.append(row)
    return result
25 def frame_dir(row):return OUT/'frames'/row['condition']/row['camera']
def context(row):
    for key,digest in row['snapshot_sha256'].items():assert hashlib.sha256(Path(row[key]).read_bytes()).hexdi
gest()==digest,(row['id'],key)
    p=load(row['initial_body_npz']);source=load(row['source_npz']);rigid=load(row['rigid_npz']);w=load(row[
30 'wilor_npz']);old=load(row['initial_mesh_npz'])
    matches=read(row['recipe_json']]['matches'];cam=FishEyeCameraCalibrated(row['calibration']);lookup={
    q['name']:q for q in read(row['sapiens_json']]['keypoints']}
    skin=np.isin(np.load(row['labels_npy']),[6,7,11,15,16,20]);outside=cv2.distanceTransform((~skin).astype(
np.uint8),cv2.DIST_L2,5);data=[]
35     for side,q in read(row['silhouette_targets_json']).items():
        if q['enabled']:data.append(dict(side=side,ids=np.array(q['vertex_ids']),outside=outside,contour=np.array(
[b[k] for b in q['bins'] for k in ['left','right']],np.float32)))
    return p,source,rigid,w,old,matches,cam,lookup,data
def metrics(p,w,old,matches,cam,lookup,data):
40     v,j=geometry(p);uv=cam.world2camera(j.astype(np.float64));names=[n for n in JOINT_MAPPING if loo
kup[n]['score']>=.3 and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
    errs={n:float(np.linalg.norm(uv[JI[JOINT_MAPPING[n]]]-[lookup[n]['x'],lookup[n]['y']])) for n in names
    }
    hands={}
45     for side,m in matches.items():
        hi=JI[n] for n in HAND_NAMES[side];goal=w['joints_2d'][m['candidate']];loop=old[side+'_wrist_loop
_local'];ids=old[side+'_local_to_mesh'];bodyloop=v[ids[loop]]
        target=native_projection(w['vertices_3d_local'][m['candidate']]+w['cam_t'][m['candidate']])[loop];puv=ca
m.world2camera(bodyloop.astype(np.float64))
50     rad=lambda x:float(np.sqrt(np.mean(np.sum((x-x.mean(0))**2,axis=1))))

```

```

hands[side]=dict(wrist_px=float(np.linalg.norm(uv[12+SIDES.index(side)]-goal[0])),palm_mcp_rmse_px
=rmse(uv[np.array(hi)[[5,9,13,17]],goal[[5,9,13,17]]),loop_rmse_px=rmse(puv,target),loop_centered_rmse
_px=rmse(puv-puv.mean(0),target-target.mean(0)),loop_radius_ratio=rad(puv)/rad(target),wrist_distance_
m=float(np.linalg.norm(j[12+SIDES.index(side)])))
5   return dict(body_rmse_px=float(np.sqrt(np.mean(np.array(list(errs.values()))**2))),body_errors_px=errs,h
ands=hands,silhouette=sil_metrics(v,cam,data))
def fit(row,steps):
    dst=frame_dir(row);dst.mkdir(parents=True,exist_ok=True)
    p,source,rigid,w,old,matches,cam,lookup,data=context(row)
10   dev='cuda' if torch.cuda.is_available() else 'cpu';torch.set_num_threads(2);torch.backends.cuda.matmul.all
ow_tf32=False;torch.backends.cudnn.allow_tf32=False
    layer=copy.deepcopy(smpl_x.layer['neutral']).to(dev).eval();layer.requires_grad_(False);t=lambda x:torch.
as_tensor(x, dtype=torch.float32, device=dev)
    data=prepare(data,dev);names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and 0<=lookup[n]['
15   x']<1280 and 0<=lookup[n]['y']<720]
    bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([lookup[n]['x'],lookup[n]['y']
] for n in names]);bw=t([lookup[n]['score']**2 for n in names]);bw/=bw.mean()
    active=list(MAX_DELTA_DEG);ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device
=dev);caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]));b0=t(rigid['body_pose']).reshape(21,3
20   );g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl']).reshape(3)
    delta=(t(p['body_pose']).reshape(21,3)-b0[ai].clone().requires_grad_());gd=(t(p['global_orient']).reshape(3
)-g0).clone().requires_grad_();tr=(t(p['transl']).reshape(3).clone().requires_grad_());variables=[delta,gd,tr]
    _sj=geometry(source);_rj=geometry(rigid);floor=t(np.maximum(np.linalg.norm(sj[[12,13]],axis=1),np.li
nalg.norm(rj[[12,13]],axis=1))*0.85)
25   _initial_j=geometry(p)
    bodylimit=min(rmse(cam.world2camera(sj.astype(np.float64))[bi.cpu().numpy()],arr(bt))+5,rmse(cam.wor
ld2camera(initial_j.astype(np.float64))[bi.cpu().numpy()],arr(bt))+.5)
    wi=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);wg=t([w['joints_2d'][m['candidate'],0]
for s,m in matches.items()])
30   ids=[];goals=[];pis=[];pgs=[]
    for s,m in matches.items():
        ids.append(torch.tensor(ol[s+'local_to_mesh'][ol[s+'wrist_loop_local']],device=dev));goals.append(t(
native_projection(w['vertices_3d_local'][m['candidate']]+w['cam_t'][m['candidate']])[ol[s+'wrist_loop_lo
cal']]))
35   pis.append(torch.tensor([JI[HAND_NAMES[s][k]] for k in [5,9,13,17]],device=dev));pgs.append(t(w['joi
nts_2d'][m['candidate'],[5,9,13,17]]))
    pp0={k:t(p[k]).reshape(1,-1) for k in PARAMS}
    def params():
        pp=pp0.copy();body=b0.clone();body[ai]+=delta;pp.update(body_pose=body.reshape(1,-1),global_orient
40   =(g0+gd).reshape(1,3));return pp
    def forward():
        o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;return v,j,cam.world2camera_p
ytorch(j)
    def huber(a,b,beta=20):return TF.smooth_l1_loss(a,b,beta=beta,reduction='none')*beta/10000
45   def objective(v,j,uv):
        residual=uv[bi]-bt
        body=(huber(uv[bi],bt).mean(1)*bw).mean()+.15*(residual.square().mean(1)*bw).mean()/10000
        wrist=huber(uv[wi],wg).mean()
        palm=torch.stack([huber(uv[ix],g).mean() for ix,g in zip(pis,pgs)]).mean()
50   loop=torch.stack([huber(cam.world2camera_pytorch(v[ix]),g).mean() for ix,g in zip(ids,goals)]).mean()

```

```

    priors=.003*(delta/caps[:,None]).square().mean()+.001*((tr-tr0)/t([.10,.10,.18])).square().mean()+.001*(g
d/np.deg2rad(8)).square().mean()
    return 4*body+4*wrist+1*palm+5*loop+1.5*sil_loss(v,cam,data)+priors
    def clip():
5      delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);gd.
mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
      def violations(v,j,uv):return torch.cat([(torch.sqrt((uv[bi]-bt).square().sum(1).mean())-bodylimit+.01).resh
ape(1)/10,(.0205-v[:,2].min()).reshape(1)/.02,(floor+1e-5-j[[12,13]].norm(dim=1))/.05])
      def valid(v,j,uv):return bool(violations(v,j,uv).max()<=0)
10     opt=torch.optim.Adam([{'params':[delta],'lr':.002},{'params':[gd],'lr':.0008},{'params':[tr],'lr':.0015}]);start
=time.time();trace=[]
    with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.clone() for x in variables]
    initial=metrics(p,w,old,matches,cam,lookup,data);rejected=0;last_improvement=0
    for step in range(steps):
15     opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward();previous=[x.detach().clone() for
x in variables];opt.step()
        for _ in range(8):
            with torch.no_grad():clip()
            v,j,uv=forward();g=violations(v,j,uv).max()
20         if float(g)<=0:break
            grads=torch.autograd.grad(g,variables);scales=[.002,.0008,.0015];den=sum(a.square().sum()*s*s for a,s i
n zip(grads,scales)).clamp_min(1e-12)
            with torch.no_grad():
                for x,a,s in zip(variables,grads,scales):x.sub_(1.05*g.detach()*a*s*s/den)
25         with torch.no_grad():
            clip();ok=False
            for _ in range(10):
                v,j,uv=forward()
                if valid(v,j,uv):ok=True;break
30         for x,pr in zip(variables,previous):x.copy_((x+pr)*.5)
            if not ok:
                rejected+=1
                for x,pr in zip(variables,previous):x.copy_(pr)
                v,j,uv=forward()
35         value=float(objective(v,j,uv))
            if ok and value<bestloss-1e-7:bestloss=value;best=[x.clone() for x in variables];last_improvement=step
            if step%100==0 or step==steps-1:
                rec=dict(step=step,loss=value,body_rmse=float(torch.sqrt((uv[bi]-bt).square().sum(1).mean()))),wrist=arr
((uv[wi]-wg).norm(dim=1)).tolist(),loop=[float(torch.sqrt((cam.world2camera_pytorch(v[ix])-g).square().su
40 m(1).mean())) for ix,g in zip(ids,goals)],rejected=rejected)
                trace.append(rec);print(row['condition'],row['camera'],rec,flush=True)
            if step-last_improvement>=200:break
            with torch.no_grad():
                for x,b in zip(variables,best):x.copy_(b)
45         q={k:x.copy() for k,x in p.items()}
            for k,x in params().items():q[k]=arr(x).reshape(p[k].shape)
            q['transl']=arr(tr).reshape(p['transl'].shape)
            candidate={k:x.copy() for k,x in q.items()};np.savez_compressed(dst/'gpu_candidate.npz',**candidate);alp
ha=1.
50         for trial in range(30):

```

```

for k in ['body_pose','global_orient','transl']:q[k]=p[k]+alpha*(candidate[k]-p[k])
try:
    check=constraints(q,source,rigid,cam,lookup)
    assert metrics(q,w,old,matches,cam,lookup,data)['body_rmse_px']<=bodylimit+.001
5     break
    except AssertionError:alpha*=.9
else:q={k:x.copy() for k,x in p.items()};alpha=0.;check=constraints(q,source,rigid,cam,lookup)
v,j=geometry(q);pix=cam.world2camera(j.astype(np.float64)).astype(np.float32)
q.update(smplx_mesh_cam=v,smplx_joint_cam=j,smplx_joint_img=pix,smplx_body_joints_3d_local=(j-
10 ql['transl'].reshape(3))[:25],smplx_body_joints_3d_render_cam=j[:25],smplx_body_joints_2d_render_cam=
    pix[:25],smplx_joint_cam_root_relative=j-j[:1])
    for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(p[k],q[k])
    np.savez_compressed(dst/'body_params.npz',**q);vv,jj=geometry(load(dst/'body_params.npz'));assert np.
    array_equal(v,vv)
15     write_obj(dst/'body_mesh.obj',v,FACES);cv2.imwrite(str(dst/'body_wireframe.jpg'),render(cv2.imread(ro
        w['image']),v.astype(np.float64),FACES,cam,.12))
    after=metrics(q,w,old,matches,cam,lookup,data)
    lengths=lambda a:np.array([np.linalg.norm(a[i]-a[k]) for i,k in [(8,10),(10,12),(9,11),(11,13)])])
    _j0=geometry(p);boneerr=float(np.max(np.abs(lengths(j)-lengths(j0))));assert boneerr<1e-5
20     write(dst/'optimization.json',dict(row=row,baseline='Current run automatic wrist/contour-refined body',alg
        orithm_source='EgoFishPose/06f5472',method='single continuation; Sapiens joints + native MANO MCP a
        nd 16 corresponding wrist-loop pixels + frozen contour; no body-shifted hand targets',manual_inputs=False,
        max_steps=steps,actual_steps=step+1,early_stop_patience=200,elapsed_seconds=time.time()-start,initial=i
        nitial,after=after,trace=trace,acceptance=check,fixed_shape=True,cpu_feasibility_step_fraction=alpha,body
25     _bone_max_change_m=boneerr,body_reload_max_abs_m=float(np.abs(vv-v).max()))
    print('DONE',row['condition'],after,flush=True)
    def main():
        ap=argparse.ArgumentParser(description=__doc__);ap.add_argument('--steps',type=int,default=1400);a=a
        p.parse_args()
30     if a.steps<1:raise ValueError('steps must be positive')
        for row in get_rows():
            if not (frame_dir(row)/'optimization.json').exists():fit(row,a.steps)
        if __name__=='__main__':main()
    文件名: egosmplx_pipeline/reference/guided_fuse.py
35    功能: Fuse the enhanced body using the unchanged original bounded contour seam.
        """"Fuse the enhanced body using the unchanged original bounded contour seam.""""
    import argparse
    import numpy as np,cv2
    from egosmplx_pipeline.reference.guided_body import (get_rows,frame_dir,context,read,write,load,geome
40    try,constraints,smpl_x,SIDES,FACES,render,draw_geometry,write_obj,metrics)
    from egosmplx_pipeline.reference.contour_seam import build,adjacency,distance
    from egosmplx_pipeline.fusion import edge_counts,oriented_edges
    from egosmplx_pipeline.render import read_obj
    from egosmplx_pipeline.reference.silhouette import metrics as silhouette_metrics
45    from egosmplx_pipeline.reference import surface_audit as audit
    def run(row,verify=False):
        dst=frame_dir(row);p0,source,rigid,w,old,matches,cam,lookup,data=context(row);p=load(dst/'body_param
        s.npz')
        v,j=geometry(p);assert np.max(np.abs(v-p['smplx_mesh_cam']))<1e-6
50    bodycheck=constraints(p,source,rigid,cam,lookup)

```

```

new,seam=build(p,w,matches,cam,data);new['method']=np.array('MANO_guided_body_plus_original_bounded_contour_seam')
assert np.array_equal(new['faces'],old['faces']);assert float(new['vertices_cam'][:,2].min())>=.02-1e-6
counts=edge_counts(new['faces']);directions=oriented_edges(new['faces']);topology=True
5   for side in matches:
        seeds=set(map(int,new[side+' _local_to_mesh'][new[side+' _wrist_loop_local']]))
        for (a,b),n in counts.items():
            if a in seeds and b in seeds:topology &= n==2 and directions[a,b]==directions[b,a]==1
        assert topology
10  if verify:
        saved=load(dst/'hybrid_mesh.npz');assert set(saved)==set(new)
        for key in new:assert np.array_equal(saved[key],new[key]),key
        ov,of=read_obj(dst/'hybrid_mesh.obj');err=float(np.abs(ov-new['vertices_cam']).max());assert np.array_equal(of,new['faces']) and err<1e-7
15  write(dst/'reload_validation.json',dict(passed=True,independent_reconstruction=True,vertex_max_abs_m=.0,obj_vertex_max_abs_m=err,face_topology_exact=True));print('VERIFIED',row['id'],flush=True);return
        np.savez_compressed(dst/'hybrid_mesh.npz',**new);write_obj(dst/'hybrid_mesh.obj',new['vertices_cam'],new['faces']);write(dst/'seam_report.json',seam)
        allhands=np.concatenate([new[s+' _local_to_mesh'] for s in matches]);bf=FACES[~np.isin(FACES,allhands)].all(1);adj=adjacency(bf,len(v));weights=smpl_x.layer['neutral'].lbs_weights.detach().cpu().numpy();checks={}
20  for side in matches:
        si=SIDES.index(side);seeds=new[side+' _local_to_mesh'][new[side+' _wrist_loop_local']];d=distance(adj,seeds)
25  # Same set of anatomical forearm vertices in before/after; local support vs full mesh.
        eligible=(weights[:,18+si]+weights[:,20+si]>.5)&~np.isin(np.arange(len(v)),allhands)
        support=np.flatnonzero(((d>0)&(d<14)&eligible))np.isin(np.arange(len(v)),seeds)
        checks[side]=dict(support_vertices=support.tolist(),before=audit.local_intersections(old['vertices_cam'].astype(float),old['faces'],support),after=audit.local_intersections(new['vertices_cam'].astype(float),new['faces'],support))
30  rec=dict(route='MANO-guided body + original bounded contour seam',manual_inputs=False,original_seam_code_unchanged=True,fixed_mano_wrist_algorithm_used=False,matches=matches,body_constraints=bodycheck,seam_edges_two_opposite_faces=True,topology_unchanged=True,minimum_depth_m=float(new['vertices_cam'][:,2].min()),silhouette_before=silhouette_metrics(old['vertices_cam'],cam,data),silhouette_after=silhouette_metrics(new['vertices_cam'],cam,data),local_intersection_checks=checks,all_local_crossing_checks_passed=all(q['after']['proper_crossings']==0 for q in checks.values()),intersection_scope='Local forearm/seam faces versus full surface; shared-vertex pairs excluded; no coplanar overlap or whole-body collision certificate')
        write(dst/'geometry_validation.json',rec)
40  im=cv2.imread(row['image']);cv2.imwrite(str(dst/'fusion_wireframe.jpg'),render(im,new['vertices_cam'].astype(float),new['faces'],cam,.12));cv2.imwrite(str(dst/'fusion_skeleton.jpg'),draw_geometry(im.copy(),cam,new))
        print('FUSED',row['id'],rec['all_local_crossing_checks_passed'],flush=True)
    def main():
45  ap=argparse.ArgumentParser(description=__doc__);ap.add_argument('--verify',action='store_true');a=ap.parse_args()
        for row in get_rows():
            required='geometry_validation.json' if a.verify else 'optimization.json'
            if not (frame_dir(row)/required).exists():raise RuntimeError('Previous stage not ready: '+row['id'])
50  if not a.verify and (frame_dir(row)/'geometry_validation.json').exists():continue

```

```

    run(row,a.verify)
if __name__=='__main__':main()
文件名: egosmplx_pipeline/reference/pipeline.py
功能: Run the archived wrist/contour stages after the two-stage body fit.
5  """Run the archived wrist/contour stages after the two-stage body fit.
    The numerical worker functions are migrated from the reference ZIP. This module
    only adapts paths, invokes them in the recorded order, and exports verified data.
    """

    import argparse
10    import os
    from pathlib import Path
    import shutil
    import numpy as np
    import cv2
15    from egosmplx_pipeline.io import read, write, load, sha
    from egosmplx_pipeline.configuration import PACKAGE_ROOT
    from egosmplx_pipeline.pipeline import command, environment
    from egosmplx_pipeline.profiles import CURRENT, LEGACY, selected
    PROFILE = LEGACY # Historical compatibility; select the configured profile for every run.
20    def prepare(rows, output):
        root = output / 'reference'
        prepared = []
        for row in rows:
            frame = output / 'frames' / row['id']
25            baseline = frame / 'baseline_fused'
            if not baseline.exists():
                shutil.copytree(frame / 'fused', baseline)
            matches = read(baseline / 'recipe.json')['matches']
            if not matches:
30                continue
            if cv2.imread(row['image']).shape[1::-1] != (1280, 720):
                raise ValueError('The archived reference profile requires 1280 x 720 images')
            camera = row.get('camera', 'camera')
            if camera not in ('cam3', 'cam4', 'camera'):
35                raise ValueError('Expected cam3/cam4 for the reference profile')
            condition = row['id']
            destination = root / 'baseline/frames' / condition / camera
            destination.mkdir(parents=True, exist_ok=True)
            for source, name in [(frame / 'body/params.npz', 'body_params.npz'),
40                (baseline / 'mesh.npz', 'hybrid_mesh.npz'),
                (baseline / 'recipe.json', 'recipe.json')]:
                shutil.copyfile(source, destination / name)
            segmentation = root / 'segmentation/frames' / condition / camera
            segmentation.mkdir(parents=True, exist_ok=True)
45            for name in ['labels.npy', 'probabilities.npz']:
                target = segmentation / name
                source = Path(row['predictions']['segmentation']) / name
                if target.exists() and sha(target) != sha(source):
                    raise ValueError('Changed segmentation input: ' + row['id'])
50            if not target.exists():

```

```

        target.symlink_to(source.resolve())
    prepared.append(dict(id=row['id'], condition=condition, camera=camera,
        image=row['image'], calibration=row['calibration'],
        source_npz=str(frame / 'body/params.npz'),
5         rigid_npz=str(frame / 'rigid/optimized_params.npz'),
        sapiens_json=row['predictions']['sapiens'],
        wilor_npz=str(baseline / 'wilor_predictions.npz'))
    write(root / 'baseline/automatic_input_manifest.json', dict(frames=prepared))
    return prepared
10 def run_workers(config, rows, output):
    root = output / 'reference'
    env = environment(config)
    env['EGOSMPLX_REFERENCE_ROOT'] = str(root)
    def execute(module, arguments, log_name):
15         command([config['paths']['body_python'], '-m',
            'egosmplx_pipeline.reference.' + module] + arguments,
            root / (log_name + '.log'), PACKAGE_ROOT, env)
    execute('wrist_refine', ['--steps', '500'], '01_wrist')
    execute('wrist_refine', ['--steps', '400', '--active-projection'], '02_wrist_active')
20     for row in rows:
        relative = Path(row['condition']) / row['camera']
        source, destination = root / 'wrist/active_frames' / relative, root / 'wrist/final_frames' / relative
        destination.mkdir(parents=True, exist_ok=True)
        for name in ['body_params.npz', 'refinement.json']:
25             shutil.copyfile(source / name, destination / name)
    execute('wrist_seam', ['--frame-dir', 'final_frames'], '03_wrist_surface')
    execute('contour_refine', ['--steps', '400', '--active-projection'], '04_contour_standard')
    execute('contour_refine_strong', ['--steps', '400', '--active-projection'], '05_contour_strong')
    execute('select_candidates', [], '06_select')
30     execute('finalize', [], '07_surface')
    execute('finalize', ['--verify'], '08_independent_reload')
    if selected(config) == CURRENT:
        execute('guided_body', ['--steps', str(config.get('fitting', {}).get('mano_guided_steps', 1400))], '09_ma
no_guided_body')
35         execute('guided_fuse', [], '10_original_seam')
        execute('guided_fuse', ['--verify'], '11_guided_independent_reload')
    def export(config, rows, prepared, output):
        os.environ['EGOSMPLX_REPO_ROOT'] = config['paths']['egosmplx_repo']
        os.environ['EGOSMPLX_CONFIG'] = config['paths']['egosmplx_config']
40         os.environ['EGOSMPLX_REFERENCE_ROOT'] = str(output / 'reference')
        from egosmplx_pipeline.reference.support import FishEyeCameraCalibrated, geometry, render, draw_ge
ometry
        from egosmplx_pipeline.render import read_obj
        mapping = {row['id']: row for row in prepared}
45         reports = []
        for row in rows:
            frame = output / 'frames' / row['id']
            report = read(frame / 'validation.json')
            report['pipeline_profile'] = selected(config)
50         if row['id'] in mapping:

```

```

reference_row = mapping[row['id']]
relative = Path(reference_row['condition']) / reference_row['camera']
legacy_source = output / 'reference/contour/final_frames' / relative
source = legacy_source
5 fused = frame / 'fused'
if selected(config) == CURRENT:
    source = output / 'reference/guided/frames' / relative
    reload = read(source / 'reload_validation.json')
    if not reload.get('passed') or not reload.get('independent_reconstruction'):
10         raise ValueError('Enhanced surface did not pass independent reconstruction')
    audit = read(source / 'geometry_validation.json')
    check = dict(audit['body_constraints'], independent_reload=True,
                 vertex_reload_max_abs_m=reload['vertex_max_abs_m'],
                 topology_preserved=audit['topology_unchanged'],
15                 local_surface_accepted=audit['all_local_crossing_checks_passed'])
    write(fused / 'acceptance.json', check)
    for name, renamed in [(('body_params.npz', 'body_params.npz'), ('hybrid_mesh.npz', 'mesh.npz'),
                           ('hybrid_mesh.obj', 'mesh.obj'), ('optimization.json', 'optimization.json'),
                           ('seam_report.json', 'seam_report.json'), ('geometry_validation.json', 'geometry_valid
20 ation.json'),
                           ('reload_validation.json', 'reload_validation.json'))]:
        shutil.copyfile(source / name, fused / renamed)
    for name in ['refinement.json', 'silhouette_targets.json']:
        shutil.copyfile(legacy_source / name, fused / name)
25 write(fused / 'geometry_report.json', dict(hands=read(source / 'seam_report.json'),
        local_intersection_checks=audit['local_intersection_checks'],
        all_local_crossing_checks_passed=audit['all_local_crossing_checks_passed']))
    guided = frame / 'guided_body'
    guided.mkdir(exist_ok=True)
30 for name, renamed in [(('body_params.npz', 'params.npz'), ('body_mesh.obj', 'mesh.obj'),
                           ('body_wireframe.jpg', 'wireframe.jpg'), ('optimization.json', 'optimization.json'))]:
        shutil.copyfile(source / name, guided / renamed)
    report.update(mano_guided_body=True, local_surface_accepted=audit['all_local_crossing_checks_passed'],
35                 surface_audit_scope=audit['intersection_scope'])
else:
    acceptance = read(source / 'acceptance.json')
    if not acceptance.get('independent_reload'):
        raise ValueError('Final surface did not pass independent reconstruction')
40 for name, renamed in [(('body_params.npz', 'body_params.npz'), ('hybrid_mesh.npz', 'mesh.npz'),
                           ('hybrid_mesh.obj', 'mesh.obj'), ('refinement.json', 'refinement.json'),
                           ('geometry_report.json', 'geometry_report.json'), ('acceptance.json', 'acceptance.json
        '),
                           ('silhouette_targets.json', 'silhouette_targets.json'))]:
45         shutil.copyfile(source / name, fused / renamed)
    shutil.copyfile(Path(row['predictions'])['segmentation']) / 'labels.npy', fused / 'segmentation_labels.n
    py')
    parameters, mesh = load(fused / 'body_params.npz'), load(fused / 'mesh.npz')
    v, joints = geometry(parameters)
50 camera = FishEyeCameraCalibrated(row['calibration'])

```

```

        image = cv2.imread(row['image'])
        cv2.imwrite(str(fused / 'wireframe.jpg'), render(image, mesh['vertices_cam'].astype(np.float64), mesh['faces'], camera, .12))
        cv2.imwrite(str(fused / 'skeleton.jpg'), draw_geometry(image, camera, mesh))
5         targets = load(frame / 'body/params.npz')
        body_errors = np.linalg.norm(camera.world2camera(joints.astype(np.float64))[targets['stage2_selected_joint_indices'].astype(int)]
                                - targets['stage2_target_joint_pixels'], axis=1)
        hands = load(fused / 'wilor_predictions.npz')
10         matches = read(frame / 'baseline_fused/recipe.json')['matches']
        hand_errors = []
        for k, side in enumerate(['left', 'right']):
            if side in matches:
                pixels = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
15                 hand_errors.extend(np.linalg.norm(pixels-hands['joints_2d'][matches[side]['candidate']], axis=1).tolist())
        obj_v, obj_f = read_obj(fused / 'mesh.obj')
        obj_error = float(np.abs(obj_v-mesh['vertices_cam']).max())
        body_error = float(np.abs(v-parameters['smplx_mesh_cam']).max())
20         if obj_error >= 1e-7 or body_error > 1e-5 or not np.array_equal(obj_f, mesh['faces']):
            raise RuntimeError('Exported final geometry differs from verified mesh')
        if 'stage2_body_error_px' not in report:
            report['stage2_body_error_px'] = report['body_error_px']
            report['stage2_body_rmse_px'] = report['body_rmse_px']
25         report.update(body_error_px=body_errors.tolist(), hand_error_px=hand_errors,
                        body_rmse_px=float(np.sqrt(np.mean(body_errors**2))),
                        hand_rmse_px=float(np.sqrt(np.mean(np.square(hand_errors)))),
                        body_reload_max_abs_m=body_error, fusion_reload_max_abs_m=0., obj_reload_max_abs_m=obj_error,
30                        minimum_fused_vertex_z_m=float(mesh['vertices_cam'][:, 2].min()),
                        hand_reports=read(fused / 'geometry_report.json')['hands'], local_surface_optimizer=True,
                        selected_body_candidate=read(fused / 'refinement.json')['selected_candidate'])
        write(fused / 'recipe.json', dict(matches=matches, pipeline_profile=selected(config),
            method=('MANO_guided_body_plus_original_bounded_contour_seam' if selected(config) ==
35            CURRENT else 'bounded_standard_wrist_harmonic_seam_with_visible_contour'),
            body_parameter_path='body_params.npz', frozen_targets='silhouette_targets.json',
            segmentation_labels='segmentation_labels.npy', local_surface_optimizer=True,
            baseline_smoothstep_rings=7, manual_annotations_used=False,
            standard_smplx_parameter_only_reconstruction=False))
40         else:
            report.update(local_surface_optimizer=False, mano_guided_body=False, local_surface_accepted=
None, reference_refinement_skipped='no_matched_hand')
            report['artifact_sha256'] = {str(p.relative_to(frame)): sha(p) for p in frame.rglob('*')
                if p.is_file() and p != frame / 'validation.json'}
45         write(frame / 'validation.json', report)
        reports.append(report)
        write(output / 'reference/validation.json', dict(pipeline_profile=selected(config), frames=reports))
        return reports
def run(config, rows, output):
50     output = Path(output).resolve()

```

```

    prepared = prepare(rows, output)
    if prepared:
        run_workers(config, prepared, output)
    return export(config, rows, prepared, output)
5 def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--job', type=Path, required=True)
    args = parser.parse_args()
    job = read(args.job)
10 run(job['config'], job['rows'], job['output'])
if __name__ == '__main__':
    main()

```

文件名： egosmplx_pipeline/reference/rebuild_archive.py

功能： Reconstruct the specified result bundle from saved parameters and constraints.

15 """Reconstruct the specified result bundle from saved parameters and constraints.
This verifies the final mesh-producing algorithm. It does not rerun the networks
or the body/wrist optimizers, and does not need the omitted probability caches.
"""

```

import argparse
20 import hashlib
import io
import json
import os
from pathlib import Path
25 import zipfile
import cv2
import numpy as np
from egosmplx_pipeline.io import write, sha
REFERENCE_SHA256 = '19b9baf80998dbd3f6f0da8f620295251d97a2c9aeeb7c2d282d5c31835e7fd6'
30 def frozen_targets(labels, report):
    """Recover exactly the saved visible strips, without re-estimating targets."""
    skin = np.isin(labels, [6,7,11,15,16,20])
    outside = cv2.distanceTransform((~skin).astype(np.uint8), cv2.DIST_L2, 5)
    return [dict(side=side, ids=np.array(item['vertex_ids'], dtype=int), outside=outside,
35         contour=np.array([b[k] for b in item['bins'] for k in ['left','right']], np.float32))
        for side, item in report.items() if item['enabled']]
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--archive', type=Path, required=True)
40 parser.add_argument('--repo-root', type=Path, required=True)
    parser.add_argument('--config', type=Path, required=True, help='EgoSMPLX backend model config')
    parser.add_argument('--output', type=Path, required=True)
    args = parser.parse_args()
    if sha(args.archive) != REFERENCE_SHA256:
45     raise ValueError('This verifier is pinned to the exact session_hand6 result bundle')
    if args.output.exists():
        raise FileExistsError(args.output)
    args.output.mkdir(parents=True)
    os.environ['EGOSMPLX_REPO_ROOT'] = str(args.repo_root.resolve())
50 os.environ['EGOSMPLX_CONFIG'] = str(args.config.resolve())

```

```

os.environ['EGOSMPLX_REFERENCE_ROOT'] = str((args.output/'reference').resolve())
from egosmplx_pipeline.reference.support import geometry, FishEyeCameraCalibrated, smpl_x, render,
write_obj
from egosmplx_pipeline.reference.contour_seam import build
5 records = []
with zipfile.ZipFile(args.archive) as archive:
    def read(name):
        return json.loads(archive.read(name))
    def array(name):
10     with np.load(io.BytesIO(archive.read(name)), allow_pickle=False) as z:
        return {key:z[key].copy() for key in z.files}
    manifest = read('ARCHIVE_MANIFEST.json')
    for name, entry in manifest.items():
        content = archive.read(name)
15     if len(content) != entry['bytes'] or hashlib.sha256(content).hexdigest() != entry['sha256']:
        raise ValueError('Archive member hash mismatch: ' + name)
    cases = sorted(name[len('frames/'):-len('/03_fusion/body_params.npz')])
        for name in manifest if name.startswith('frames/') and name.endswith('/03_fusion/body_para
ms.npz'))
20     for case in cases:
        # No arbitrary ZIP paths are extracted. Only these generated case paths are used.
        condition, camera_name = case.split('/')
        if not condition.startswith('sample_') or camera_name not in ['cam3','cam4']:
            raise ValueError('Unexpected reference case')
25     d = args.output/'frames'/condition/camera_name
        d.mkdir(parents=True)
        calibration = d/'calibration.json'
        calibration.write_bytes(archive.read('calibration/'+camera_name+'.json'))
        camera = FishEyeCameraCalibrated(calibration)
30     prefix = 'frames/'+case+'/'
        parameters = array(prefix+'03_fusion/body_params.npz')
        hands = array(prefix+'automatic_observations/wilor_predictions.npz')
        hands.update(focal_length_px=np.array(25000.), image_size_wh=np.array([1280,720]))
        matches = read(prefix+'automatic_observations/identity_recipe.json')['matches']
35     labels = np.load(io.BytesIO(archive.read(prefix+'automatic_observations/segmentation_labels.npy')
), allow_pickle=False)
        targets = frozen_targets(labels, read(prefix+'03_fusion/silhouette_targets.json'))
        mesh, surface_report = build(parameters, hands, matches, camera, targets)
        expected = array(prefix+'03_fusion/hybrid_mesh.npz')
40     if set(mesh) != set(expected):
        raise RuntimeError('Rebuilt mesh fields differ from archive')
        equality = {key:bool(np.array_equal(mesh[key], expected[key])) for key in mesh}
        maximum = float(np.max(np.abs(mesh['vertices_cam']-expected['vertices_cam'])))
        if not all(equality.values()):
45         raise RuntimeError('Archive reconstruction differs: '+case+' '+str(equality))
        np.savez_compressed(d/'hybrid_mesh.npz', **mesh)
        write_obj(d/'hybrid_mesh.obj', mesh['vertices_cam'], mesh['faces'])
        image = cv2.imdecode(np.frombuffer(archive.read(prefix+'input.jpg'),np.uint8), cv2.IMREAD_CO
LOR)
50     cv2.imwrite(str(d/'projection.jpg'), render(image,mesh['vertices_cam'].astype(np.float64),mesh['fac

```

```

es'],camera,.12))
    errors = []
    for k, side in enumerate(['left','right']):
        if side in matches:
5            pixels = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
            errors.extend(np.linalg.norm(pixels-hands['joints_2d'][matches[side]['candidate']],axis=1).tolist())
    record = dict(case=case, all_fields_bitwise_equal=True, vertex_max_abs_m=maximum,
        matched_hands=len(matches), hand_error_px=errors, hand_rmse_px=float(np.sqrt(np.mean(np.square(errors)))))
10    selected_surface_candidates={s:r['selected'] for s,r in surface_report.items()}
    write(d/'verification.json', record)
    records.append(record)
    print('ARCHIVE_REBUILT',case,maximum,flush=True)
15    errors = [e for record in records for e in record['hand_error_px']]
    write(args.output/'archive_verification.json', dict(reference_archive_sha256=REFERENCE_SHA256,frames=len(records),
        verified_archive_members=len(manifest), all_mesh_fields_bitwise_equal=True,
        hand_rmse_px=float(np.sqrt(np.mean(np.square(errors)))),frames=records,
20    verification_scope='Final meshes rebuilt from archived body parameters, WiLoR predictions, identities and frozen contour targets; networks and optimizers were not rerun'))
    if __name__ == '__main__':
        main()

```

文件名: egosmplx_pipeline/reference/select_candidates.py

25 功能: 项目程序模块。

```

from pathlib import Path
import os
import sys,shutil
import numpy as np
30 from egosmplx_pipeline.reference.silhouette import OUT,PREV,targets,metrics,load,read,write,geometry,
    FishEyeCameraCalibrated,SIDES
    from egosmplx_pipeline.reference.support import constraints,JI,JOINT_MAPPING,MAX_DELTA_DEG,
        BODY_POSE_NAMES
    from egosmplx_pipeline.reference.support import native_projection
35 BASE=OUT.parent/'baseline'
def merit(p,row,lookup,w,matches,cam,oldmesh):
    v,j=geometry(p);uv=cam.world2camera(j.astype(np.float64));rig=load(row['rigid_npz'])
    names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
40    bi=[JI[JOINT_MAPPING[n]] for n in names];bt=np.array([lookup[n]['x'],lookup[n]['y']] for n in names);
    bw=np.array([min(1,lookup[n]['score'])*2 for n in names]);bw/=bw.mean()
    huber=lambda x:np.where(np.abs(x)<.003,x*(2*.003),np.abs(x)-.0015)
    body=float((huber((uv[bi]-bt)/[1280,720]).mean(1)*bw).mean());wrist=[];shape=[]
    for s,a in matches.items():
45    i=a['candidate'];k=12 if s=='left' else 13;wrist.append(huber((uv[k]-w['joints_2d'][i,0])/[1280,720]).mean())
    ids=oldmesh[s+' _local_to_mesh'][oldmesh[s+' _wrist_loop_local'];q=cam.world2camera(v[ids].astype(np.float64));q=q.mean(0)
    t=native_projection(w['vertices_3d_local'][i]+w['cam_t'][i])[oldmesh[s+' _wrist_loop_local'];t=t.mean(0)
50    shape.append(np.log(np.sqrt(np.mean(np.sum(q*q,axis=1)))/max(1,np.sqrt(np.mean(np.sum(t*t,axis=1)))))

```

```

))**2)
ai=[BODY_POSE_NAMES.index(n) for n in MAX_DELTA_DEG];caps=np.deg2rad(list(MAX_DELTA
_DEG.values()))
pp=.0008*np.mean(((p['body_pose'].reshape(21,3)[ai]-rig['body_pose'].reshape(21,3)[ai])/caps[:,None])**
5 2)
pp+=.0002*np.mean(((p['transl'].reshape(3)-rig['transl'].reshape(3)) / [.1,.1,.18])**2)+.0004*np.mean(((p['
global_orient']-rig['global_orient'])/np.deg2rad(8))**2)
depth=.02*max(0,(.03-v[:,2].min())/0.02)**2
return float(body+4*np.mean(wrist)+.0008*np.mean(shape)+pp+depth)
10 for row in read(BASE/'automatic_input_manifest.json')['frames']:
dst=OUT/'selected_frames'/row['condition']/row['camera'];dst.mkdir(parents=True,exist_ok=True)
bd=BASE/'frames'/row['condition']/row['camera'];matches=read(bd/'recipe.json')['matches'];om=load(bd/'h
ybrid_mesh.npz');w=load(row['wilor_npz']);cam=FishEyeCameraCalibrated(row['calibration']);lookup={q['
name']:q for q in read(row['sapiens_json'])['keypoints']};prev=PREV/'final_frames'/row['condition']/row['ca
15 mera'];p0=load(prev/'body_params.npz');data=targets(row,p0,matches,w,cam,dst)
for d in data:
s=d['side'];d['ids']=np.unique(np.r_[d['ids'],om[s+'_local_to_mesh'][om[s+'_wrist_loop_local']]])
paths={'previous':prev,'standard':OUT/'active_frames'/row['condition']/row['camera'],'strong':OUT/'strong
_frames'/row['condition']/row['camera']};scores={};diag={}
20 _j0=geometry(p0);inituv=cam.world2camera(j0.astype(np.float64))
for name,path in paths.items():
if not (path/'body_params.npz').exists():continue
p=load(path/'body_params.npz');constraints(p,load(row['source_npz']),load(row['rigid_npz']),cam,lookup)
;v,j=geometry(p);uv=cam.world2camera(j.astype(np.float64))
25 safe=all(np.linalg.norm(uv[12+SIDES.index(s)]-w['joints_2d'][a['candidate'],0])<=np.linalg.norm(inituv[
12+SIDES.index(s)]-w['joints_2d'][a['candidate'],0])+3.001 for s,a in matches.items())
mm=metrics(v,cam,data);sil=float(np.mean([x['score'] for x in mm.values()])) if mm else 0.;base=merit(p,
row,lookup,w,matches,cam,om)
diag[name]=dict(merit_without_silhouette=base,silhouette=sil,wrist_guard_passed=safe)
30 if safe:scores[name]=base+.4*sil
choice=min(scores,key=scores.get);shutil.copyfile(paths[choice]/'body_params.npz',dst/'body_params.npz'
)
report=read(paths[choice]/'refinement.json');report.update(selected_candidate=choice,selection_scores=sc
ores,selection_diagnostics=diag,matches=matches,original_identity_preserved=True)
35 write(dst/'refinement.json',report);print(row['id'],choice,scores,flush=True)
文件名: egosmplx_pipeline/reference/silhouette.py
功能: 项目程序模块。
from pathlib import Path
import os
40 import sys,json
import cv2,numpy as np,torch
import torch.nn.functional as F
OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour';PREV=OUT.parent/'wrist';SEG=
OUT.parent/'segmentation'
45 from egosmplx_pipeline.reference.wrist_seam import load,read,write,geometry,smpl_x,FishEyeCameraCal
ibrated,SIDES
def targets(row,p,matches,w,cam,dst):
v,j=geometry(p);lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']};labels=np.load(SEG/
'frames'/row['condition']/row['camera']/labels.npy);prob=np.load(SEG/'frames'/row['condition']/row['camer
50 a']/probabilities.npz')['probabilities'].astype(np.float32);skin=np.isin(labels,[6,7,11,15,16,20]);skinprob=pro

```

```

b[[6,7,11,15,16,20]].sum(0)
weights=smpl_x.layer['neutral'].lbs_weights.detach().cpu().numpy();allhands=np.concatenate(list(smpl_x.
hand_vertex_idx.values()));im=cv2.imread(row['image']);data=[];reports={}
for side,a in matches.items():
5   si=SIDES.index(side);q=lookup[side+'_elbow'];E=np.array([q['x'],q['y']]);W=w['joints_2d'][a['candidate'],
0];vec=W-E;length=np.linalg.norm(vec);u=vec/max(1,length);n=np.array([-u[1],u[0]]);half=min(70,max(2
5,length*.30));bins=[]
    if q['score']>=.3 and length>35:
        for t in np.linspace(.15,.88,12):
10       offsets=np.arange(-half,half+1);xy=E+t*vec+offsets[:,None]*n;ix=np rint(xy).astype(int);inside=(ix[:,0]
>=0)&(ix[:,0]<1280)&(ix[:,1]>=0)&(ix[:,1]<720);ok=np.zeros(len(ix),bool);ok[inside]=skin[ix[inside,1],ix
[inside,0]]&(skinprob[ix[inside,1],ix[inside,0]]>.55)
        starts=np.where(np.diff(np.r_[False,ok,False].astype(int))==1)[0];ends=np.where(np.diff(np.r_[False,ok,
False].astype(int))=-1)[0]
15       candidates=[(abs((offsets[l]+offsets[r-1])/2),l,r) for l,r in zip(starts,ends) if r-l>=8 and l>0 and r<len(off
ets)]
        if not candidates:continue
        _l,r=min(candidates);center=(offsets[l]+offsets[r-1])/2
        if abs(center)>max(25,length*.15):continue
20       bins.append(dict(t=float(t),lo=float(offsets[l]),hi=float(offsets[r-1]),left=xy[l].tolist(),right=xy[r-1].tolist(
)))
    # Only segments with repeated observations are eligible; no inference of hidden forearm.
    if len(bins)<3:reports[side]=dict(enabled=False,reason='insufficient_visible_forearm_cross_sections',bins
=bins);continue
25     alpha=np.array([x['t'] for x in bins]);tmin=max(.05,alpha.min()-.025);tmax=min(.96,alpha.max()+.025)
    elbow=j[10+si];wrist=j[12+si];axis=wrist-elbow;s=(v-elbow)@axis/(axis@axis);rad=np.linalg.norm(v-el
bow-s[:,None]*axis,axis=1)
    ids=np.where((weights[:,18+si]+weights[:,20+si]>.5)&(s>=tmin)&(s<=tmax)&(rad<.075)&~np.isin(np.ar
ange(len(v)),allhands))[0]
30     if len(ids)<12:reports[side]=dict(enabled=False,reason='too_few_model_forearm_vertices',bins=bins);con
tinue
    contour=np.array([x[k] for x in bins for k in ['left','right']],np.float32)
    # Target is the six-class union, not the model's erroneous left/right semantic labels.
    outside=cv2.distanceTransform((~skin).astype(np.uint8),cv2.DIST_L2,5)
35     data.append(dict(side=side,ids=ids,outside=outside,contour=contour,axis_origin=E,axis_vector=vec,tmin
=tmin,tmax=tmax))
    reports[side]=dict(enabled=True,bins=bins,vertex_count=len(ids),vertex_ids=ids.tolist(),tmin=tmin,tmax
=tmax,identity_source='unchanged_previous_WiLoR_assignment_and_Sapiens_elbow',occlusion_handling
='visible_forearm_cross_sections_only_no_hand_object_boundary_penalty')
40     for b in bins:cv2.line(im,tuple(np.rint(b['left']).astype(int)),tuple(np.rint(b['right']).astype(int)),(0,255,255)
,2)
    for uv in cam.world2camera(v[ids].astype(np.float64)):
        if np.isfinite(uv).all() and np.abs(uv).max()<5000:cv2.circle(im,tuple(np.rint(uv).astype(int)),1,(255,100,
0),-1)
45     dst.mkdir(parents=True,exist_ok=True);write(dst/'silhouette_targets.json',reports);cv2.imwrite(str(dst/'silh
ouette_targets.jpg'),im)
    return data
def prepare(data,device):
    for d in data:
50     d['tid']=torch.as_tensor(d['ids'],device=device);d['map']=torch.tensor(d['outside'],device=device)[None,No

```

```

ne];d['goal']=torch.tensor(d['contour'],device=device)
    return data
    def loss(v,cam,data):
        terms=[]
5        for d in data:
            uv=cam.world2camera_pytorch(v[d['tid']]);grid=uv/uv.new_tensor([1279,719])*2-1;outside=F.grid_sample(d['map'],grid.reshape(1,1,-1,2),align_corners=True,padding_mode='border').reshape(-1)
            overflow=F.relu(-uv)+F.relu(uv-uv.new_tensor([1279,719]));a=(outside.square().mean()+overflow.square().sum(1).mean())/10000
10           coverage=torch.cdist(d['goal'][None],uv[None])[0].min(1).values.square().mean()/10000
            terms.append(a+.5*coverage)
        return torch.stack(terms).mean() if terms else v.sum()*0
    def metrics(v,cam,data):
        result={}
15        for d in data:
            uv=cam.world2camera(v[d['ids']].astype(np.float64)).astype(np.float32);dist=cv2.remap(d['outside'],uv[:,0,None],uv[:,1,None],cv2.INTER_LINEAR,borderMode=cv2.BORDER_REPLICATE).ravel();coverage=np.linalg.norm(d['contour'][:,None]-uv[None],axis=2).min(1)
            result[d['side']]=dict(outside_rms_px=float(np.sqrt(np.mean(dist**2))),boundary_coverage_rms_px=float(np.sqrt(np.mean(coverage**2))),score=float((np.mean(dist**2)+.5*np.mean(coverage**2))/10000))
20        return result
    文件名: egosmplx_pipeline/reference/support.py
    功能: Project helpers for the archived algorithm; model implementations stay external.
    """"Project helpers for the archived algorithm; model implementations stay external.""""
25    import os
    from pathlib import Path
    import numpy as np
    from egosmplx_pipeline.bootstrap import setup
    from egosmplx_pipeline.io import read, write, load as _load, arr, rmse
30    from egosmplx_pipeline.body_common import BODY_POSE_NAMES, MAX_DELTA_DEG, JOINT_MAPPING
    from egosmplx_pipeline.matching import assign
    from egosmplx_pipeline import fusion
    from egosmplx_pipeline.render import write_obj, read_obj, render
35    from egosmplx_pipeline.lift import native_projection as _project
    smpl_x, FishEyeCameraCalibrated = setup(os.environ['EGOSMPLX_REPO_ROOT'])
    fusion.initialize(smpl_x)
    geometry, stitch, draw_geometry = fusion.geometry, fusion.stitch, fusion.draw_geometry
    PARAMS, SIDES, FACES, JI = fusion.PARAMS, fusion.SIDES, fusion.FACES, fusion.JI
40    FIXED = ['betas', 'jaw_pose', 'leye_pose', 'reye_pose', 'expression']
    def load(path):
        data = _load(path)
        if 'vertices_3d_local' in data and 'mano_faces' in data:
            # Exact archived WiLoR convention: 1280 x 720 and 25000 px focal.
45            if 'focal_length_px' in data:
                if float(data['focal_length_px']) != 25000. or not np.array_equal(data['image_size_wh'],[1280,720]):
                    raise ValueError('Reference profile requires the archived WiLoR projection convention')
                data['focal_length_px'] = np.array(25000.)
                data['image_size_wh'] = np.array([1280,720])
50            return data

```

```

def native_projection(points, focal=25000.):
    return _project(points, focal, [640.,360.])
def constraints(p, source, rigid, cam, lookup):
    v,j=geometry(p); _j0=geometry(source); _jr=geometry(rigid)
5    active=list(MAX_DELTA_DEG); ai=[BODY_POSE_NAMES.index(n) for n in active]
    cap=np.array([MAX_DELTA_DEG[n] for n in active])
    delta=np.rad2deg(np.linalg.norm((p['body_pose']-rigid['body_pose']).reshape(21,3)[ai],axis=1))
    assert np.all(delta<=cap+2e-4)
    assert all(np.array_equal(p[k],source[k]) for k in FIXED)
10    assert np.array_equal(p['body_pose'].reshape(21,3)[[9,10]],source['body_pose'].reshape(21,3)[[9,10]])
    assert np.linalg.norm(p['global_orient']-rigid['global_orient'])<=np.deg2rad(8)+1e-6
    assert .05<=p['transl'].reshape(3)[2]<=3
    floor=np.maximum(np.linalg.norm(j0[[12,13]],axis=1),np.linalg.norm(jr[[12,13]],axis=1))*0.85
    assert np.all(np.linalg.norm(j[[12,13]],axis=1)>=floor-1e-6)
15    assert v[:,2].min()>=.02-1e-6
    names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
    idx=[JI[JOINT_MAPPING[n]] for n in names]; target=np.array([[lookup[n]['x'],lookup[n]['y']] for n in names])
20    before=rmse(cam.world2camera(j0.astype(np.float64))[idx],target)
    after=rmse(cam.world2camera(j.astype(np.float64))[idx],target)
    assert after<=before+5.001
    return dict(passed=True,body_delta_deg=dict(zip(active,map(float,delta))),sapiens_body_rmse_px=after
    ,
25    sapiens_body_source_rmse_px=before,wrist_floor_m=floor.tolist(),body_minimum_z_m=float(
    v[:,2].min()))
文件名: egosmplx_pipeline/reference/surface_audit.py
功能: Local proper noncoplanar crossing checks; not a whole-body collision proof.
"""Local proper noncoplanar crossing checks; not a whole-body collision proof."""
30 import numpy as np
def segment_triangle(p,q,t):
    d=q-p;e1=t[:,1]-t[:,0];e2=t[:,2]-t[:,0]
    h=np.cross(d,e2);det=np.einsum('ij,ij->i',e1,h)
    valid=np.abs(det)>1e-14;inv=np.zeros_like(det);inv[valid]=1/det[valid]
35    s=p-t[:,0];u=inv*np.einsum('ij,ij->i',s,h);qq=np.cross(s,e1)
    v=inv*np.einsum('ij,ij->i',d,qq);a=inv*np.einsum('ij,ij->i',e2,qq)
    return valid&(u>1e-8)&(v>1e-8)&(u+v<1-1e-8)&(a>1e-8)&(a<1-1e-8)
def local_intersections(v,f,changed_body_ids):
    """Proper non-coplanar crossings, excluding faces sharing vertices.
40    The test is local: at least one face touches a changed forearm vertex.
    Coplanar overlap/contact is not certified by this test.
    """
    triangles=v[f];lo=triangles.min(1);hi=triangles.max(1)
    active=np.flatnonzero(np.isin(f,changed_body_ids).any(1));active_set=set(active.tolist())
45    pairs=[];tested=0
    for i in active:
        cand=np.flatnonzero((lo<=hi[i]+1e-10).all(1)&(hi>=lo[i]-1e-10).all(1))
        cand=np.array([j for j in cand if (j>i or j not in active_set) and not np.isin(f[j],f[i]).any()],int)
        if len(cand)==0:continue
50    a=np.broadcast_to(triangles[i],(len(cand),3,3));b=triangles[cand];hit=np.zeros(len(cand),bool)

```

```

    for k in range(3):
        hit|=segment_triangle(a[:,k],a[:,(k+1)%3],b)
        hit|=segment_triangle(b[:,k],b[:,(k+1)%3],a)
    tested+=len(cand)
5    pairs.extend([[int(i),int(j)] for j in cand[hit]])
    return dict(proper_crossings=len(pairs),face_pairs=pairs,candidate_face_pairs_tested=tested,
                local_faces_tested=len(active),coplanar_overlaps_checked=False)
文件名: egosmplx_pipeline/reference/wrist_refine.py
功能: Fixed-shape SMPL-X wrist and projected seam compatibility refinement.
10    """Fixed-shape SMPL-X wrist and projected seam compatibility refinement."""
    from pathlib import Path
    import os
    import sys,copy,argparse,time
    import numpy as np
15    import torch
    import torch.nn.functional as F
    OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'wrist'
    BASE=OUT.parent/'baseline'
    from egosmplx_pipeline.reference.support import (read,write,load,arr,geometry,constraints,PARAMS,FIX
20    ED,SIDES,JI,MAX_DELTA_DEG,BODY_POSE_NAMES,JOINT_MAPPING,smpl_x,FishEyeCameraCa
    librated)
    from egosmplx_pipeline.reference.support import native_projection
    def main():
        ap=argparse.ArgumentParser();ap.add_argument('--steps',type=int,default=500);ap.add_argument('--indice
25    s',type=int,nargs='*');ap.add_argument('--active-projection',action='store_true');a=ap.parse_args()
        torch.backends.cuda.matmul.allow_tf32=False;torch.backends.cudnn.allow_tf32=False
        torch.set_num_threads(2);dev='cuda' if torch.cuda.is_available() else 'cpu';layer=copy.deepcopy(smpl_x.la
        yer['neutral']).to(dev).eval();layer.requires_grad_(False)
        t=lambda x:torch.as_tensor(x, dtype=torch.float32, device=dev)
30    rows=read(BASE/'automatic_input_manifest.json')['frames']
        for ii,row in enumerate(rows):
            if a.indices and ii not in a.indices:continue
            dst=OUT/'active_frames' if a.active_projection else 'frames'/row['condition']/row['camera'];dst.mkdir(pa
            rents=True,exist_ok=True)
35    if (dst/'refinement.json').exists():continue
            bd=BASE/'frames'/row['condition']/row['camera'];p=load(bd/'body_params.npz');source=load(row['source
            _npz']);rigid=load(row['rigid_npz']);w=load(row['wilor_npz']);oldmesh=load(bd/'hybrid_mesh.npz');matche
            s=read(bd/'recipe.json')['matches'];cam=FishEyeCameraCalibrated(row['calibration'])
            lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']}
40    active=list(MAX_DELTA_DEG);ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device
            =dev);caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]))
            b0=t(rigid['body_pose']).reshape(21,3);g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl']).reshape
            (3)
            init_path=OUT/'frames'/row['condition']/row['camera']/'body_params.npz'
45    initp=load(init_path) if a.active_projection and init_path.exists() else p
            delta=(t(initp['body_pose']).reshape(21,3)-b0)[ai].clone().requires_grad_();gd=(t(initp['global_orient']).res
            hape(3)-g0).clone().requires_grad_();tr=t(initp['transl']).reshape(3).clone().requires_grad_()
            names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]['y'
            '])].all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
50    bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([[lookup[n]['x'],lookup[n]['y'

```

```

]] for n in names]);bw=t([min(1,lookup[n]['score'])*2 for n in names]);bw/=bw.mean()
ix=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);goals=t([w['joints_2d'][m['candidate'],
0] for s,m in matches.items()])
_,srcj=geometry(source);_,rigj=geometry(rigid);floor=t(np.maximum(np.linalg.norm(srcj[[12,13]],axis=1
5 ),np.linalg.norm(rigj[[12,13]],axis=1))*0.85)
bodylimit=float(np.sqrt(np.mean(np.sum((cam.world2camera(srcj.astype(np.float64))[bi.cpu().numpy()]-
arr(bt))*2,axis=1))))+5
sv,sj=geometry(p);initial_werr=np.linalg.norm(cam.world2camera(sj.astype(np.float64))[ix.cpu().numpy(
)]-arr(goals),axis=1)
10 loopids=[];loopgoals=[]
for s,m in matches.items():
ids=oldmesh[s+'_local_to_mesh'];bv=oldmesh[s+'_wrist_loop_local'];loopids.append(torch.tensor(ids[bv
],device=dev))
uv=native_projection(w['vertices_3d_local'][m['candidate']] + w['cam_t'][m['candidate']]);uv=uv[bv];loop
15 goals.append(t(uv-uv.mean(0)))
def params():
pp={k:t(p[k]).reshape(1,-1) for k in PARAMS};body=b0.clone();body[ai]+=delta;pp.update(body_pose=
body.reshape(1,-1),global_orient=(g0+gd).reshape(1,3));return pp
def forward():
20 o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;uv=cam.world2camera_pytorc
h(j);return v,j,uv
def feasible(v,j,uv):
err=(uv[ix]-goals).norm(dim=1)
return bool(v[:,2].min())>=.020001 and (j[[12,13]].norm(dim=1)>=floor+1e-6).all() and torch.sqrt((uv[bi
25 -bt).square().sum(1).mean())<=bodylimit-1e-4 and (err<=(initial_werr)+3).all())
def objective(v,j,uv):
body=(F.smooth_l1_loss((uv[bi]-bt)/t([1280,720]),torch.zeros_like(bt),beta=.003,reduction='none').mean
(1)*bw).mean()
wrists=F.smooth_l1_loss((uv[ix]-goals)/t([1280,720]),torch.zeros_like(goals),beta=.003)
30 seam=[]
for ids,target in zip(loopids,loopgoals):
q=cam.world2camera_pytorch(v[ids]);q=q-q.mean(0);radius=q.square().sum(1).mean().sqrt();rr=target.s
quare().sum(1).mean().sqrt();seam.append(torch.log(radius/rr.clamp_min(1)).square())
shape=torch.stack(seam).mean()
35 priors=.0008*(delta/caps[:,None]).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean()+.000
4*(gd/np.deg2rad(8)).square().mean()
depth=.02*F.relu((.03-v[:,2].min())/0.02).square();guard=.002*F.relu((floor-j[[12,13]].norm(dim=1))/0.05).
square().mean()
return body+4*wrists+.0008*shape+priors+depth+guard
40 opt=torch.optim.Adam([{'params':[delta],'lr':.0015},{'params':[gd],'lr':.0005},{'params':[tr],'lr':.0008}])
with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.detach().clone() for x in [d
elta,gd,tr]]
rejected=0;start=time.time()
for step in range(a.steps):
45 opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward()
previous=[x.detach().clone() for x in [delta,gd,tr]];opt.step()
if a.active_projection:
# Project infeasible Adam proposals back onto nonlinear constraint surfaces.
for project_step in range(12):
50 with torch.no_grad():

```

```

        delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);g
d.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
        pv,pj,pu=forward()
        br=(pu[bi]-bt).square().sum(1).mean().sqrt()
5        violations=torch.cat([(br-bodylimit+.02).reshape(1)/5,(.02001-pv[:,2].min()).reshape(1)/.02,(floor+1e-5-
pi[[12,13]].norm(dim=1))/.05,((pu[ix]-goals).norm(dim=1)-t(initial_werr)-2.99)/5])
        g=violations.max()
        if float(g)<=0:break
        grads=torch.autograd.grad(g,[delta,gd,tr]);scales=[.0015,.0005,.0008]
10        denom=sum((grad.square().sum()*scale**2) for grad,scale in zip(grads,scales)).clamp_min(1e-12)
        with torch.no_grad():
            for x,grad,scale in zip([delta,gd,tr],grads,scales):x.sub_(1.05*g.detach()*grad*scale**2/denom)
        with torch.no_grad():
            delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);gd
15        .mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
        ok=False
        for trial in range(10):
            v,j,uv=forward()
            if feasible(v,j,uv):ok=True;break
20        for x,prev in zip([delta,gd,tr],previous):x.copy_((x+prev)*.5)
        if not ok:
            rejected+=1
            for x,prev in zip([delta,gd,tr],previous):x.copy_(prev)
            v,j,uv=forward()
25        value=float(objective(v,j,uv))
        if ok and value<bestloss:bestloss=value;best=[x.detach().clone() for x in [delta,gd,tr]]
        if step%100==0:print(row['id'],step,'wrist',arr((uv[ix]-goals).norm(dim=1)).round(2).tolist(),'rejects',rejec
ted,flush=True)
        with torch.no_grad():
30        for x,b in zip([delta,gd,tr],best):x.copy_(b)
        pp=params();saved={k:x.copy() for k,x in p.items()}
        for k,x in pp.items():saved[k]=arr(x).reshape(p[k].shape)
        saved['transl']=arr(tr).reshape(p['transl'].shape)
        candidate={k:x.copy() for k,x in saved.items()};cpu_alpha=1.
35        for trial in range(22):
            for k in ['body_pose','global_orient','transl']:saved[k]=p[k]+cpu_alpha*(candidate[k]-p[k])
            try:acceptance=constraints(saved,source,rigid,cam,lookup);break
            except AssertionError:cpu_alpha*=.9
            else:
40            saved={k:x.copy() for k,x in p.items()};cpu_alpha=0.;acceptance=constraints(saved,source,rigid,cam,loo
kup)
            vv,jj=geometry(saved);local=jj-saved['transl'].reshape(3);pixels=cam.world2camera(jj.astype(np.float64))
            .astype(np.float32)
            saved.update(smplx_mesh_cam=vv,smplx_joint_cam=jj,smplx_joint_img=pixels,smplx_body_joints_3d_
45            local=local[:25],smplx_body_joints_3d_render_cam=jj[:25],smplx_body_joints_2d_render_cam=pixels[:25
],smplx_joint_cam_root_relative=jj-jj[:1])
            if 'smplx_body_joints_3d_cam' in saved:saved['smplx_body_joints_3d_cam']=jj[:25]
            if 'smplx_body_joints_2d' in saved:saved['smplx_body_joints_2d']=pixels[:25]
            for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(saved[k],p[k])
50        # Fixed beta kinematic limb lengths (selected joints) are conserved under pose.

```

```

lengths=lambda j:np.array([np.linalg.norm(j[a]-j[b]) for a,b in [(8,10),(10,12),(9,11),(11,13)])]
boneerr=float(np.max(np.abs(lengths(jj)-lengths(sj))));assert boneerr<1e-5
np.savez_compressed(dst/'body_params.npz',**saved)
report=dict(id=row['id'],active_constraint_projection=a.active_projection,steps=a.steps,cpu_feasibility_st
5 ep_fraction=cpu_alpha,elapsed_seconds=time.time()-start>manual_inputs=False,original_hard_bounds_pres
erved=True,body_shape_fixed=True,bone_length_max_change_m=boneerr,wrist_before_px=initial_werr.t
olist(),wrist_after_px=np.linalg.norm(cam.world2camera(jj.astype(np.float64))[ix.cpu().numpy()]-arr(goals
),axis=1).tolist(),matches=matches,loss=bestloss,rejected_steps=rejected,acceptance=acceptance)
write(dst/'refinement.json',report);print('DONE',row['id'],report['wrist_after_px'],flush=True)
10 if __name__=='__main__':main()
文件名: egosmplx_pipeline/reference/wrist_seam.py
功能: Standard-shape bounded wrist loop plus harmonic displacement on both sides.
"""Standard-shape bounded wrist loop plus harmonic displacement on both sides."""
from pathlib import Path
15 import os
import sys,argparse
from collections import deque
import numpy as np,cv2
from scipy.sparse import coo_matrix
20 from scipy.sparse.linalg import spsolve
from scipy.spatial.transform import Rotation
OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'wrist';BASE=OUT.parent/'baseline'
from egosmplx_pipeline.reference.support import (read,write,load,geometry,stitch,draw_geometry,FACES
,SIDES,smpl_x,FishEyeCameraCalibrated,write_obj,read_obj,constraints)
25 def adjacency(faces,n):
a=[set() for _ in range(n)]
for face in faces:
for x,y in zip(face,np.roll(face,-1)):a[int(x)].add(int(y));a[int(y)].add(int(x))
return a
30 def distance(a,seeds):
d=np.full(len(a),9999, dtype=int);q=deque()
for v in seeds:d[v]=0;q.append(int(v))
while q:
v=q.popleft()
35 for u in a[v]:
if d[u]>d[v]+1:d[u]=d[v]+1;q.append(u)
return d
def harmonic(a,known,values,free):
"""Dirichlet boundary displacements; everything outside free/known is fixed."""
40 n=len(a);disp=np.zeros((n,3));disp[known]=values;idx=np.full(n,-1,int);idx[free]=np.arange(len(free));rr=[
];cc=[];vv=[];rhs=np.zeros((len(free),3))
for r,v in enumerate(free):
rr.append(r);cc.append(r);vv.append(len(a[v]))
for u in a[v]:
45 if idx[u]>=0:rr.append(r);cc.append(idx[u]);vv.append(-1)
else:rhs[r]+=disp[u]
if len(free):disp[free]=spsolve(coo_matrix((vv,(rr,cc)),shape=(len(free),len(free))).tocsr(),rhs)
return disp
def normals(v,f):return np.cross(v[f[:,1]]-v[f[:,0]],v[f[:,2]]-v[f[:,0]])
50 def edges(f):return np.unique(np.sort(np.concatenate([f[:,[0,1]],f[:,[1,2]],f[:,[2,0]]]),axis=1),axis=0)

```

```

def hand_metrics(v,target,f):
    e=edges(f);rat=np.linalg.norm(v[e[:,0]]-v[e[:,1]],axis=1)/np.maximum(1e-10,np.linalg.norm(target[e[:,0]]-
target[e[:,1]],axis=1))
    n=normals(v,f);nt=normals(target,f)
5    return dict(edge_log_rms=float(np.sqrt(np.mean(np.log(np.maximum(rat,1e-8))*2))),edge_ratio_p01=float
at(np.percentile(rat,1)),edge_ratio_p99=float(np.percentile(rat,99)),normal_reversals_vs_target=int((np.sum
(n*nt,axis=1)<0).sum()),minimum_double_area=float(np.linalg.norm(n,axis=1).min()))
    def build(p,w,matches,cam):
        base,reports=stitch(p,w,matches,cam);original,j=geometry(p);v=base['vertices_cam'].copy();mf=np.asarra
10    y(w['mano_faces']);ha=adjacency(mf,778);report={ }
        neutral={k:x.copy() for k,x in p.items()}
        for k in ['body_pose','global_orient','left_hand_pose','right_hand_pose']:neutral[k]=np.zeros_like(neutral[k
])
        nv,_=geometry(neutral)
15    for s,a in matches.items():
        ids=base[s+'_local_to_mesh'];bv=base[s+'_wrist_loop_local'];global_b=ids[bv];target=base[s+'_lifted_ver
tices'];posed_loop=original[global_b]
        dx=posed_loop-posed_loop.mean(0);dy=target[bv]-target[bv].mean(0)
        u,_vt=np.linalg.svd(dx.T@dy);rotation=u@vt
20    if np.linalg.det(rotation)<0:u[:,-1]*=-1;rotation=u@vt
        rotvec=Rotation.from_matrix(rotation.T).as_rotvec();rm=np.linalg.norm(rotvec);rotvec*=min(1,np.deg2ra
d(30)/max(rm,1e-8));rotation=Rotation.from_rotvec(rotvec).as_matrix().T
        standard_radius=float(np.linalg.norm(nv[global_b]-nv[global_b].mean(0),axis=1).mean());posed_radius=
float(np.linalg.norm(dx,axis=1).mean());target_radius=float(np.linalg.norm(dy,axis=1).mean())
25    scale=np.clip(target_radius/standard_radius,.9,1.1)*standard_radius/posed_radius
        center_move=target[bv].mean(0)-posed_loop.mean(0);center_move*=min(1,.015/max(np.linalg.norm(ce
nter_move),1e-10))
        goal_loop=dx@rotation*scale+posed_loop.mean(0)+center_move
        internal=np.isin(FACES,ids).all(axis=1);bodyfaces=FACES[~internal];ba=adjacency(bodyfaces,len(v));b
30    d=distance(ba,global_b);hd=distance(ha,bv)
        freebody=np.where((bd>0)&(bd<10))[0];freehand=np.where((hd>0)&(hd<7))[0]
        touched_faces=bodyfaces[np.isin(bodyfaces,np.concatenate([global_b,freebody])).any(axis=1)];be=edges
(touched_faces);bn=normals(original,touched_faces);bl=np.linalg.norm(original[be[:,0]]-original[be[:,1]],a
xis=1)
35    before=hand_metrics(v[ids],target,mf);candidates=[]
        for alpha in [0.,.25,.5,.75,1.]:
            newloop=posed_loop+alpha*(goal_loop-posed_loop)
            bdsp=harmonic(ba,global_b,newloop-posed_loop,freebody)
            hdsp=harmonic(ha,bv,newloop-target[bv],freehand)
40    proposal=v.copy();proposal[freebody]=original[freebody]+bdsp[freebody];proposal[ids]=target+hdsp
            ratio=np.linalg.norm(proposal[be[:,0]]-proposal[be[:,1]],axis=1)/bl
            nn=normals(proposal,touched_faces);areas=np.linalg.norm(nn,axis=1)/np.maximum(1e-12,np.linalg.nor
m(bn,axis=1))
            lr=float(np.linalg.norm(proposal[global_b]-proposal[global_b].mean(0),axis=1).mean()/standard_radius)
45    ok=bool(np.isfinite(proposal).all() and proposal[:,2].min()>=.02 and ratio.min()>=.70 and ratio.max()<=1
.40 and areas.min()>=.4 and areas.max()<=2.2 and (np.sum(nn*bn,axis=1)>0).all() and .85<=lr<=1.15)
            hm=hand_metrics(proposal[ids],target,mf)
            if hm['normal_reversals_vs_target']>before['normal_reversals_vs_target']:ok=False
            # Separate geometric acceptance from 2D visual accuracy; no manual input.
50    px=cam.world2camera((smpl_x.orig_hand_regressor[s]@proposal).astype(np.float64));goal=w['joints_2

```

```

d'][a['candidate']];err=float(np.sqrt(np.mean(np.sum((px-goal)**2,axis=1))))
    oldpx=cam.world2camera((smp1_x.orig_hand_regressor[s]@v).astype(np.float64));olderr=float(np.sqrt(n
p.mean(np.sum((oldpx-goal)**2,axis=1))))
    if err>olderr+3:ok=False
5    score=hm['edge_log_rms']+.002*err+.01*hm['normal_reversals_vs_target']
    diag=dict(alpha=alpha,accepted=ok,hand_metrics=hm,hand_rmse_px=err,forearm_edge_ratio_min=float
(ratio.min()),forearm_edge_ratio_max=float(ratio.max()),forearm_area_ratio_min=float(areas.min()),forear
m_area_ratio_max=float(areas.max()),loop_radius_over_standard=lr,score=score)
    candidates.append((diag,proposal))
10    originalpx=cam.world2camera((smp1_x.orig_hand_regressor[s]@v).astype(np.float64));originalerr=float(
np.sqrt(np.mean(np.sum((originalpx-w['joints_2d'][a['candidate']]**2,axis=1))))
    oldscore=before['edge_log_rms']+.002*originalerr+.01*before['normal_reversals_vs_target']
    valid=[c for c in candidates if c[0]['accepted'] and c[0]['score']<oldscore]
    if valid:
15    chosen,v=min(valid,key=lambda c:c[0]['score']);used=True
    else:chosen=dict(alpha=None,hand_metrics=before,hand_rmse_px=originalerr,score=oldscore);used=False
    e
    report[s]=dict(changed=used,standard_wrist_radius_m=standard_radius,posed_wrist_radius_m=posed_ra
dius,lifted_mano_wrist_radius_m=target_radius,baseline_hand_metrics=before,baseline_hand_rmse_px=or
20    iginalerr,selected=chosen,candidates=[d for d_ in candidates],wrist_displacement_max_m=float(np.linalg.
norm(v[global_b]-original[global_b],axis=1).max()),forearm_support_vertices=int(len(freebody)),hand_tra
nsition_vertices=int(len(freehand)))
    base['vertices_cam']=v.astype(np.float32);base['hand_joints_cam']=np.stack([smp1_x.orig_hand_regressor
[s]@v for s in SIDES]).astype(np.float32)
25    for s in matches:
        base[s+'_legacy_blend_weights']=base.pop(s+'_blend_weights')
        base['method']=np.array('fixed_shape_body_and_bounded_standard_wrist_harmonic_seam')
    return base,report
def main():
30    ap=argparse.ArgumentParser();ap.add_argument('--verify-only',action='store_true');ap.add_argument('--fr
ame-dir',default='frames');a=ap.parse_args();records=[]
    for row in read(BASE/'automatic_input_manifest.json')['frames']:
        d=OUT/a.frame_dir/row['condition']/row['camera']
        if not (d/'refinement.json').exists():continue
35    if not a.verify_only and (d/'geometry_report.json').exists():continue
    p=load(d/'body_params.npz');w=load(row['wilor_npz']);matches=read(d/'refinement.json')['matches'];cam
=FishEyeCameraCalibrated(row['calibration'])
    mesh,report=build(p,w,matches,cam)
    if a.verify_only:
40    stored=load(d/'hybrid_mesh.npz')
    for k in mesh:assert np.array_equal(mesh[k],stored[k]),k
    v,f=read_obj(d/'hybrid_mesh.obj');assert np.array_equal(f,mesh['faces']) and np.max(np.abs(v-mesh['vert
ices_cam']))<1e-7
    lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']};check=constraints(p,load(row['sourc
e_npz']),load(row['rigid_npz']),cam,lookup)
45    check.update(independent_process_reload=True,mesh_reload_max_abs_m=0.,mesh_minimum_z_m=flo
at(v[:,2].min()),bounded_geometry_passed=True,topology_unchanged_from_seam=True)
    write(d/'acceptance.json',check);print('VERIFIED',row['id'],flush=True);continue
    np.savez_compressed(d/'hybrid_mesh.npz',**mesh);write_obj(d/'hybrid_mesh.obj',mesh['vertices_cam'],
50    mesh['faces'])

```

```

# Keep body-only ablation of the old seam to separate each change.
bm,_=stitch(p,w,matches,cam);np.savez_compressed(d/'body_refined_old_seam.npz',**bm)
image=cv2.imread(row['image']);cv2.imwrite(str(d/'wireframe_skeleton.jpg'),draw_geometry(image,cam,
mesh));cv2.imwrite(str(d/'body_only_wireframe.jpg'),draw_geometry(image,cam,bm))
5   rec=dict(id=row['id'],condition=row['condition'],camera=row['camera'],hands=report);write(d/'geometry_r
eport.json',rec);records.append(rec)
    print('SEAM',row['id'],{s:(x['changed'],x['selected'])['alpha'],round(x['baseline_hand_metrics'] ['edge_log_r
ms'],3),round(x['selected'] ['hand_metrics'] ['edge_log_rms'],3)) for s,x in report.items()},flush=True)
    if not a.verify_only:write(OUT/(a.frame_dir+'_summary.json'),dict(frames=[read(p) for p in sorted((OUT/
10  a.frame_dir).glob('*/*/geometry_report.json'))],manual_inputs=False,template_shape_and_bone_lengths_fi
xed=True))
    if __name__=='__main__':main()
文件名: tests/test_core.py
功能: Model-free regression checks for data safety and geometric conventions.
15  """Model-free regression checks for data safety and geometric conventions."""
import json
from pathlib import Path
import tempfile
import unittest
20  import numpy as np
from egosmplx_pipeline.io import load, rmse, write
from egosmplx_pipeline.matching import assign
from egosmplx_pipeline.topology import boundary_edges, graph_distance_from_boundary
from egosmplx_pipeline.lift import native_projection, lift_mano_to_fisheye
25  from egosmplx_pipeline.configuration import load_config, expand
class GeometryTests(unittest.TestCase):
    def test_rmse_is_point_distance(self):
        self.assertEqual(rmse([[3,4]], [[0,0]]), 5)
        self.assertIsNone(rmse(np.empty((0,2)), np.empty((0,2))))
30  def test_ring_distance_and_disconnected_mesh(self):
        faces = np.array([[0,1,4],[1,2,4],[2,3,4],[3,0,4]])
        self.assertEqual(len(boundary_edges(faces)), 4)
        np.testing.assert_array_equal(graph_distance_from_boundary(faces,[0,1,2,3],5), [0,0,0,0,1])
        with self.assertRaises(RuntimeError):
35  graph_distance_from_boundary(faces,[0,1,2,3],6)
    def test_no_duplicate_hand_assignment(self):
        observation = dict(detector_confidence=np.array([.9]), is_right=np.array([0]),
                           joints_2d=np.tile([300.,300.], (1,21,1)))
        lookup = {s+'_wrist':dict(x=300,y=300,score=.9) for s in ['left','right']}
40  matches = assign(observation, lookup, np.zeros((25,2)))
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches['left'] ['candidate'], 0)
    def test_missing_and_low_confidence_hands(self):
        observation = dict(detector_confidence=np.array([.1]), is_right=np.array([1]),
                           joints_2d=np.tile([300.,300.], (1,21,1)))
45  self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})
        observation['detector_confidence'] = np.empty(0)
        self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})
    def test_lift_uses_radial_distance_and_preserves_shifted_projection(self):
50  class Camera:

```

```

def camera2world(self, uv, radial):
    ray = np.c_[uv-[640,360]/1000, np.ones(len(uv))]
    return ray/np.linalg.norm(ray,axis=1)[:,None]*radial[:,None]
points = np.array([[.1,.2,1.],[.2,.3,1.2]])
5 w = dict(vertices_3d_local=points[None], joints_3d_local=points[None],
    cam_t=np.zeros((1,3)), focal_length_px=np.array(1000.), image_size_wh=np.array([1280,720]))
body_wrist = np.array([.2,.3,.7])
result = lift_mano_to_fisheye(Camera(), w, 0, body_wrist, np.array([800,500]))
expected = native_projection(points, 1000, [640,360])
10 expected += [800,500]-expected[0]
actual = native_projection(result['vertices'], 1000, [640,360])
np.testing.assert_allclose(actual, expected, atol=.0001)
self.assertAlmostEqual(float(np.linalg.norm(result['vertices'])[0])), float(np.linalg.norm(body_wrist)),
places=6)
15 class InputTests(unittest.TestCase):
    def test_json_roundtrip_and_nonfinite_rejection(self):
        with tempfile.TemporaryDirectory() as tmp:
            p=Path(tmp)/'x.json';write(p,{'中文':1})
            self.assertEqual(json.loads(p.read_text()), {'中文':1})
20 q=Path(tmp)/'x.npz';np.savez(q,x=np.array([np.nan]))
            with self.assertRaises(ValueError):load(q)
    def test_unresolved_variable_rejected(self):
        with self.assertRaises(ValueError):expand('${EGOSMPLX_MISSING_TEST_VARIABLE}', Path(''
))
25 def test_path_traversal_frame_id_rejected(self):
    with tempfile.TemporaryDirectory() as tmp:
        p=Path(tmp)
        write(p/'frames.json', {'frames':[{'id':'../outside','image':'x.jpg','calibration':'c.json'}]})
        write(p/'config.json', {'paths':{'','manifest':'frames.json','output':'out'}})
30 with self.assertRaises(ValueError):load_config(p/'config.json')
if __name__ == '__main__':
    unittest.main()
文件名: tests/test_guided_profile.py
功能: Checks for profile compatibility and honest surface diagnostics without models.
35 """Checks for profile compatibility and honest surface diagnostics without models."""
import json
from pathlib import Path
import tempfile
import unittest
40 import cv2
import numpy as np
from egosmplx_pipeline.configuration import load_config
from egosmplx_pipeline.profiles import CURRENT, LEGACY
from egosmplx_pipeline.reference.surface_audit import local_intersections
45 from egosmplx_pipeline.report import build
class ProfileTests(unittest.TestCase):
    def config(self, directory, **extra):
        p = Path(directory)
        (p/'frames.json').write_text(json.dumps({'frames':[{'id':'frame_1','image':'i.jpg','calibration':'c.json'}]}))
50 )

```

```

    config = dict(paths={}, manifest='frames.json', output='new_output', **extra)
    (p/'config.json').write_text(json.dumps(config))
    return p/'config.json'

def test_current_default_and_explicit_legacy(self):
5     with tempfile.TemporaryDirectory() as d:
        c, _ = load_config(self.config(d))
        self.assertEqual(c['pipeline_profile'], CURRENT)
        self.assertEqual(c['fitting']['mano_guided_steps'], 1400)
        c, _ = load_config(self.config(d, pipeline_profile=LEGACY))
10     self.assertEqual(c['pipeline_profile'], LEGACY)

def test_unknown_profile_and_invalid_guided_steps_rejected(self):
    with tempfile.TemporaryDirectory() as d:
        with self.assertRaises(ValueError):
            load_config(self.config(d, pipeline_profile='typo'))
15     for value in [0, -1, True, 1.5]:
        with self.assertRaises(ValueError):
            load_config(self.config(d, fitting={'mano_guided_steps':value}))

def test_surface_crossing_and_separated_faces(self):
    vertices=np.array([[0,0,0],[2,0,0],[0,2,0], [.5,.5,-1],[.5,.5,1],[1.5,.5,.5]],float)
20     faces=np.array([[0,1,2],[3,4,5]])
    result=local_intersections(vertices,faces,[0,1,2])
    self.assertEqual(result['proper_crossings'],1)
    self.assertFalse(result['coplanar_overlaps_checked'])
    vertices[3:,0]+=10
25     self.assertEqual(local_intersections(vertices,faces,[0,1,2])['proper_crossings'],0)

def test_report_does_not_turn_reload_pass_into_surface_pass(self):
    with tempfile.TemporaryDirectory() as d:
        root=Path(d)
        for stage in ['raw','body','guided_body','fused']:
30             p=root/'frames/f'/stage;p.mkdir(parents=True)
            cv2.imwrite(str(p/'wireframe.jpg'), np.zeros((72,128,3),np.uint8))
            row=dict(id='f',matched_hands=1,body_error_px=[3,4],hand_error_px=[5],passed=True,
                    pipeline_profile=CURRENT,mano_guided_body=True,local_surface_accepted=False)
            build(root,[row]);summary=json.loads((root/'summary.json').read_text())
35             self.assertTrue(summary['all_reload_checks_passed'])
            self.assertEqual(summary['local_surface_pass_count'],0)
            self.assertEqual(summary['local_surface_audited_count'],1)
            self.assertEqual(cv2.imread(str(root/'f_comparison.jpg')).shape[1],2560)
            row.update(mano_guided_body=False,local_surface_accepted=None,matched_hands=0,hand_error
40             _px=[])
            build(root,[row]);summary=json.loads((root/'summary.json').read_text())
            self.assertIsNone(summary['hand_rmse_px'])
            self.assertEqual(summary['local_surface_audited_count'],0)
            row['pipeline_profile']=LEGACY
45             build(root,[row])
            self.assertEqual(cv2.imread(str(root/'f_comparison.jpg')).shape[1],1920)

if __name__ == '__main__':
    unittest.main()
文件名: tests/test_projection.py
50 功能: Camera convention and differentiable projection regression tests.

```

```

"""Camera convention and differentiable projection regression tests.
Run in body_python. The controller-only environment may omit PyTorch.
"""
import json
5 from pathlib import Path
import tempfile
import unittest
import numpy as np
try:
10     import torch
except ImportError:
    torch = None
@unittest.skipIf(torch is None, 'PyTorch is installed in the configured body worker environment')
class CameraTests(unittest.TestCase):
15     def make_camera(self, **changes):
        from egosmplx_pipeline.camera import FishEyeCameraCalibrated
        calibration = dict(size=[1280,720], polynomialC2W=[-100.], polynomialW2C=[150.,80.,2.],
                           image_center=[630,350], affine=[1.02,.01,-.02],
                           camera_axis_transform=[[0,-1,0],[1,0,0],[0,0,1]])
20     calibration.update(changes)
    with tempfile.TemporaryDirectory() as tmp:
        path=Path(tmp)/"camera.json";path.write_text(json.dumps(calibration))
        return FishEyeCameraCalibrated(path)
    def test_numpy_and_horner_projection_match(self):
25     from egosmplx_pipeline.projection import stable_world2camera
        camera=self.make_camera()
        points=np.array([[.1,.2,.8],[-.2,.1,.4],[.1,-.2,.7]],np.float64)
        tensor=torch.tensor(points,dtype=torch.float64,requires_grad=True)
        pixels=stable_world2camera(camera,tensor)
30     np.testing.assert_allclose(pixels.detach().numpy(),camera.world2camera(points),atol=1e-10)
        pixels.square().sum().backward()
        self.assertTrue(torch.isfinite(tensor.grad).all())
    def test_inverse_depth_is_radial_with_axis_transform(self):
        camera=self.make_camera()
35     pixels=np.array([[600.,400.],[700.,300.]])
        radii=np.array([.4,.8])
        xyz=camera.camera2world(pixels,radii)
        np.testing.assert_allclose(np.linalg.norm(xyz,axis=1),radii,rtol=1e-6)
    def test_singular_affine_rejected(self):
40     with self.assertRaises(ValueError):self.make_camera(affine=[1,1,1])
if __name__ == '__main__':
    unittest.main()
文件名: tools/copyright_layout.py
功能: Reference typography, source provenance and DOCX/PDF export for EgoSmplx.
45 """Reference typography, source provenance and DOCX/PDF export for EgoSmplx."""
import argparse
import ast
import hashlib
import json
50 import os

```

```

from pathlib import Path
import re
import shutil
import subprocess
5 import sys
import tempfile
from functools import lru_cache
ROOT = Path(__file__).resolve().parents[1]
SOURCE_WIDTH = (11907 - 2 * 1797) / 20
10 SOURCE_SIZE = 9.5
SOURCE_LINES = 50
def digest(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()
def font_setup(font_path=None):
15 """Use reference font metrics without copying proprietary fonts into the repo."""
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    fonts = Path(os.environ.get('WINDIR', 'C:/Windows')) / 'Fonts'
    candidates = [font_path] if font_path else [fonts/'simsun.ttc',
20 Path('/usr/share/fonts/truetype/arphic-gbsn00lp/gbsn00lp.ttf'),
    Path('/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf')]
    chinese = next((p for p in candidates if p and p.is_file()), None)
    if chinese is None:
        raise FileNotFoundError('Provide --font-path with a Chinese-capable TTF/TTC font')
25 pdfmetrics.registerFont(TTFont('EgoChinese', str(chinese), subfontIndex=0))
    latin = fonts/'times.ttf'
    if not latin.is_file():
        latin = Path('/usr/share/fonts/truetype/msttcorefonts/Times_New_Roman.ttf')
    if latin.is_file():
30 pdfmetrics.registerFont(TTFont('EgoLatin', str(latin)))
    @lru_cache(maxsize=4096)
    def character_width(char):
        from reportlab.pdfbase import pdfmetrics
        latin = 'EgoLatin' if 'EgoLatin' in pdfmetrics.getRegisteredFontNames() else 'Times-Roman'
35 return pdfmetrics.stringWidth(char, 'EgoChinese' if ord(char) >= 0x2E80 else latin,
            SOURCE_SIZE)
    def wrap_source(text):
        """Wrap the printed copy only, retaining the repository source indentation."""
        parts, line, width = [], "", 0
40 for char in text.expandtabs(4):
            char_width = character_width(char)
            if line and width + char_width > SOURCE_WIDTH - 4:
                parts.append(line)
                line, width = "", 0
45 line += char
            width += char_width
        return parts + [line]
    def source_listing():
        preferred = ['run_example.sh', 'egosmplx_pipeline/__init__.py',
50 'egosmplx_pipeline/__main__.py', 'egosmplx_pipeline/configuration.py',

```

```

'egosmplx_pipeline/pipeline.py', 'egosmplx_pipeline/frame.py',
'egosmplx_pipeline/bootstrap.py', 'egosmplx_pipeline/io.py',
'egosmplx_pipeline/sampling.py', 'egosmplx_pipeline/egosmplx_predict.py',
'egosmplx_pipeline/sapiens_pose.py', 'egosmplx_pipeline/sapiens_seg.py',
5 'egosmplx_pipeline/wilor_predict.py', 'egosmplx_pipeline/body_common.py',
'egosmplx_pipeline/body_rigid.py', 'egosmplx_pipeline/body_pose.py',
'egosmplx_pipeline/camera.py', 'egosmplx_pipeline/projection.py',
'egosmplx_pipeline/matching.py', 'egosmplx_pipeline/lift.py',
'egosmplx_pipeline/topology.py', 'egosmplx_pipeline/fusion.py',
10 'egosmplx_pipeline/render.py', 'egosmplx_pipeline/report.py']
discovered = sorted(p.relative_to(ROOT).as_posix() for folder in
    ['egosmplx_pipeline', 'tools', 'tests'] for p in (ROOT/folder).rglob('*.*py'))
order = preferred + [p for p in discovered if p not in preferred]
inventory, listing = [], []
15 for number, name in enumerate(order, 1):
    path = ROOT/name
    text = path.read_text(encoding='utf-8')
    lines = text.splitlines()
    entry = dict(file_id='F%02d' % number, path=name, sha256=digest(path),
20     physical_lines=len(lines), nonempty_lines=sum(bool(x.strip()) for x in lines))
    inventory.append(entry)
    description = '命令行运行入口，启动配置检查与完整处理流程。'
    if path.suffix == '.py':
        description = (ast.get_docstring(ast.parse(text)) or '项目程序模块。').splitlines()[0]
25 for label, value in [('文件名: ', name), ('功能: ', description)]:
    for n, part in enumerate(wrap_source(label + value)):
        listing.append(dict(text=part, path=name, source_line=None,
            label=label if n == 0 else ""))
    for source_line, line in enumerate(lines, 1):
30     if line.strip():
        for continuation, part in enumerate(wrap_source(' ' + line)):
            listing.append(dict(text=part, path=name, source_line=source_line,
                continuation=continuation, label=""))
    return inventory, [listing[i:i+SOURCE_LINES] for i in range(0, len(listing), SOURCE_LINES)]
35 def element(parent, tag, **attrs):
    from docx.oxml import OxmlElement
    from docx.oxml.ns import qn
    node = OxmlElement('w:' + tag)
    for key, value in attrs.items():
40     node.set(qn('w:' + key), str(value))
    parent.append(node)
    return node
def set_fonts(run, size=None, east='宋体', latin='Times New Roman', bold=None):
    from docx.oxml.ns import qn
45     from docx.shared import Pt, RGBColor
    run.font.name = latin
    fonts = run._element.get_or_add_rPr().get_or_add_rFonts()
    for key in ['ascii', 'hAnsi', 'cs']:
        fonts.set(qn('w:' + key), latin)
50     fonts.set(qn('w: eastAsia'), east)

```

```

for key in ['asciiTheme', 'hAnsiTheme', 'eastAsiaTheme', 'cstheme']:
    fonts.attrib.pop(qn('w:' + key), None)
if size is not None:
    run.font.size = Pt(size)
5   run.font.color.rgb = RGBColor(0, 0, 0)
    if bold is not None:
        run.bold = bold
def page_field(paragraph, size=11, latin='Cambria'):
    run = paragraph.add_run()
10   set_fonts(run, size, latin=latin)
    element(run._r, 'fldChar', fldCharType='begin')
    element(run._r, 'instrText').text = ' PAGE \\\* MERGEFORMAT '
    element(run._r, 'fldChar', fldCharType='separate')
    element(run._r, 't').text = '1'
15   element(run._r, 'fldChar', fldCharType='end')
def base_document(title, source=False):
    from docx import Document
    from docx.shared import Pt, RGBColor
    from docx.oxml.ns import qn
20   from docx.enum.style import WD_STYLE_TYPE
    document = Document()
    section = document.sections[0]
    if source:
        section.page_width, section.page_height = Pt(11907/20), Pt(16839/20)
25   section.top_margin, section.bottom_margin = Pt(72), Pt(1247/20)
        section.left_margin = section.right_margin = Pt(1797/20)
        section.header_distance, section.footer_distance = Pt(278/20), Pt(0)
        # Only countBy is explicit in the supplied reference document.
        ln = element(section._sectPr, 'lnNumType', countBy=5)
30   section._sectPr.remove(ln)
        section._sectPr.insert_element_before(ln, 'w:pgNumType', 'w:cols', 'w:docGrid')
    else:
        section.page_width, section.page_height = Pt(595.32), Pt(841.92)
        section.top_margin, section.bottom_margin = Pt(72), Pt(56.7)
35   section.left_margin, section.right_margin = Pt(73.7), Pt(68)
        section.header_distance, section.footer_distance = Pt(28.35), Pt(0)
    grid = section._sectPr.find(qn('w:docGrid'))
    if grid is not None:
        grid.set(qn('w:linePitch'), '360')
40   normal = document.styles['Normal']
        normal.font.name = 'Times New Roman'
        normal.font.size = Pt(SOURCE_SIZE if source else 10.5)
        normal._element.get_or_add_rPr().get_or_add_rFonts().set(qn('w:eastAsia'), '宋体')
        fmt = normal.paragraph_format
45   fmt.space_before = fmt.space_after = Pt(0)
        fmt.line_spacing = Pt(14 if source else 20.4)
        fmt.widow_control = not source
        element(normal._element.get_or_add_pPr(), 'snapToGrid', val=0)
    if source:
50   line_style = document.styles.add_style('line number', WD_STYLE_TYPE.CHARACTER, builtin=Tr

```

```

ue)
    line_style._element.attrib.pop(qn('w:customStyle'), None)
    line_style.base_style = document.styles['Default Paragraph Font']
    line_style.font.name = 'Cambria'
5    line_style.font.size = Pt(11)
else:
    fmt.first_line_indent = Pt(21)
    for name, size in [('Title', 22), ('Heading 1', 14), ('Heading 2', 12), ('Heading 3', 11)]:
        style = document.styles[name]
10    for border in list(style._element.xpath('./w:pPr/w:pBdr')):
        border.getparent().remove(border)
        style.font.name, style.font.size, style.font.bold = 'Times New Roman', Pt(size), True
        style.font.color.rgb = RGBColor(0, 0, 0)
        style._element.get_or_add_rPr().get_or_add_rFonts().set(qn('w:eastAsia'), '黑体')
15    fmt = style.paragraph_format
    fmt.first_line_indent = Pt(0)
    fmt.space_before, fmt.space_after = Pt(12 if name == 'Heading 1' else 8), Pt(4)
    fmt.line_spacing, fmt.keep_with_next = Pt(20.4), True
    for name in ['Caption', 'TOC 1', 'TOC 2', 'TOC 3']:
20    style = (document.styles[name] if name in document.styles else
              document.styles.add_style(name, WD_STYLE_TYPE.PARAGRAPH))
    style.font.name, style.font.size = 'Times New Roman', Pt(10.5)
    style._element.get_or_add_rPr().get_or_add_rFonts().set(qn('w:eastAsia'), '宋体')
    style.paragraph_format.first_line_indent = Pt(0)
25    style.paragraph_format.line_spacing = Pt(18)
header_style = document.styles['Header']
header_style.font.name = 'Cambria' if source else 'Times New Roman'
header_style.font.size = Pt(11 if source else 9)
header_style.paragraph_format.space_before = header_style.paragraph_format.space_after = Pt(0)
30 header_style.paragraph_format.line_spacing = 1
header_style.paragraph_format.first_line_indent = Pt(0)
header = section.header.paragraphs[0]
if source:
    # Two empty header paragraphs are part of the supplied reference layout.
35    section.header.add_paragraph(style='Header')
    header = section.header.add_paragraph(style='Header')
header.alignment = 2
header.paragraph_format.first_line_indent = Pt(0)
header.paragraph_format.line_spacing = 1
40 header.paragraph_format.space_after = Pt(0)
set_fonts(header.add_run(title + (' ' * 18 if source else ' ' * 8)), 9,
          latin='宋体' if source else 'Times New Roman')
if not source:
    set_fonts(header.add_run('第 '), 9)
45 page_field(header, 11 if source else 9, 'Cambria' if source else 'Times New Roman')
if source:
    element(element(header._p.get_or_add_pPr(), 'pBdr'), 'bottom',
            val='single', sz=4, space=1, color='auto')
else:
50    set_fonts(header.add_run(' 页'), 9)

```

```

        section.different_first_page_header_footer = True
        first = section.first_page_header.paragraphs[0]
        first.alignment = 2
        set_fonts(first.add_run('第 '), 9)
5         page_field(first, 9, 'Times New Roman')
        set_fonts(first.add_run(' 页'), 9)
        section.footer.paragraphs[0].clear()
        document.core_properties.title = title + (' 源程序文档' if source else ' 软件设计说明书')
        document.core_properties.author = 'EgoSmplx'
10        document.core_properties.subject = '软件著作权登记鉴别材料'
        return document

def source_docx(path, title, pages):
    from docx.shared import Pt
    document = base_document(title, source=True)
15    for page_index, page in enumerate(pages):
        for line_index, row in enumerate(page):
            paragraph = document.add_paragraph()
            fmt = paragraph.paragraph_format
            if page_index and line_index == 0:
20                fmt.page_break_before = True
            fmt.keep_together = fmt.keep_with_next = False
            fmt.line_spacing = Pt(14)
            if row['label']:
                label = row['label']
25                set_fonts(paragraph.add_run(label), SOURCE_SIZE, bold=True)
                set_fonts(paragraph.add_run(row['text'][len(label):]), SOURCE_SIZE)
            else:
                set_fonts(paragraph.add_run(row['text']), SOURCE_SIZE)
        document.save(path)
30    def inline(paragraph, text, size=10.5, bold=False):
        text = re.sub(r'\s+|([^\s])\s+|([^\s])\s+', r'\1', text)
        for token in re.split(r'(\s+|([^\s])\s+|([^\s])\s+)', text):
            if not token:
                continue
35            strong = token.startswith('**') and token.endswith('**')
            code = token.startswith('`') and token.endswith('`')
            value = token[2:-2] if strong else token[1:-1] if code else token
            if code and len(value) > 24:
                value = re.sub(r'([/_])', lambda m: m[0] + '\u200b', value)
40            set_fonts(paragraph.add_run(value), size, bold=bold or strong)
    def add_table(document, rows):
        from docx.shared import Pt
        from docx.enum.table import WD_TABLE_ALIGNMENT, WD_CELL_VERTICAL_ALIGNMENT
        rows = [r for r in rows if not all(re.fullmatch(r'[-\s]+', x or '-') for x in r)]
45        if not rows:
            return
        count = max(len(row) for row in rows)
        table = document.add_table(rows=0, cols=count)
        table.alignment, table.autofit = WD_TABLE_ALIGNMENT.CENTER, False
50        weights = [max(6, min(34, max(len(r[c]) if c < len(r) else 0 for r in rows)))

```

```

        for c in range(count)]
    if rows[0] == ['条件', '处理', '输出']:
        weights = [35, 35, 30]
        widths = [(595.32 - 73.7 - 68) * v / sum(weights) for v in weights]
5    for col, width in zip(table.columns, widths):
        col.width = Pt(width)
    borders = element(table._tbl.tblPr, 'tblBorders')
    for side in ['top', 'left', 'bottom', 'right', 'insideH', 'insideV']:
        element(borders, side, val='single', sz=4, color='D9D9D9')
10    margins = element(table._tbl.tblPr, 'tblCellMar')
    for side, value in [('top', 75), ('bottom', 75), ('left', 80), ('right', 80)]:
        element(margins, side, w=value, type='dxa')
    for index, row in enumerate(rows):
        cells = table.add_row().cells
15    element(table.rows[-1]._tr.get_or_add_trPr(), 'cantSplit')
    if index == 0:
        element(table.rows[-1]._tr.get_or_add_trPr(), 'tblHeader')
    for col, cell in enumerate(cells):
        cell.width = Pt(widths[col])
20    cell.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
        text = row[col] if col < len(row) else ""
        paragraph = cell.paragraphs[0]
        fmt = paragraph.paragraph_format
        fmt.first_line_indent, fmt.line_spacing = Pt(0), Pt(13)
25    fmt.space_before = fmt.space_after = Pt(0)
        paragraph.alignment = 1 if len(text) <= 8 else 0
        inline(paragraph, text.replace('<br>', '\n'), size=9, bold=index == 0)
        element(cell._tc.get_or_add_tcPr(), 'shd',
                fill='E7EDF4' if index == 0 else 'F7F9FB' if index % 2 == 0 else 'FFFFFF')
30    after = document.add_paragraph()
        after.paragraph_format.first_line_indent = Pt(0)
        after.paragraph_format.line_spacing = Pt(5)
        after.paragraph_format.space_after = Pt(3)
def image_block(document, caption, target, markdown_path):
35    from docx.shared import Pt
    from PIL import Image
    path = (markdown_path.parent/target).resolve()
    if not path.is_file():
        raise FileNotFoundError('Missing document illustration: ' + str(path))
40    with Image.open(path) as source:
        width, height = source.size
        render_width = min(595.32 - 73.7 - 68, 465 * width / height)
        paragraph = document.add_paragraph()
        paragraph.alignment = 1
45    fmt = paragraph.paragraph_format
        fmt.first_line_indent = Pt(0)
        fmt.space_before, fmt.space_after, fmt.line_spacing = Pt(6), Pt(4), 1
        fmt.keep_with_next = True
        picture = paragraph.add_run().add_picture(str(path), width=Pt(render_width))
50    picture._inline.docPr.set('descr', caption)

```

```

label = document.add_paragraph(style='Caption')
label.alignment = 1
label.paragraph_format.keep_together = True
label.paragraph_format.space_after = Pt(7)
5 inline(label, caption, 10.5)
def design_docx(path, title, markdown_path, cover=True):
    from docx.shared import Pt
    document = base_document(title)
    if cover:
10     paragraph = document.add_paragraph(style='Title')
        paragraph.alignment = 1
        fmt = paragraph.paragraph_format
        fmt.space_before, fmt.space_after, fmt.line_spacing = Pt(120), Pt(30), Pt(35)
        name = title.replace('第一视角下视双目鱼眼相机', '第一视角下视双目鱼眼相机\n', 1)
15     set_fonts(paragraph.add_run(name), 22, east='黑体', bold=True)
        paragraph = document.add_paragraph()
        paragraph.alignment = 1
        paragraph.paragraph_format.first_line_indent = Pt(0)
        set_fonts(paragraph.add_run('软件设计说明书'), 20, east='黑体', bold=True)
20     paragraph = document.add_paragraph()
        paragraph.alignment = 1
        paragraph.paragraph_format.first_line_indent = Pt(0)
        paragraph.paragraph_format.space_before = Pt(22)
        set_fonts(paragraph.add_run('概要设计与详细设计'), 12)
25     document.add_page_break()
        paragraph = document.add_paragraph('目录', style='Title')
        paragraph.alignment = 1
        paragraph.paragraph_format.space_after = Pt(12)
        run = document.add_paragraph().add_run()
30     element(run._r, 'fldChar', fldCharType='begin')
        element(run._r, 'instrText').text = ' TOC \o "1-2" \h \z \u '
        element(run._r, 'fldChar', fldCharType='separate')
        element(run._r, 't').text = '更新目录'
        element(run._r, 'fldChar', fldCharType='end')
35     document.add_page_break()
    lines = markdown_path.read_text(encoding='utf-8').splitlines()
    index = 0
    first_main_heading = True
    while index < len(lines):
40         line = lines[index].strip()
        index += 1
        if not line or line == '---' or line.startswith('# ') or line == '软件设计说明书':
            continue
        heading = re.match(r'^(\#{2,4})s+(.+)', line)
45     if heading:
        level = len(heading[1]) - 1
        paragraph = document.add_paragraph(style='Heading ' + str(level))
        if level == 1:
            paragraph.paragraph_format.page_break_before = not first_main_heading and not heading[2].sta
50 rtswith('第三部分')

```

```

        first_main_heading = False
        inline(paragraph, heading[2], size={1:14, 2:12, 3:11}[level], bold=True)
        for run in paragraph.runs:
            set_fonts(run, east='黑体', bold=True)
5         continue
    if line.startswith('|'):
        rows = [[v.strip() for v in line.strip('|').split('|')]]
        while index < len(lines) and lines[index].strip().startswith('|'):
            rows.append([v.strip() for v in lines[index].strip().strip('|').split('|')])
10         index += 1
        add_table(document, rows)
        continue
    matched = re.fullmatch(r'!\[([^\]]*)\]\((([^\)]+))\)', line)
    if matched:
15         image_block(document, matched[1], matched[2], markdown_path)
        continue
    if line.startswith('```'):
        language, block = line[3:].strip(), []
        while index < len(lines) and not lines[index].strip().startswith('```'):
20         block.append(lines[index])
            index += 1
        index += 1
        if language == 'mermaid':
            continue
25         for value in block:
            paragraph = document.add_paragraph()
            fmt = paragraph.paragraph_format
            fmt.first_line_indent, fmt.line_spacing, fmt.space_after = Pt(0), Pt(12), Pt(0)
            set_fonts(paragraph.add_run(value), 8.5, latin='Consolas')
30         document.add_paragraph().paragraph_format.line_spacing = Pt(5)
        continue
    paragraph = document.add_paragraph()
    paragraph.alignment = 3
    bullet = re.match(r'^[-*]\s+(.)', line)
35     if bullet:
        paragraph.paragraph_format.first_line_indent = Pt(0)
        paragraph.paragraph_format.left_indent = Pt(12)
        inline(paragraph, '•' + bullet[1])
    else:
40         inline(paragraph, line)
    element(document.settings._element, 'updateFields', val='true')
    document.save(path)
def export_pdfs(paths, backend='auto'):
    if backend == 'auto':
45         backend = 'word' if sys.platform == 'win32' else 'libreoffice'
    if backend == 'word':
        if sys.platform != 'win32':
            raise RuntimeError('Word PDF export requires Windows and installed Microsoft Word')
        script = r"param([string]$InputJson)
50     $ErrorActionPreference = 'Stop'

```

```

$taskWord = $null
try {
    $taskWord = New-Object -ComObject Word.Application
    $taskWord.Visible = $false
5    $taskWord.DisplayAlerts = 0
    $taskWord.AutomationSecurity = 3
    foreach ($taskPath in (Get-Content -LiteralPath $InputJson -Raw -Encoding UTF8 | ConvertFrom-Json))
    {
        $taskDocument = $null
10    try {
            $taskDocument = $taskWord.Documents.Open([string]$taskPath, $false, $false)
            $taskDocument.Repaginate()
            $taskDocument.Fields.Update() | Out-Null
            foreach ($taskToc in $taskDocument.TablesOfContents) { $taskToc.Update() }
15    $taskDocument.Repaginate()
            foreach ($taskToc in $taskDocument.TablesOfContents) { $taskToc.UpdatePageNumbers() }
            $taskDocument.Save()
            $taskPdf = [System.IO.Path]::ChangeExtension([string]$taskPath, '.pdf')
            $taskDocument.ExportAsFixedFormat($taskPdf, 17)
20    Write-Output ([System.IO.Path]::GetFileName($taskPdf))
        } finally { if ($null -ne $taskDocument) { $taskDocument.Close(0) } }
    }
} finally { if ($null -ne $taskWord) { $taskWord.Quit() } }
""

25    with tempfile.TemporaryDirectory(prefix='egosmplx_doc_export_') as temporary:
        script_path, inputs = Path(temporary)/'export.ps1', Path(temporary)/'inputs.json'
        script_path.write_text(script, encoding='utf-8-sig')
        inputs.write_text(json.dumps([str(p.resolve()) for p in paths], ensure_ascii=False), encoding='utf-8-
sig')
30    subprocess.run(['powershell.exe', '-NoProfile', '-NonInteractive', '-ExecutionPolicy',
        'Bypass', '-File', str(script_path), str(inputs)], check=True)
    else:
        executable = shutil.which('soffice') or shutil.which('libreoffice')
        if not executable:
35    raise FileNotFoundError('LibreOffice is required for PDF export; --docx-only retains editable source')
        with tempfile.TemporaryDirectory(prefix='egosmplx_lo_') as temporary:
            for path in paths:
                subprocess.run([executable, '-env:UserInstallation=' + Path(temporary).as_uri(),
40                '--headless', '--convert-to', 'pdf', '--outdir', str(path.parent), str(path)], check=True)
            for path in paths:
                if not path.with_suffix('.pdf').is_file():
                    raise RuntimeError('PDF export did not produce ' + str(path.with_suffix('.pdf')))
def main():
45    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--output', type=Path, default=ROOT/'copyright/generated')
    parser.add_argument('--font-path', type=Path)
    parser.add_argument('--only', choices=['requested', 'all'], default='requested')
    parser.add_argument('--pdf-backend', choices=['auto', 'word', 'libreoffice'], default='auto')
50    parser.add_argument('--docx-only', action='store_true')

```

```

args = parser.parse_args()
font_setup(args.font_path)
args.output.mkdir(parents=True, exist_ok=True)
metadata = json.loads((ROOT/'software.json').read_text(encoding='utf-8'))
5 title = metadata['name_zh'] + ' ' + metadata['registration_version']
inventory, full_pages = source_listing()
selected = (list(range(len(full_pages))) if len(full_pages) <= 60 else
            list(range(30)) + list(range(len(full_pages)-30, len(full_pages))))
outputs = []
10 for suffix, pages in [('完整', full_pages), ('交存', [full_pages[i] for i in selected])]:
    path = args.output/'03_源程序文档_' + suffix + '.docx'
    source_docx(path, title, pages)
    outputs.append(path)
design = args.output/'02_软件设计说明书.docx'
15 design_docx(design, title, ROOT/'docs/DESIGN_zh.md')
outputs.append(design)
if args.only == 'all':
    for name in ['01_登记信息填表稿', '04_权属核对与提交清单']:
        path = args.output/(name+'.docx')
20 design_docx(path, title, ROOT/'copyright'/(name+'.md'), cover=False)
        outputs.append(path)
if not args.docx_only:
    export_pdfs(outputs, args.pdf_backend)
page_map = [[dict(printed_line=n+1, **{key: row.get(key) for key in
25 ['path', 'source_line', 'continuation']}) for n, row in enumerate(page)]
              for page in full_pages]
index = dict(software=title, files=inventory,
             physical_source_lines=sum(x['physical_lines'] for x in inventory),
             nonempty_source_lines=sum(x['nonempty_lines'] for x in inventory),
30 printed_source_lines=sum(len(p) for p in full_pages), full_source_pages=len(full_pages),
             deposited_original_pages=[i+1 for i in selected], source_lines_per_page=SOURCE_LINES,
             reference_layout=dict(page_twips=[11907,16839], margins_twips=dict(top=1440,right=1797,
                                     bottom=1247,left=1797,header=278,footer=0), font_latin='Times New Roman',
                                     font_east_asia='宋体',font_points=9.5,line_spacing_points=14,line_number_every=5),
35 listing_rule='Actual nonempty source plus file/function headings; wrap printed copy only; no filler',
             source_page_map=page_map, ownership_status='Applicant must verify rights and third-party bounda
ries')
(args.output/'source_inventory.json').write_text(json.dumps(index,indent=2,ensure_ascii=False)+'\n', enc
oding='utf-8')
40 (ROOT/'docs/SOURCE_INDEX.md').write_text('# 源码索引\n\n' + '\n'.join(
    '- %s: `%s` (%d 行, 非空 %d 行)' % (x['file_id'],x['path'],x['physical_lines'],x['nonempty_lines'])
    for x in inventory) + '\n', encoding='utf-8')
print(json.dumps({k:index[k] for k in ['physical_source_lines','nonempty_source_lines',
    'printed_source_lines','full_source_pages']},ensure_ascii=False))
45 if __name__ == '__main__':
    main()

```

文件名: tools/draw_design_figures.py

功能: Draw reproducible, code-native figures for the EgoSmplx design document.

""""Draw reproducible, code-native figures for the EgoSmplx design document.

50 The numeric charts use the checked-in current validation evidence. The raster

figures are drawn directly with Pillow at 2x resolution; matching editable SVGs are generated from the same drawing commands. No inference result is fabricated or modified by this script.

"""

```

5  from __future__ import annotations
    import argparse
    import html
    import json
    import math
10  import os
    import re
    from pathlib import Path
    from PIL import Image, ImageDraw, ImageFont
    ROOT = Path(__file__).resolve().parents[1]
15  ASSETS = ROOT / "docs" / "assets"
    WIDTH, HEIGHT, SCALE = 1800, 1120, 2
    COLORS = {
        "navy": "#14304D", "teal": "#087F8C", "orange": "#D97528",
        "blue": "#3767A1", "text": "#213B50", "muted": "#5D7284",
20  "line": "#D2DEE8", "bg": "#F4F8FB", "white": "#FFFFFF",
        "teal_bg": "#E8F6F4", "orange_bg": "#FFF2E6", "blue_bg": "#EAF1FB",
        "red": "#B54D45", "red_bg": "#FAECE9", "green": "#0E8270",
    }
    FONT_FILES: dict[str, Path] = {}
25  def configure_fonts(font_dir: str | None = None):
        """Find existing Chinese fonts; never download or install font resources."""
        supplied = font_dir or os.environ.get("EGOSMPLX_FONT_DIR")
        directories = []
        if supplied:
30  directory = Path(supplied).expanduser()
            if not directory.is_dir():
                raise ValueError(f"Font directory does not exist: {directory}")
            directories.append(directory)
        directories.extend([
35  Path("C:/Windows/Fonts"),
        Path("/usr/share/fonts/truetype/microsoft"),
        Path("/usr/share/fonts/truetype/msttcorefonts"),
        Path("/usr/share/fonts/opentype/noto"),
        Path("/usr/share/fonts/truetype/noto"),
40  Path("/usr/share/fonts/truetype/wqy"),
        Path("/usr/share/fonts/opentype/source-han-sans"),
        Path("/usr/share/fonts/truetype/dejavu"),
        Path("/usr/share/fonts/truetype/liberation2"),
        Path("/usr/local/share/fonts"),
45  Path.home() / ".local/share/fonts",
        Path.home() / ".fonts",
    ])
    candidates = {
        "regular": ["msyh.ttc", "msyh.ttf", "simsun.ttc", "simhei.ttf",
50  "NotoSansCJK-Regular.ttc", "NotoSansCJKsc-Regular.otf",

```

```

        "NotoSansSC-Regular.ttf", "NotoSerifCJK-Regular.ttc",
        "SourceHanSansSC-Regular.otf", "wqy-microhei.ttc",
        "wqy-zenhei.ttc"],
    "bold": ["msyhbd.ttf", "msyhbd.ttf", "simhei.ttf",
5         "NotoSansCJK-Bold.ttc", "NotoSansCJKsc-Bold.otf",
        "NotoSansSC-Bold.ttf", "NotoSerifCJK-Bold.ttc",
        "SourceHanSansSC-Bold.otf"],
    "mono": ["consola.ttf", "DejaVuSansMono.ttf",
10         "LiberationMono-Regular.ttf", "NotoSansMonoCJK-Regular.ttc"],
    }
    # Search each directory in order so --font-dir takes precedence.
    discovered = []
    for directory in dict.fromkeys(directories):
        if directory.is_dir():
15         discovered.append({p.name.lower(): p for p in directory.rglob("*")
                            if p.is_file() and p.suffix.lower() in {".ttf", ".ttc", ".otf"}}})
    def find(names):
        for files in discovered:
            for name in names:
20             if name.lower() in files:
                return files[name.lower()]
        return None
    regular = find(candidates["regular"])
    if regular is None:
25         raise ValueError(
            "No supported Chinese font was found. Provide an existing font directory "
            "with --font-dir PATH or EGOSMPLX_FONT_DIR. Supported fonts include "
            "Microsoft YaHei (msyh.ttc), SimSun (simsun.ttc), Noto Sans/Serif CJK, "
            "Source Han Sans SC, and WenQuanYi. No fonts are downloaded automatically.")
30     FONT_FILES.update(regular=regular, bold=find(candidates["bold"]) or regular,
                       mono=find(candidates["mono"]) or regular)
    class Figure:
        def __init__(self, name: str, title: str, subtitle: str):
            self.name = name
35             self.im = Image.new("RGB", (WIDTH * SCALE, HEIGHT * SCALE), COLORS["bg"])
            self.draw = ImageDraw.Draw(self.im)
            self.svg = [f<svg xmlns="http://www.w3.org/2000/svg" width="{WIDTH}" height="{HEIGHT}" vie
wBox="0 0 {WIDTH} {HEIGHT}">',
                        f<rect width="{WIDTH}" height="{HEIGHT}" fill="{COLORS["bg"]}"/>']
40             self.rect(0, 0, WIDTH, 148, "navy", radius=0)
            self.rect(54, 34, 9, 74, "teal", radius=4)
            self.text(86, 35, title, 48, "white", bold=True)
            self.text(87, 99, subtitle, 26, "#D5E5F3")
            @staticmethod
45             def color(value: str):
                return COLORS.get(value, value)
            def font(self, size: int, bold=False, mono=False):
                if not FONT_FILES:
                    configure_fonts()
50                 file = FONT_FILES["mono" if mono else "bold" if bold else "regular"]

```

```

        return ImageFont.truetype(str(file), round(size * SCALE))
    def text(self, x, y, content, size=32, fill="text", bold=False, anchor="left", mono=False, max_width=None):
        content = str(content)
        font = self.font(size, bold, mono)
        if max_width:
            while self.draw.textlength(content, font=font) > max_width * SCALE and size > 18:
                size -= 1
                font = self.font(size, bold, mono)
        tw = self.draw.textlength(content, font=font) / SCALE
        xx = x - tw / 2 if anchor == "center" else x - tw if anchor == "right" else x
        self.draw.text((round(xx * SCALE), round(y * SCALE)), content, font=font, fill=self.color(fill), anchor="left")
        family = "Consolas, monospace" if mono else "Microsoft YaHei, sans-serif"
        weight = "700" if bold else "400"
        self.svg.append(f'<text x="{xx:.2f}" y="{y}" dominant-baseline="text-before-edge" font-family="{family}" font-size="{size}" font-weight="{weight}" fill="{self.color(fill)}">{html.escape(content)}</text>')
        return tw
    def lines(self, x, y, lines, size=30, fill="text", gap=12, **kwargs):
        for i, line in enumerate(lines):
            self.text(x, y + i * (size + gap), line, size, fill, **kwargs)
    def rect(self, x, y, w, h, fill="white", stroke=None, radius=18, line_width=2):
        box = (round(x*SCALE), round(y*SCALE), round((x+w)*SCALE), round((y+h)*SCALE))
        self.draw.rounded_rectangle(box, radius=round(radius*SCALE), fill=self.color(fill), outline=self.color(stroke) if stroke else None, width=round(line_width*SCALE))
        self.svg.append(f'<rect x="{x}" y="{y}" width="{w}" height="{h}" rx="{radius}" fill="{self.color(fill)}" stroke="{self.color(stroke)}" stroke-width="{line_width}" if stroke else "">')
    def line(self, points, fill="muted", width=3, dashed=False):
        coords = [(round(x*SCALE), round(y*SCALE)) for x, y in points]
        if dashed:
            for a, b in zip(coords[:-1], coords[1:]):
                d=math.dist(a,b)
                for start in range(0, int(d), 18*SCALE):
                    end=min(start+10*SCALE,d)
                    aa=(a[0]+(b[0]-a[0])*start/d,a[1]+(b[1]-a[1])*start/d)
                    bb=(a[0]+(b[0]-a[0])*end/d,a[1]+(b[1]-a[1])*end/d)
                    self.draw.line([aa,bb],fill=self.color(fill),width=width*SCALE)
            else:
                self.draw.line(coords, fill=self.color(fill), width=width*SCALE, joint="curve")
        self.svg.append('<polyline points="'+'.join(f'{x},{y}' for x,y in points)+f'" fill="none" stroke="{self.color(fill)}" stroke-width="{width}" stroke-linejoin="round"'+(' stroke-dasharray="10 8"' if dashed else "")+>')
    def arrow(self, points, fill="teal", width=4):
        self.line(points,fill,width)
        a,b=points[-2:]
        angle=math.atan2(b[1]-a[1],b[0]-a[0]); head=14
        ps=[b,(b[0]-head*math.cos(angle-.5),b[1]-head*math.sin(angle-.5)),(b[0]-head*math.cos(angle+.5),b[1]-head*math.sin(angle+.5))]
        self.draw.polygon([(round(x*SCALE),round(y*SCALE)) for x,y in ps],fill=self.color(fill))
        self.svg.append('<polygon points="'+'.join(f'{x:.2f},{y:.2f}' for x,y in ps)+f'" fill="{self.color(fill)}"/>')

```

```

>')
def circle(self, x, y, r, fill="teal", stroke=None, width=2):
    self.draw.ellipse((round((x-r)*SCALE),round((y-r)*SCALE),round((x+r)*SCALE),round((y+r)*SCALE)),fill=self.color(fill),outline=self.color(stroke) if stroke else None,width=width*SCALE)
5     self.svg.append(f'<circle cx="{x}" cy="{y}" r="{r}" fill="{self.color(fill)}"'+(f' stroke="{self.color(stroke)}" stroke-width="{width}"' if stroke else "")+'>')
def card(self,x,y,w,h,title,lines,accent="teal", details_size=29, title_size=35):
    self.rect(x,y,w,h,"white", "line")
    self.rect(x,y,8,h,accent,radius=4)
10    self.text(x+28,y+24,title,title_size,"navy",bold=True,max_width=w-54)
    self.lines(x+28,y+83,lines,details_size,"muted",gap=13,max_width=w-54)
def label(self,x,y,w,text,fill="teal",bg="teal_bg",size=27):
    self.rect(x,y,w,50,bg,radius=25)
    self.text(x+w/2,y+8,text,size,fill,bold=True,anchor="center",max_width=w-20)
15 def footer(self,source):
    self.line([(55,1040),(1745,1040)],"line",2)
    self.text(56,1058,source,24,"muted",max_width=1430)
    self.text(1744,1058,"EgoSmplx V1.0",24,"navy",anchor="right")
def save(self):
20    ASSETS.mkdir(parents=True,exist_ok=True)
    self.im.save(ASSETS/(self.name+".png"),dpi=(300,300),optimize=True)
    (ASSETS/(self.name+".svg")).write_text("\n".join(self.svg+['</svg>']),encoding='utf-8')
    print(self.name)
def architecture():
25    f=Figure("design_architecture","模块架构与运行边界","配置驱动的逐帧流水线·多模型独立环境·可核验的结果资产")
    f.rect(55,188,1690,115,"blue_bg",radius=18)
    f.text(86,214,"统一控制层",34,"navy",bold=True)
    f.text(374,219,"CLI / 配置加载 / 输入预检 / SHA-256 指纹 / 日志 / 校验后续跑",32,"navy")
30    f.arrow([(900,303),(900,348)])
    f.card(55,358,360,382,"输入与观测",[f"图像、每相机鱼眼标定","EgoSMPLX 初始参数","Sapiens2 关节点与分割","WiLoR / MANO 手部"],"blue")
    f.card(490,358,405,382,"身体与手部校正",[f"刚体 → 19 关节微调","左右手一对一匹配","有界腕点校正","可见前臂轮廓选择"],"teal")
35    f.card(970,358,355,382,"增强与融合",[f"MANO 辅助身体优化","固定形状和骨长","原有有界轮廓接缝","局部谐波位移"],"teal")
    f.card(1400,358,345,382,"验收与导出",[f"独立重载 / OBJ 核验","拓扑 / 硬约束诊断","局部表面穿插检查","报告、网格与结果包"],"orange")
    for a,b in [(415,490),(895,970),(1325,1400)]:f.arrow([(a,548),(b-9,548)])
40    f.rect(55,790,1690,184,"navy",radius=18)
    f.text(86,818,"运行边界",34,"white",bold=True)
    f.text(325,823,"cam3、cam4 逐张处理，各用对应标定；当前不进行双目联合三角化或时序跟踪。",30,"#E3EFF8")
    f.text(86,890,"外部依赖",31,"white",bold=True)
45    f.text(325,893,"EgoSMPLX / Sapiens2 / WiLoR 模型环境、权重与 SMPL-X / MANO 模型资源",30,"#E3EFF8")
    f.footer("依据：README.md、CURRENT_PIPELINE_zh.md 与 pipeline.py。")
    f.save()
def pipeline():
50    f=Figure("design_pipeline","默认处理流程与四阶段结果","MANO 辅助身体优化 → 原有有界轮廓

```

接缝融合")

```

xs=[55,500,945,1390];w=355
top=[("01","输入预检",["帧清单、图像和标定","相机尺寸与摘要核对"]),
      ("02","自动观测",["EgoSMPLX / Sapiens2","WiLoR 手部候选"]),
      ("03","身体两阶段拟合",["全局姿态与平移","19 关节及刚体微调"]),
      ("04","腕点与轮廓校正",["冻结一对一身份","有限候选、可见轮廓"])]
5      bot=[("08","报告与结果包",["四列投影与误差汇总","资产摘要、诊断与 ZIP"]),
          ("07","独立验收",["参数 / 网格 / OBJ 重载","局部穿插单独记录"]),
          ("06","原接缝融合",["鱼眼反投影放置 MANO","局部谐波与形变限制"]),
          ("05","MANO 辅助身体",["身体、腕点、掌根、腕环","冻结轮廓与姿态先验"])]
10      for x,(num,title,lines) in zip(xs,top):
          f.card(x,210,w,238,title,lines,"blue",28,33)
          f.circle(x+30,192,27,"blue");f.text(x+30,173,num,27,"white",bold=True,anchor="center")
          for x,(num,title,lines) in zip(xs,bot):
15              f.card(x,562,w,238,title,lines,"orange" if num in ["07","08"] else "teal",28,33)
              f.circle(x+30,544,27,"orange" if num in ["07","08"] else "teal");f.text(x+30,525,num,27,"white",bold=
True,anchor="center")
              for a,b in [(410,500),(855,945),(1300,1390)]:f.arrow([(a,330),(b-10,330)],"blue")
              f.arrow([(1567,448),(1567,519)],"teal")
20              for a,b in [(1390,1300),(945,855),(500,410)]:f.arrow([(a,683),(b+10,683)],"teal")
              f.rect(55,866,1690,112,"white","line")
              f.text(85,892,"四列展示",31,"navy",bold=True)
              labels=[("raw","原始预测"),("body","Sapiens2 身体"),("guided_body","MANO 辅助身体"),("fused","
最终融合")]
25              for i,(name,label) in enumerate(labels):
                  x=300+i*350
                  f.text(x,882,label,28,"navy",bold=True)
                  f.text(x,930,name+"/"+",25,"muted",mono=True)
                  f.footer("依据: CURRENT_PIPELINES_zh.md; 无匹配手时保留原身体, 跳过新增手部增强。")
30              f.save()
def data_contract():
    f=Figure("design_data_contract","数据接口与结果组织","原始观测 → 标准身体参数 → 混合格网及
    独立诊断")
    f.card(55,204,490,238,"输入清单与观测",["manifest: id / image / calibration","原图像素坐标、逐帧
35    图像摘要","NPZ 参数 / JSON 关节点 / NPY 分割"],"blue",27)
    f.card(640,204,495,238,"标准身体阶段",["raw / body / guided_body","params.npz: SMPL-X 参数及
    派生量","mesh.obj / wireframe.jpg"],"teal",27)
    f.card(1230,204,515,238,"混合格网与报告",["mesh.npz / 同几何 OBJ","recipe、冻结目标和验收 JSO
    N","index.html / summary.json / ZIP"],"orange",27)
40    f.arrow([(545,323),(630,323)])
    f.arrow([(1135,323),(1220,323)])
    f.rect(55,500,1690,478,"white","line")
    f.text(88,527,"最终融合网格的重建依赖",36,"navy",bold=True)
    xleft,xright=88,650
45    rows=[("body_params.npz","最终身体基底参数"),
            ("wilor_predictions.npz","匹配的 MANO 手部预测"),
            ("recipe.json + calibration","冻结身份、配方与相机标定"),
            ("silhouette_targets.json","冻结轮廓目标与顶点支持集"),
            ("segmentation_labels.npy","图像部位标签")]
50    for i,(name,desc) in enumerate(rows):

```

```

y=593+i*66
f.circle(102,y+17,6,"teal")
f.text(126,y,name,28,"navy",mono=True)
f.text(xright,y,desc,29,"muted")
5 f.rect(1240,586,455,305,"teal_bg",radius=18)
f.text(1269,612,"融合结果",33,"teal",bold=True)
f.lines(1269,675,["由身体基底与手部、标定、","冻结目标共同恢复。","单一标准 SMPL-X 参数","
无法恢复最终混合网格。"],28,"navy",gap=16)
f.footer("依据: DATA_FORMAT_zh.md; 三维坐标单位为米, 二维误差使用原始图像像素。")
10 f.save()
def guided_constraints():
    f=Figure("design_guided_constraints","MANO 辅助身体模块: IPO 与约束","在已有身体上连续优化·原接缝算法单独执行")
    f.card(55,205,440,343,"输入 I",["腕点 / 轮廓校正后的身体","Sapiens2 自动身体点","WiLoR 腕点和
15 非拇指 MCP","MANO 16 点腕环与冻结轮廓"],"blue",28)
    f.card(630,205,540,343,"处理 P",["优化 19 个身体关节及刚体","Huber 残差+姿态先验","Adam 最
多 1400 步","约束投影、迭代回退、CPU 复核"],"teal",28)
    f.card(1305,205,440,343,"输出 O",["guided_body 标准参数","网格 / 关节 / 四阶段投影","optimization.json 日志","无可行新增参数则保留原身体"],"orange",28)
20 f.arrow([(495,381),(620,381)])
f.arrow([(1170,381),(1295,381)])
f.rect(55,591,1690,96,"navy",radius=18)
f.text(84,620,"组合目标",31,"white",bold=True)
f.text(309,618,"4×身体 + 4×手腕 + 掌根 + 5×腕环 + 1.5×轮廓 + 先验",34,"white")
25 cards=[("保持固定",["betas、手指、面部、双脚","骨长、匹配身份、轮廓支持集"]),
("几何硬约束",["整体角增量 ≤ 8°","平移 z: 0.05–3 m; 网格 z ≥ 0.02 m"]),
("误差与回退",["身体 RMSE 设上界","连续 200 步无有效改善即停止"])]
for x,(title,lines) in zip([55,630,1205],cards):f.card(x,741,540,227,title,lines,"teal",27,32)
f.footer("依据: CURRENT_PIPELINE_zh.md、reference/guided_body.py; 自动拟合不读取人工标
30 注。")
f.save()
def metrics():
    data=json.loads((ROOT/'docs/current_body_rmse.json').read_text(encoding='utf-8'))
    # Hand values are explicitly published in the current eight-frame table.
35 text=(ROOT/'docs/CURRENT_PIPELINE_zh.md').read_text(encoding='utf-8')
hand=re.search(r"WiLoR 手部 294 点 RMSE \ ( [ \d . ] + ) px \ ( [ \d . ] + ) px",text)
if not hand:raise ValueError('Current hand metric row was not found')
rows=[("身体: 100 点",data['pooled_rmse_px']['before_guided'],data['pooled_rmse_px']['latest']),
("身体: 排除鼻点 94 点",data['excluding_nose_rmse_px']['before_guided'],data['excluding_nose_r
40 mse_px']['latest']),
("肩 / 肘 / 腕: 47 点",data['upper_limb_rmse_px']['before_guided'],data['upper_limb_rmse_px']['lat
est']),
("手部: 294 点",float(hand[1]),float(hand[2]))]
f=Figure("design_metrics","四场景八帧: 自动目标点拟合误差","同批观测、14 只匹配手·合并所
45 有点平方误差计算 RMSE·越低越好")
f.label(55,183,326,"增强前自动基线","blue","blue_bg")
f.label(410,183,402,"增强身体+原接缝","teal","teal_bg")
f.text(1744,194,"单位: px",29,"muted",anchor="right")
f.text(1715,250,"变化",26,"muted",anchor="right")
50 plotx,plotw=535,1050

```

```

for tick in range(0,61,10):
    x=plotx+plotw*tick/60
    f.line([(x,274),(x,854)],"line",2)
    f.text(x,868,str(tick),27,"muted",anchor="center")
5  for i,(label,before,after) in enumerate(rows):
    y=284+i*142
    f.text(55,y+22,label,32,"navy",bold=True,max_width=435)
    f.rect(plotx,y,plotw*before/60,35,"blue",radius=9)
    f.rect(plotx,y+49,plotw*after/60,35,"teal",radius=9)
10  f.text(plotx+plotw*before/60+16,y-2,f'{before:.2f}',29,"blue",bold=True)
    f.text(plotx+plotw*after/60+16,y+47,f'{after:.2f}',29,"teal",bold=True)
    delta=after-before
    f.text(1715,y+20,('+' if delta>0 else "")+f'{delta:.2f}',29,"orange" if delta>0 else "teal",bold=True,anch
or="right")
15  f.rect(55,937,1690,72,"orange_bg",radius=18)
    f.text(86,957,"身体指标下降，手部指标上升；这些指标以 Sapiens2 / WiLoR 自动点为参考，不代表三维精度。",29,"navy")
    f.footer("数据：current_body_rmse.json；手部 294 点见 CURRENT_PIPELINE_zh.md / VALIDATIO
N_zh.md。")
20  f.save()
    def validation():
        data=json.loads((ROOT/'docs/guided_migration_validation.json').read_text(encoding='utf-8'))
        frames=data['frames'];n=len(frames)
        reload_count=sum(bool(r['independent_reload']) for r in frames)
        surface_count=sum(bool(r['local_crossings_passed']) for r in frames)
25  f=Figure("design_validation","八帧验收：重载与局部穿插分别报告","独立重建及 OBJ 一致性通过
，不自动代表表面视觉精度合格")
    f.rect(55,190,817,193,"teal_bg",radius=20)
    f.text(86,220,"独立重载通过",33,"teal",bold=True)
30  f.text(810,229,f'{reload_count}/{n}',78,"teal",bold=True,anchor="right")
    f.text(86,303,"全部网格字段逐值一致，最大顶点差 0 m",27,"navy")
    f.rect(928,190,817,193,"orange_bg",radius=20)
    f.text(959,220,"局部穿插检查通过",33,"orange",bold=True)
    f.text(1683,229,f'{surface_count}/{n}',78,"orange",bold=True,anchor="right")
35  f.text(959,303,"固定前臂 / 腕口支持集，对完整表面检查",27,"navy")
    f.rect(55,432,1690,462,"white","line")
    headers=[('四场景',86),('cam3 重载',665),('cam3 局部',917),('cam4 重载',1199),('cam4 局部',1475)]
    for title,x in headers:f.text(x,458,title,29,"navy",bold=True)
    f.line([(85,517),(1715,517)],"line",2)
40  conditions=[('condition_01','常规姿态'),('condition_02','手物交互'),('condition_03','手物遮挡'),('conditi
on_04','失败病例')]
    for i,(key,label) in enumerate(conditions):
        y=548+i*79
        if i%2==0:f.rect(74,y-7,1651,65,"bg",radius=8)
45  f.text(86,y,label,30,"navy",bold=True)
        f.text(333,y+3,key,25,"muted",mono=True)
        for cam,x1,x2 in [('cam3',726,981),('cam4',1260,1539)]:
            row=next(r for r in frames if r['condition'].startswith(key) and r['camera']==cam)
            for x,passed in [(x1,row['independent_reload']),(x2,row['local_crossings_passed'])]:
50  f.label(x-73,y-1,146,"通过" if passed else "未通过","teal" if passed else "red","teal_bg" if passed

```

```

else "red_bg",26)
    f.rect(55,932,1690,78,"navy",radius=18)
    f.text(87,952,"验收范围: 不检查共面重叠; 不构成全身无碰撞证明; 失败帧保留诊断和实际输出
。",30,"white")
5    f.footer("数据: guided_migration_validation.json; 优化重跑 2 帧, 网络与八帧上游身体拟合未全部
重跑。")
    f.save()
if __name__=='__main__':
    parser = argparse.ArgumentParser(description=__doc__)
10    parser.add_argument("--font-dir", help="Existing font directory; overrides EGOSMPLX_FONT_DIR")
    args = parser.parse_args()
    try:
        configure_fonts(args.font_dir)
    except ValueError as error:
15    parser.error(str(error))
    for draw_figure in [architecture,pipeline,data_contract,guided_constraints,metrics,validation]:
        draw_figure()
文件名: tools/export_copyright.py
功能: Export actual source and illustrated design documents in the reference layout.
20    #!/usr/bin/env python3
    """Export actual source and illustrated design documents in the reference layout."""
    from copyright_layout import main
    if __name__ == '__main__':
        main()
25 文件名: tools/verify_guided_migration.py
功能: Verify the migrated guided-body/original-seam implementation on saved automatic inputs.
    """Verify the migrated guided-body/original-seam implementation on saved automatic inputs.
    Requires a frozen automatic manifest and the 2026-09-29 expected experiment.
    Selected frames rerun the full added optimizer; all frames rebuild and reload
30    final meshes. Neural networks and upstream two-stage fitting are not rerun.
    """

    import argparse
    import os
    from pathlib import Path
35    import shutil
    import sys
    import numpy as np
    sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
    from egosmplx_pipeline.io import read, write, load, sha
40    def main():
        ap = argparse.ArgumentParser(description=__doc__)
        ap.add_argument('--manifest', type=Path, required=True)
        ap.add_argument('--expected', type=Path, required=True)
        ap.add_argument('--output', type=Path, required=True)
45    ap.add_argument('--repo-root', type=Path, required=True)
        ap.add_argument('--config', type=Path, required=True)
        ap.add_argument('--optimize-indices', type=int, nargs='*', default=[0, 6])
        args = ap.parse_args()
        if args.output.exists():
50    raise FileExistsError(args.output)

```

```

args.output.mkdir(parents=True)
os.environ['EGOSMPLX_REPO_ROOT'] = str(args.repo_root.resolve())
os.environ['EGOSMPLX_CONFIG'] = str(args.config.resolve())
os.environ['EGOSMPLX_REFERENCE_ROOT'] = str(args.output.resolve())
5 from egosmplx_pipeline.reference import guided_body as guided
from egosmplx_pipeline.reference.guided_fuse import run as fuse
rows = read(args.manifest)['frames']
records = []
for i, row in enumerate(rows):
10     expected = args.expected / 'frames' / row['condition'] / row['camera']
    destination = guided.frame_dir(row)
    destination.mkdir(parents=True, exist_ok=True)
    # context() verifies the manifest's per-input SHA-256 before using observations.
    guided.context(row)
15     if i in args.optimize_indices:
        guided.fit(row, 1400)
    else:
        shutil.copyfile(expected / 'body_params.npz', destination / 'body_params.npz')
        actual_p, expected_p = load(destination / 'body_params.npz'), load(expected / 'body_params.npz')
20     core = guided.PARAMS + ['transl']
    exact = all(np.array_equal(actual_p[k], expected_p[k]) for k in core)
    if not exact:
        raise AssertionError('Body optimization differs from expected frame ' + row['id'])
    fuse(row)
25     fuse(row, verify=True)
    actual, baseline = load(destination / 'hybrid_mesh.npz'), load(expected / 'hybrid_mesh.npz')
    equality = {k:bool(np.array_equal(actual[k], baseline[k])) for k in baseline if k in actual}
    if set(actual) != set(baseline) or not all(equality.values()):
        raise AssertionError('Fusion differs from expected frame ' + row['id'])
30     record = dict(id=row['id'], condition=row['condition'], camera=row['camera'],
        optimizer_rerun=i in args.optimize_indices, body_parameters_bitwise_equal=exact,
        all_mesh_fields_bitwise_equal=True,
        vertex_max_abs_m=float(np.abs(actual['vertices_cam']-baseline['vertices_cam']).max()),
        local_crossings_passed=read(destination / 'geometry_validation.json')['all_local_crossing_che
35     cks_passed'],
        independent_reload=read(destination / 'reload_validation.json')['passed'])
    records.append(record)
    write(args.output / 'progress.json', dict(frames=records))
    print('MIGRATION_VERIFIED', row['id'], flush=True)
40     write(args.output / 'migration_validation.json', dict(frame_count=len(records),
        manifest_sha256=sha(args.manifest), optimizer_rerun_count=len(args.optimize_indices),
        all_body_parameters_bitwise_equal=all(r['body_parameters_bitwise_equal'] for r in records),
        all_mesh_fields_bitwise_equal=True, local_surface_pass_count=sum(r['local_crossings_passed'] for
r in records),
45     scope='Added optimizer rerun on selected frozen automatic inputs; all final meshes independently re
built and reloaded; networks and upstream fitting not rerun', frames=records))
if __name__ == '__main__':
    main()

```